



EE5110/EE6110

**Special Topics in Automation and Control
Segment B**

From Frames to Events:
Theory and Applications of Event-based Vision

Jiang Rui, Ph.D., BEng
rui.j@nus.edu.sg

Adjunct Lecturer
Department of Electrical and Computer Engineering

Course Structure

- **Lecture 1 – Event-based Vision System**
 - Conventional machine vision systems
 - Event-based sensors and cameras
 - Event camera calibration
 - Event camera autofocus
 - Event data simulation and dataset
 - Event data representation and compression

Course Structure

- **Lecture 2 – Algorithms and Applications**

- Detection and tracking
- Optical flow estimation
- Pose estimation and SLAM
- Image reconstruction

Supplementary Materials for Self-Learning

- Motion segmentation
- Motion recognition

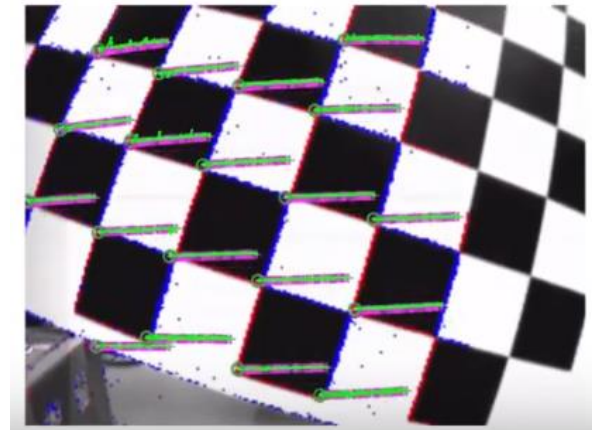
Learning Objectives

- In this lecture, you are expected to:
 1. Describe and explain the basic concepts and methods in event-based computer vision algorithms and applications;
 2. Understand the pros and cons by comparing the event-based methods with the frame-based counterparts.

Detection and Tracking

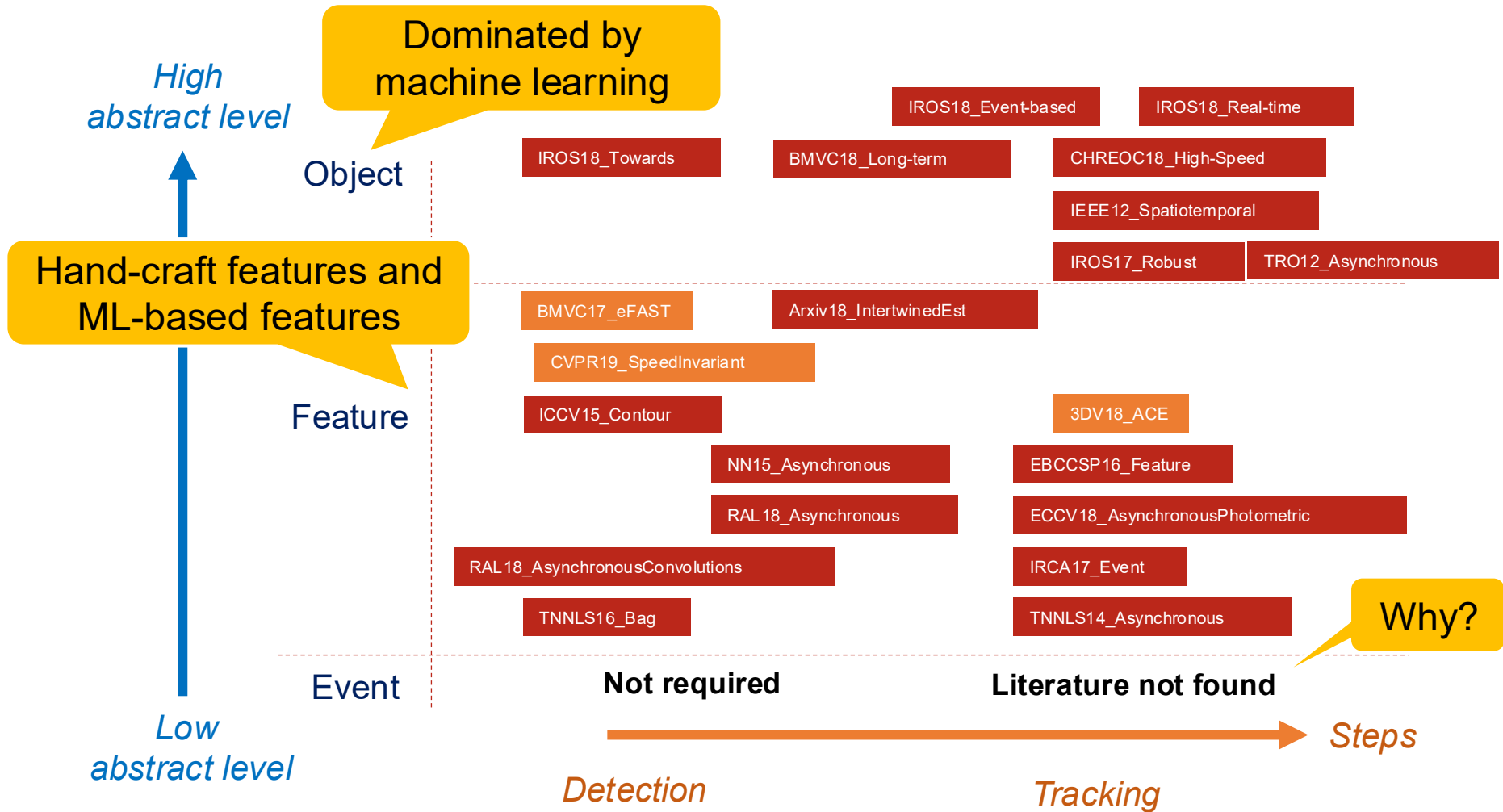
- What is detection/tracking?
 - **Feature detection:**
Extract those robust, representative pixels ("features").
 - **Object detection:**
Identify and find the interested objects.
- **Feature/object tracking:**
Establish correspondences between features or objects.

Which local patch is better for a feature?



Event-based Detection and Tracking

- Detection and tracking at different abstract levels



*Above references are for demonstration only, and do not reflect the recent progress of this field.

Event-based Detection and Tracking

- References' details

-  3DV18_ACE_An Efficient Asynchronous Corner Tracker for Event Cameras
-  Arxiv18_Event-Based Features Selection and Tracking from Intertwined Estimation of Velocity and Generative Contours
-  BMVC17_Fast Event-based Corner Detection
-  BMVC18_Long-term object tracking with a moving event camera
-  CHREOC18_High-Speed Object Tracking with Dynamic Vision Sensor
-  CVPR19_Speed Invariant Time Surface for Learning to Detect Corner Points with Event-Based Cameras
-  EBCCSP16_Feature detection and tracking with the dynamic and active-pixel vision sensor (DAVIS)
-  ECCV18_Asynchronous_Photometric Feature Tracking using Events and Frames
-  ICCV15_Contour Detection and Characterization for Asynchronous Event Sensors
-  IEEE12_Spatiotemporal Multiple Persons Tracking Using Dynamic Vision Sensor
-  IRCA17_Event-based Feature Tracking with Probabilistic Data Association
-  IROS17_Robust Visual Tracking with a Freely-moving Event Camera
-  IROS18_Event-based Moving Object Detection and Tracking
-  IROS18_Real-time clustering and multi-target tracking using event-based sensors
-  IROS18_Towards Event-Driven Object Detection with Off-the-Shelf Deep Learning
-  NN15_Asynchronous event-based corner detection and matching
-  RAL18_Asynchronous Corner Detection and Tracking for Event Cameras in Real Time
-  RAL18_Asynchronous Spatial Image Convolutions for Event Cameras
-  TNNLS14_Asynchronous Event-Based Multi-kernel Algorithm for High Speed Visual Features Tracking
-  TNNLS16_Bag of Events_An Efficient Probability-Based Feature Extraction Method for AER Image Sensors
-  TRO12_Asynchronous Event-based Visual Shape Tracking for Stable Haptic Feedback in Microrobotics

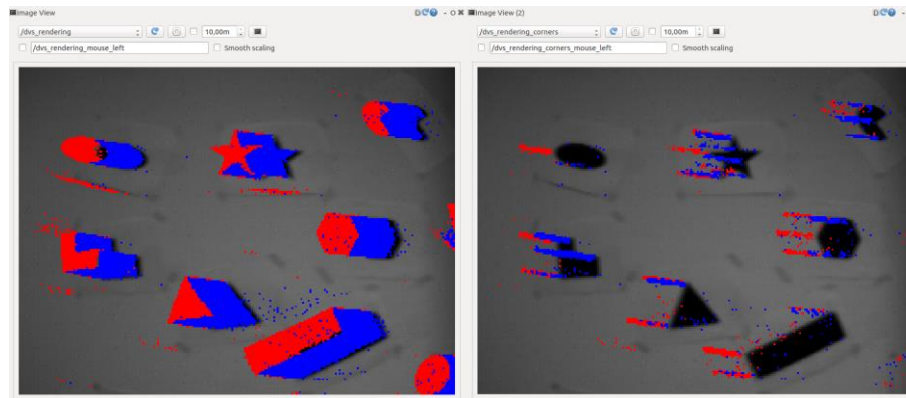
*Above references are for demonstration only, and do not reflect the recent progress of this field.

Paper: Fast Event-based Corner Detection

- “Event camera generates much less data than conventional camera.”

Is this always true?

- Not always! **Sometimes, more data from event camera...**
- That’s why we need feature detection.
- Paper highlights:
 - A fast method for corner detection in an event stream
 - Efficient computing using only comparison operations

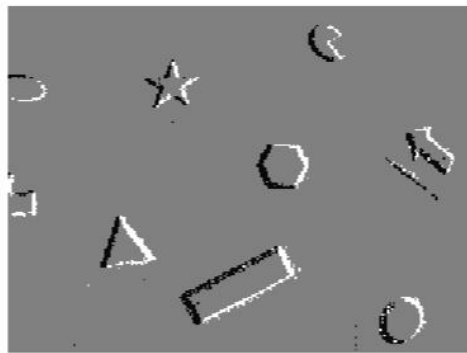


Reduce the event stream to a **corner event stream!**

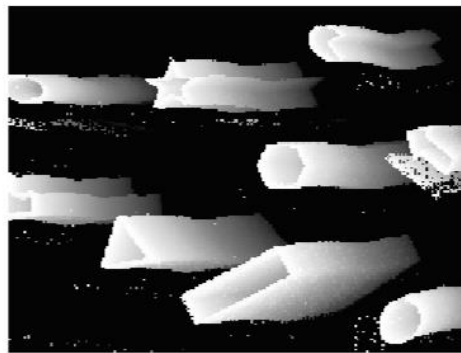
[PowerPoint Presentation \(uzh.ch\)](#)
[uzh-rpg/rpg_corner_events: Fast Event-based Corner Detection \(github.com\)](#)

Paper: Fast Event-based Corner Detection

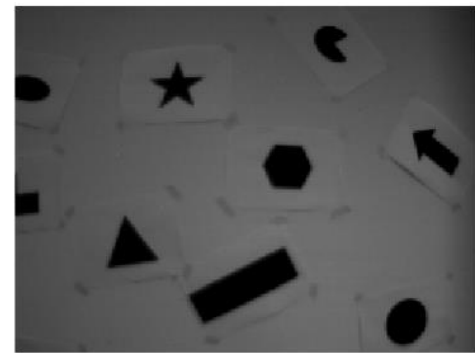
- Approach
 - Data representation: Surface of Active Events (a map with the timestamp of the latest event at each pixel)
 - Analyze timestamp distribution around current event
 - Detect corners by **searching for contiguous pixels with higher timestamps than the rest**



(a) Events over the last 33 ms. Positive events (white) and negative events (black)



(b) Surface of Active Events (SAE). Brightness represents time, from past (dark) to present (bright)



(c) Intensity Image (absolute brightness $I(x,y)$ in (1))

Figure 2: Signal used for corner detection: the Surface of Active Events (SAE).

Paper: Fast Event-based Corner Detection

- Idea: The corner will create local SAE pattern as shown in the right figure.

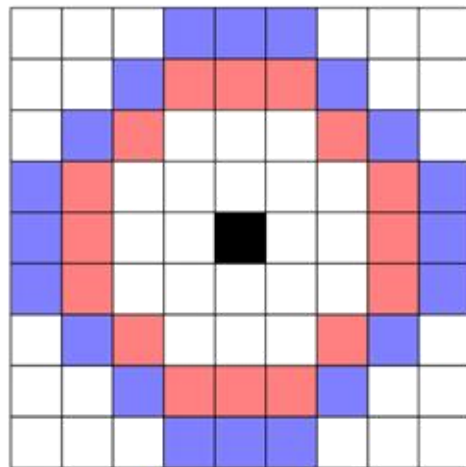
How to detect the pattern?

- Along each circle, the algorithm searches for segments of contiguous pixels (arcs) that are higher (i.e., they have a more recent timestamp) than all other pixels on the circle.

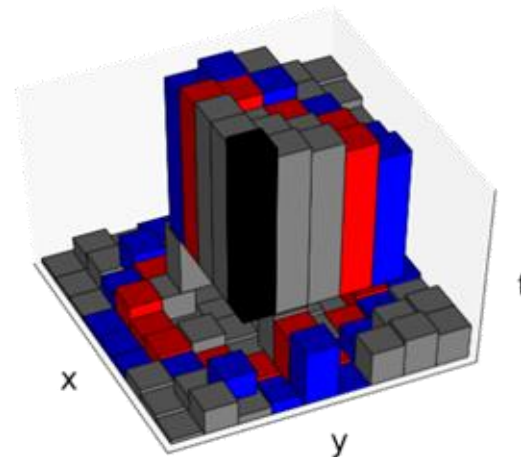
Outer circle: search for a segment of length 4-8

Inner circle: search for a segment of length 3-6

The center is a corner event!



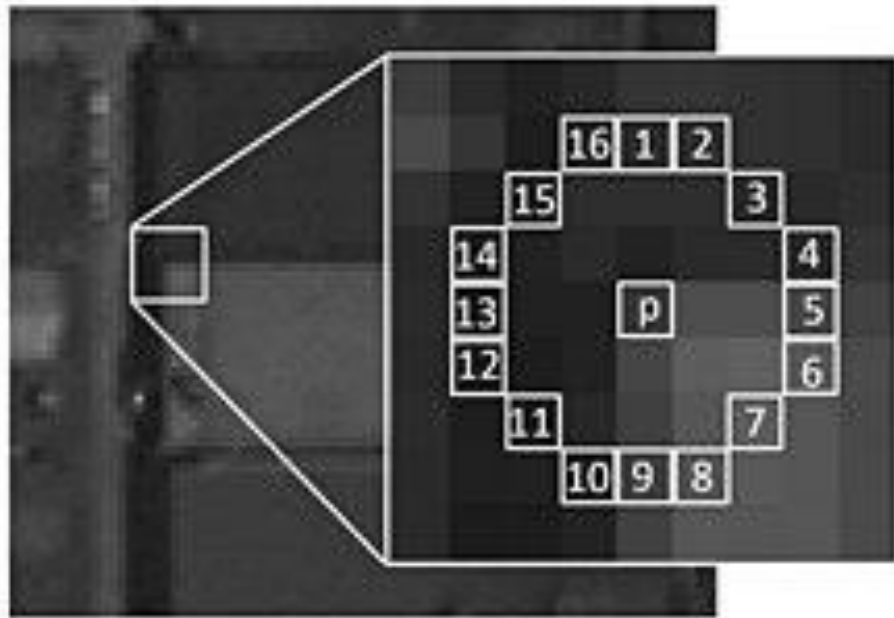
(a) Circles of radius 3 and 4 pixels



(b) Visualization of Surface of Active Events (SAE)

Paper: Fast Event-based Corner Detection

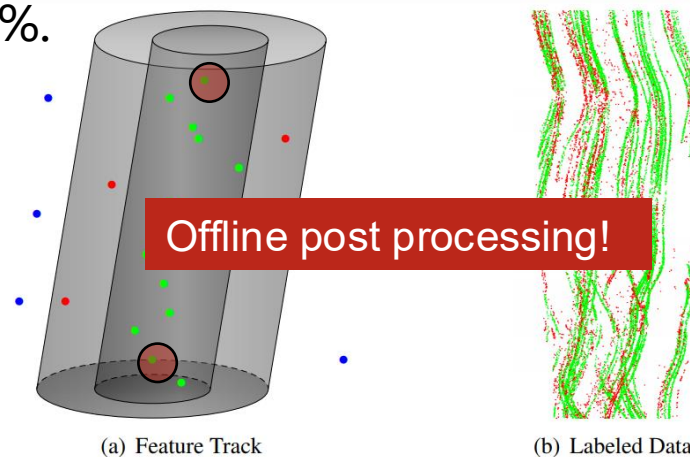
- Idea similar to “FAST” feature in conventional computer vision.



So it is also called “eFAST”!

Paper: Fast Event-based Corner Detection

- Find feature tracks
 - A feature track is composed of an inner and an outer oblique cylinder in space-time.
 - Exhaustively search for feature tracks by creating hypotheses **using two corner events** and checking whether there are **enough corner events in the inner cylinder** and **few corner events in the outer cylinder**.
 - Use 3 and 5 pixels for the inner and outer radius, respectively.
 - Condition: Minimum of 30 inner events, and maximum ratio of outer-to-inner events of 25%.



Paper: Fast Event-based Corner Detection

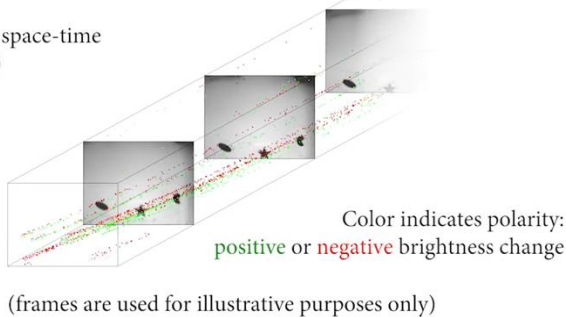
- How fast/accurate is eFAST corner detection?

event
reduction rate

success rate of
corner tracking

Corner Event Stream

Real data in space-time
(10x slower)



Texture	Dataset	Harris [17]		Ours	
		Red.	FT	Red.	FT
low	shapes_rotation	92.7	74.1	88.9	74.7
	shapes_translation	91.7	78.3	87.8	77.5
	shapes_6dof	90.6	77.1	87.0	76.8
medium	dynamic_rotation	95.1	53.3	96.4	46.4
	dynamic_translation	95.3	62.1	96.7	52.1
	dynamic_6dof	95.4	55.9	96.4	49.4
high	poster_rotation	92.6	35.3	95.7	30.5
	poster_translation	92.3	39.5	95.8	35.9
	poster_6dof	92.4	35.3	95.6	32.1
high	boxes_rotation	92.1	32.9	96.7	25.2
	boxes_translation	92.4	37.0	96.7	30.5
	boxes_6dof	92.7	34.4	96.8	26.7

Table 1: Performance of Harris and our detector expressed as reduction rate (Red.) of the event stream and matched Feature Tracks (FT). Values are given in percentages.

Corner tracking fails in high texture environments. Why?

No “description” for each feature’s uniqueness.

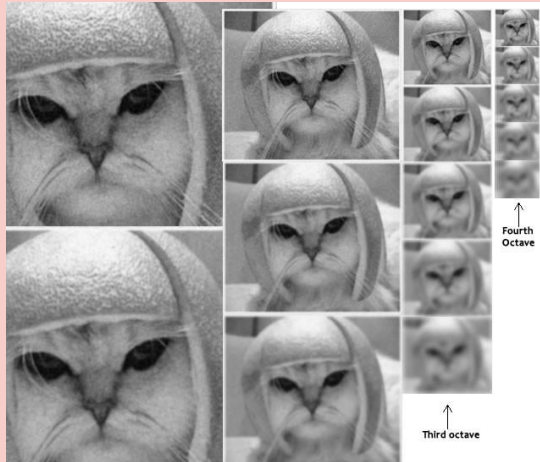
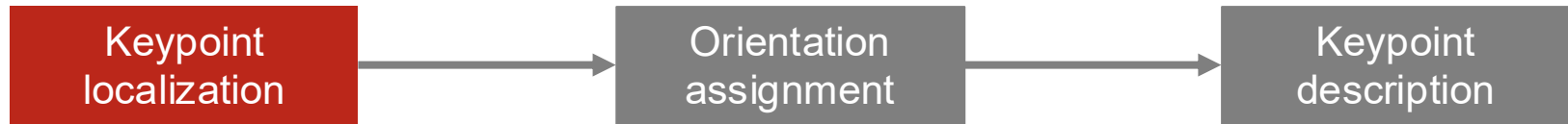
Method	Time per event [μs]	Max. event rate [c/s]
Harris [17]	11.6	86,230
Ours	0.78	1,275,000

Table 2: Runtime comparison per event and corresponding maximum event rate.

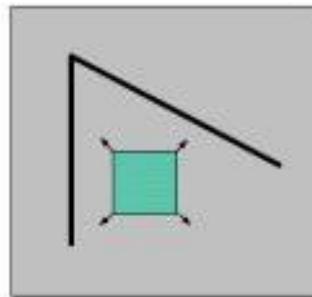
Recap: Feature Detection and Description

- In conventional computer vision, a feature usually comes with a **descriptor**.

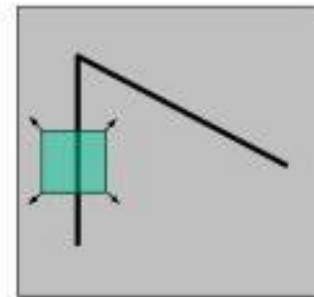
Step 1: localize the feature



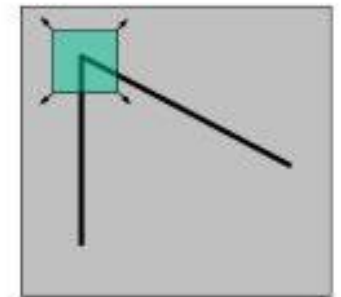
Create a scale space to look for possible keypoints at different scales, and ...



"flat" region:
no change in all
directions



"edge":
no change along the
edge direction

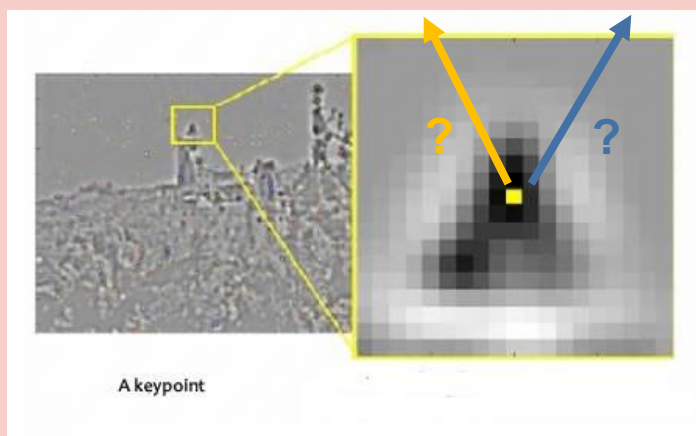
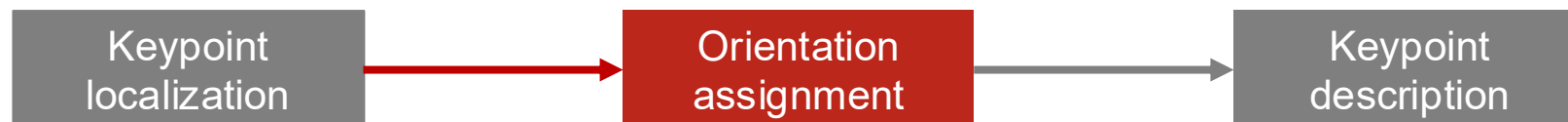


"corner":
significant change in
all directions

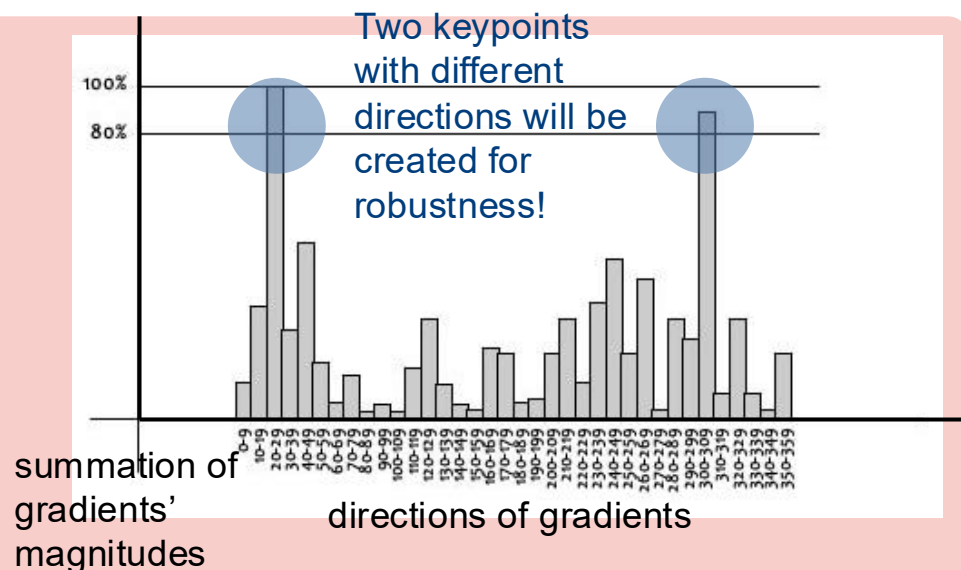
Check the gradients to select "corners" and reject "edges".

Recap: Feature Detection and Description

- **Step 2: find the orientation of the feature**



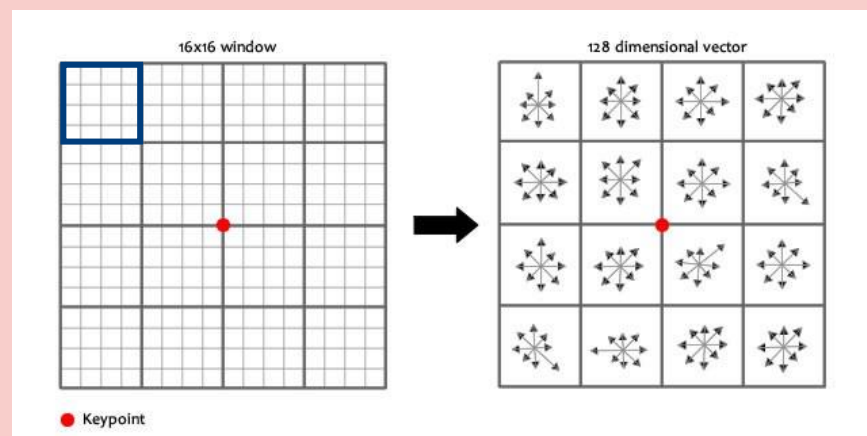
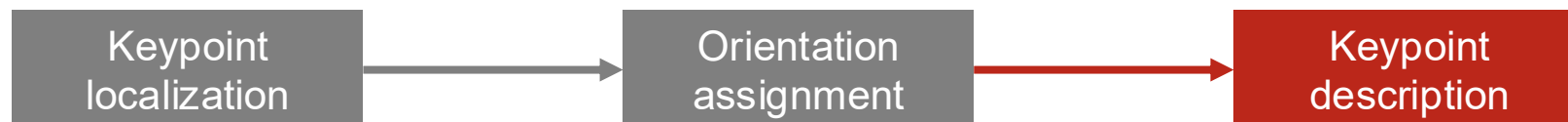
We want to find the orientation of the feature. Take a small patch near the feature point, and ...



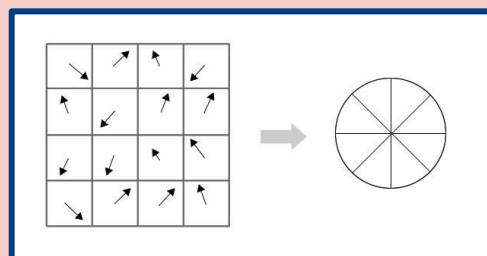
Compute gradients for all neighboring pixels, and create a histogram of gradient directions.

Recap: Feature Detection and Description

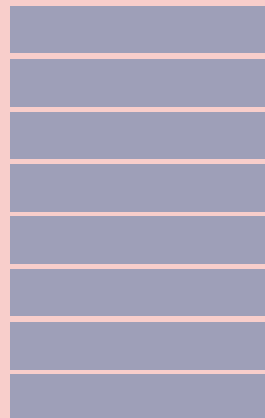
- **Step 3: compute the descriptor of the feature**



For each sub-block, 8 bin orientation histogram is created.



feature vector



A 16x16 window around the keypoint is taken.
It is divided into 16 sub-blocks of 4x4 size.

We will have 128 bin values
to form a **128-dimensional
feature vector**.

Recap: Feature Detection and Description

- We have just discussed **SIFT (Scale Invariant Feature Transform)**
 - **Scale invariance:**
In the 1st step, we extract feature candidates in different scales.
 - **Rotation invariance:**
In the 2nd step, direction of the feature is obtained, so that the descriptor contains **relative rotation angle**, which is compared to the direction of the feature, not the direction of the image.
 - **Translation invariance:**
Obvious.
 - **Illumination invariance:**
Normalization on gradient histogram is used to reduce the influence from lighting conditions and illuminance.

Paper: DART: Distribution Aware Retinal Transform for Event-based Cameras

- Highlights
 - Introduced a generic **visual descriptor** “DART”, that encodes the structural context using log distance-polar grids for event cameras
 - DART is **robust to scale, rotation, and view-point variations**.

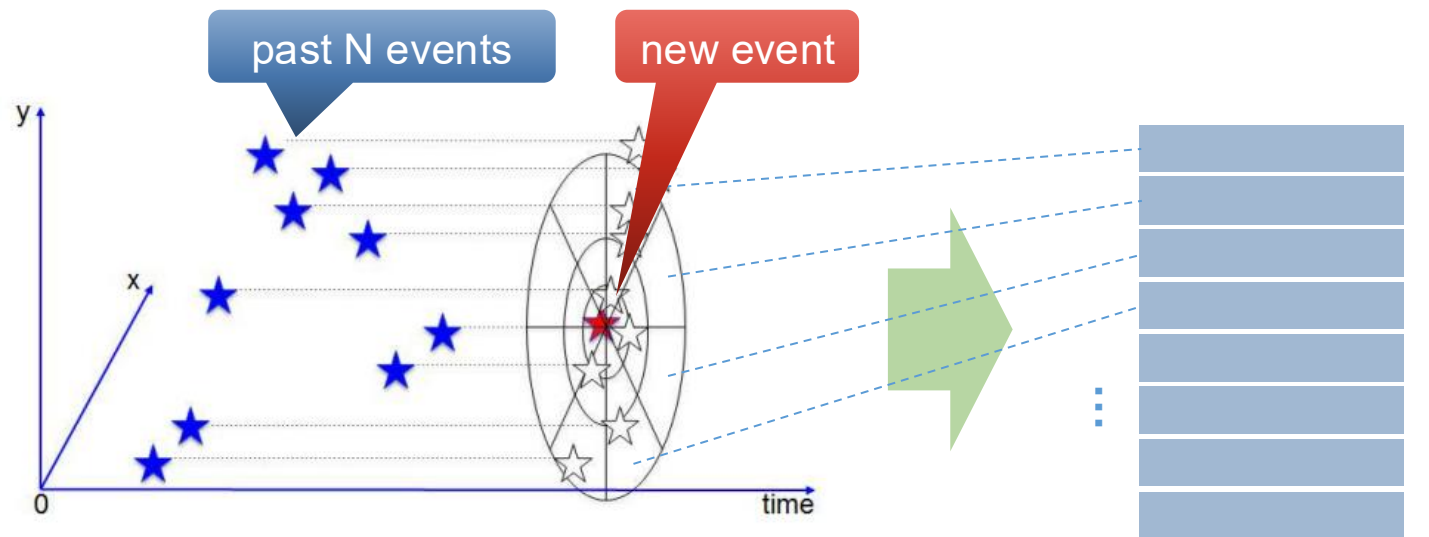
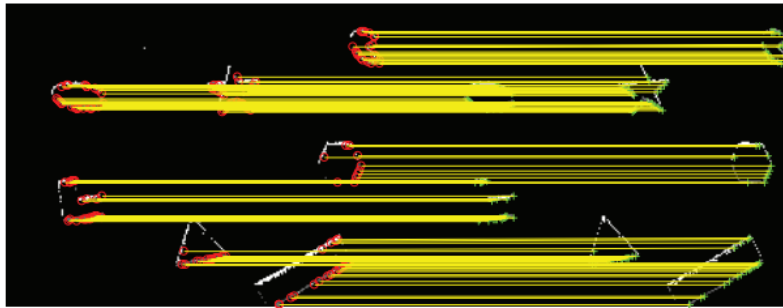


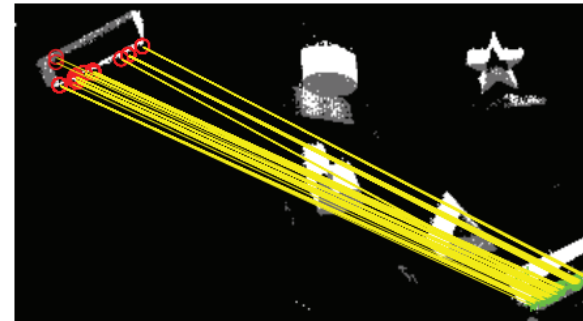
Fig. 1. The most recent event is indicated in red and the previous events are indicated in blue. White stars indicate the position of previous events mapped onto the log-polar bins at the current time.

Paper: DART: Distribution Aware Retinal Transform for Event-based Cameras

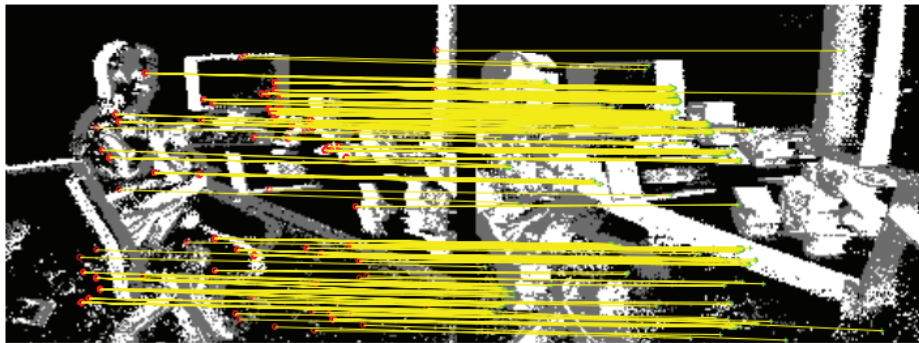
- Feature matching results



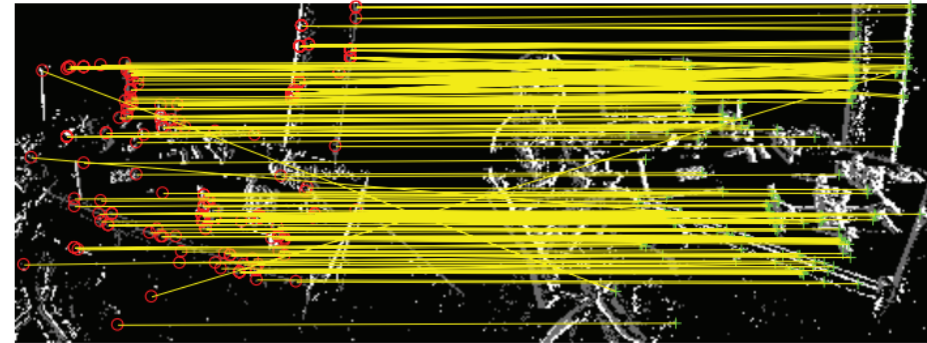
(a) Shapes data of temporally close sequences.



(b) Shapes data of temporally far sequences with occlusion.



(c) Indoor scene with different camera motion speeds.



(d) Indoor scene with scale and rotation changes.

Red circles indicate the features in the left time-slice and the green crosses indicate the corresponding match in the right time-slice connected by yellow lines.

Paper: Data-driven Feature Tracking for Event Cameras

- CVPR 2023 award candidates (0.13% of submitted paper)

Data-driven Feature Tracking for Event Cameras

Nico Messikommer*, Carter Fang*, Mathias Gehrig, Davide Scaramuzza

Source Code: https://github.com/uzh-rpg/deep_ev_tracker



**University of
Zurich** ^{UZH}

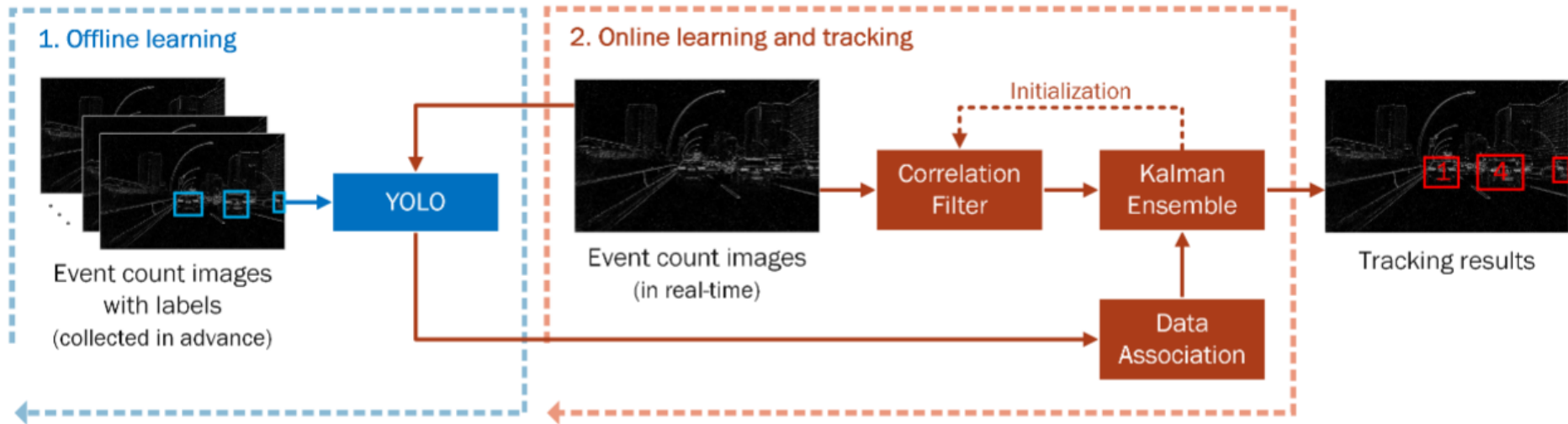


Paper: Object Tracking on Event Cameras with Offline-Online Learning

- We have just discussed feature detection.
What about object-level detection and tracking?
- Possible ways for object detection/tracking based on event cameras
 - **Detect and track objects on event frames** – straightforward and easy
 - Detect and track object based on point clouds
- Highlights
 - Contains YOLO-Based **offline-trained** deep detector + Correlation Filter (CF)-based **online-trained** tracker, which complements each other;
 - A **real-time fusion scheme** designed based on the Kalman filter;
 - It is verified by experiments that: **event count images** contains enough information for object detection and tracking.

Paper: Object Tracking on Event Cameras with Offline-Online Learning

- Object tracking framework

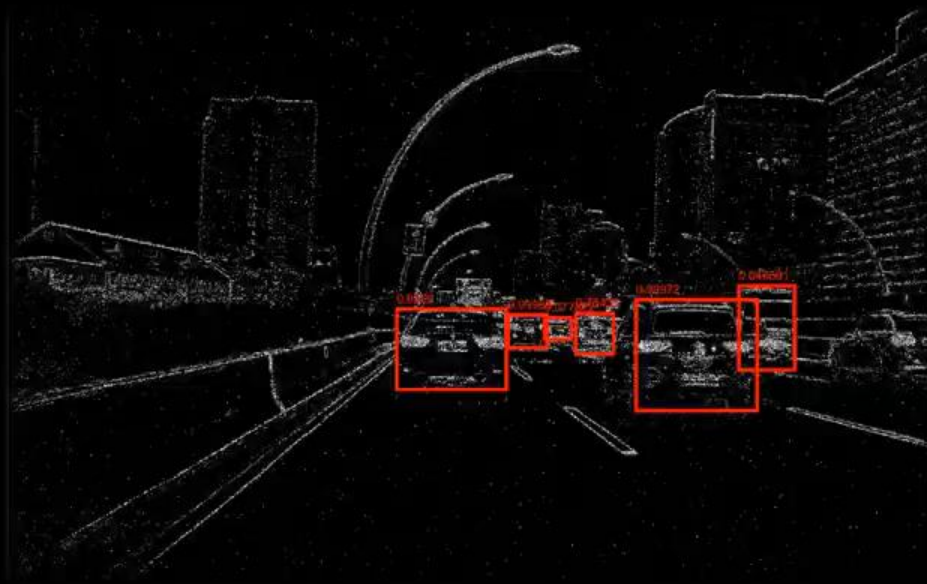


- The 1st stage needs to be completed off-line before tracking such that a YOLO detector is trained;
- The correlation filter, which tracks single object, is initialized/reinitialized using results from the YOLO detector.
- The Kalman filter fuses measurements from both the detector and the tracker to achieve multi-object tracking.

(PDF) Object Tracking on Event Cameras with Offline-Online Learning (researchgate.net)

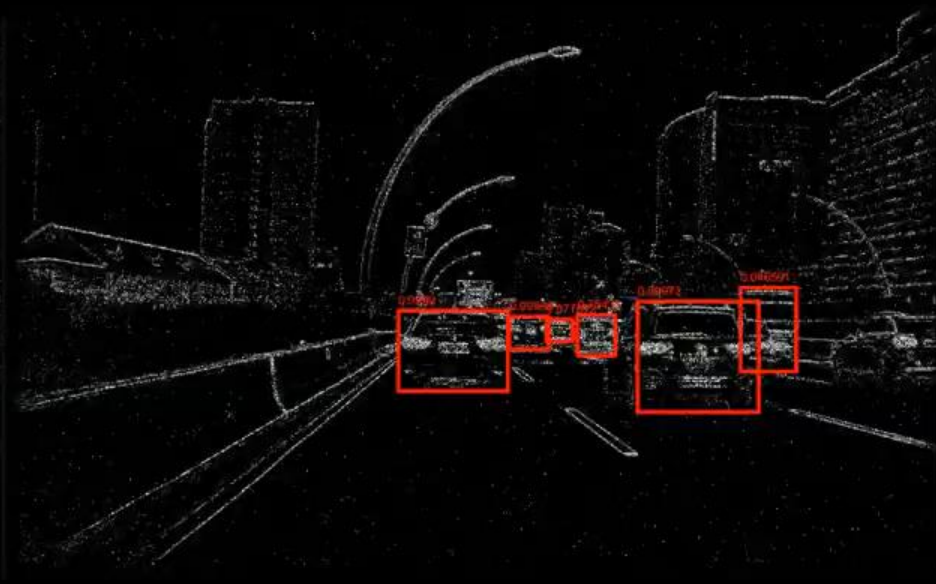
Paper: Object Tracking on Event Cameras with Offline-Online Learning

Object Tracking on Event Cameras with Offline-Online Learning Urban Scenario



Partial Configuration

without CF, tracking at 459 fps

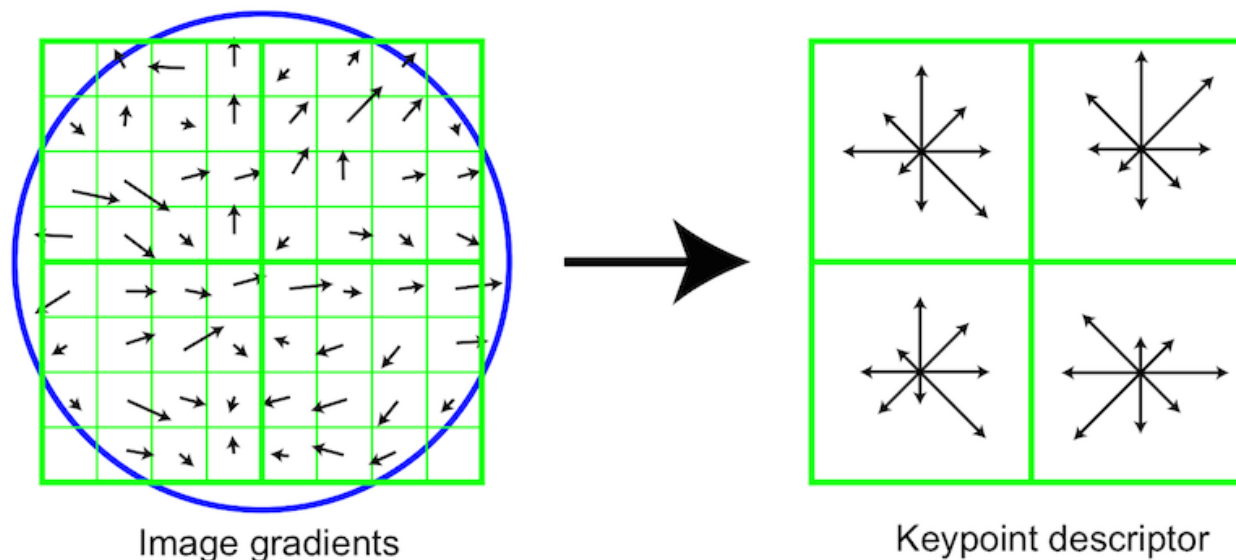


Full Configuration

with offline-online tracker, at ~50 fps

History and Future for Description and Detection

- **Manually-crafted** features/descriptors v.s. **Learned** features/descriptors
 - **The rise of the local feature descriptors: ~1995 to ~2000**
e.g., SIFT (1999, [David Lowe](#) @ University of British Columbia)



From comparing raw data (intensities) to comparing features!

History and Future for Description and Detection

- **The modern dataset: ~2000 to ~2005**

Geometric problems (e.g., homograph estimation, ground-plane estimation, robotic vision, SfM) greatly benefited from robust image features such as SIFT.

Images were going online. Datasets were being created.
e.g., Caltech-101 (2003, Fei-Fei Li, et al. @ Caltech)



[From feature descriptors to deep learning: 20 years of computer vision Eyes of Things](#)
[Introduction to SIFT \(Scale Invariant Feature Transform\) | by Deepanshu Tyagi | Data Breach | Medium](#)

History and Future for Description and Detection

- **Bins, Grids, and Visual Words (aka Machine Learning meets descriptors): ~2000 to ~2005**

Raw SIFT: not powerful enough to provide object-level metric for matching

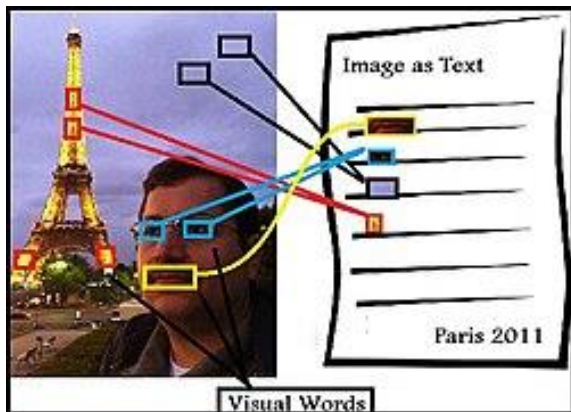
Statistical way: To combine more features and descriptors and compare the **feature set** instead of a single feature!

- e.g., Visual words: *text – image* analogy

Lower
abstract level



Higher
abstract level



+

Classifier (e.g., SVM)

=

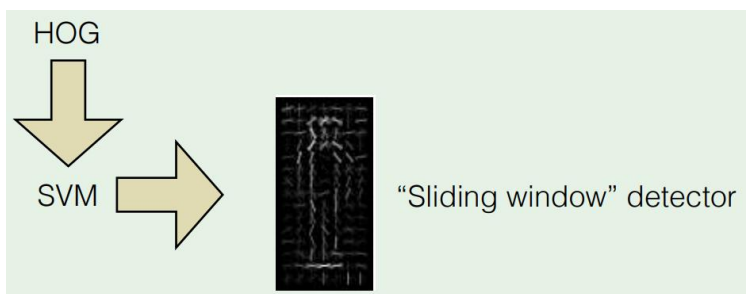
Higher-level detection

History and Future for Description and Detection

- **Object Templates (aka the reign of HOG and DPM): ~2005 to ~2010**

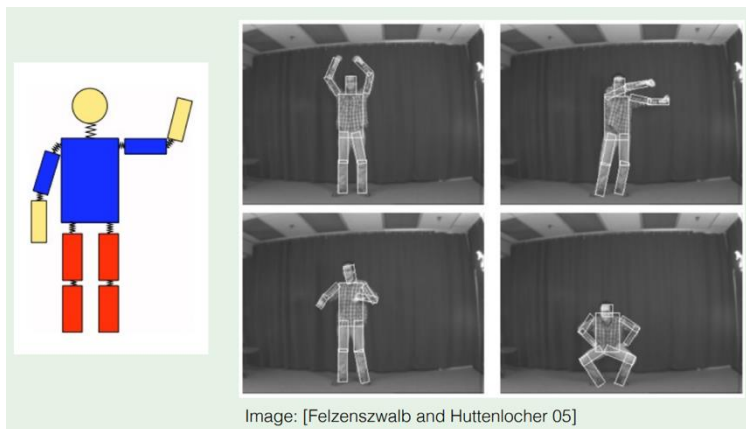
e.g., HOG: Histogram of oriented gradients

describe local object appearance and shape by the **distribution of intensity gradients** or edge directions.



HOG-based human detection

Based on HOG, later we have
DPM: **Deformable Part Model**.



DPM-based human detection

Idea: To separate object into parts
and make them movable!

History and Future for Description and Detection

- **Big data, Convolutional Neural Networks and the promise of Deep Learning: ~2010 to ~2015 (until present)**

Bigger and bigger data!

+

Learning algorithms (since 1980s)

+

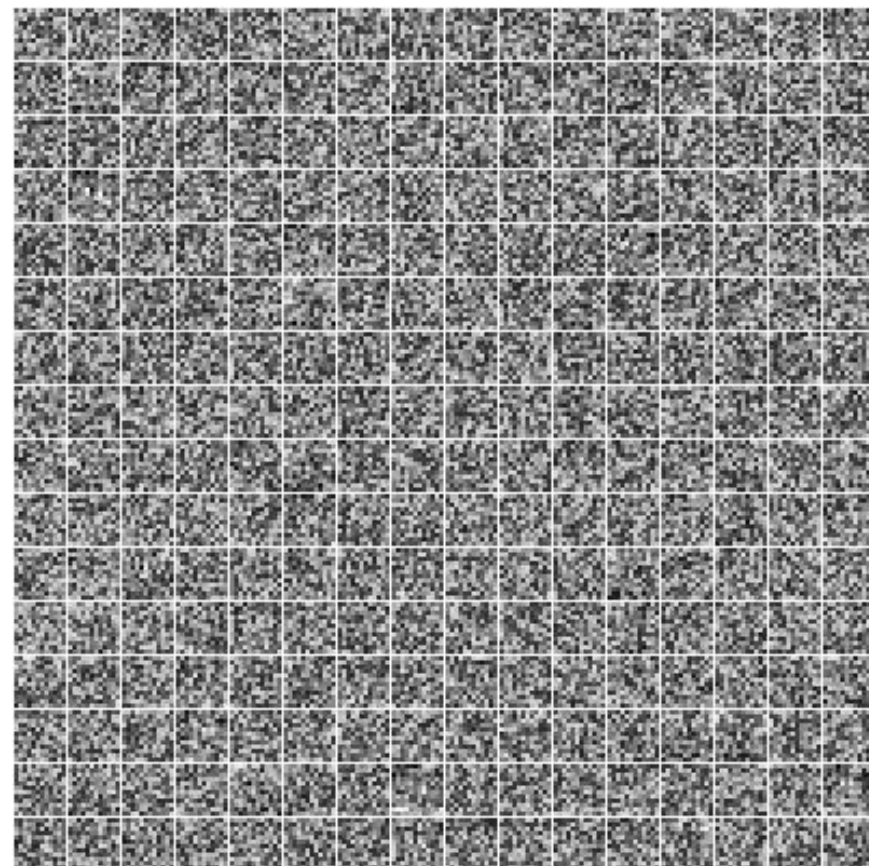
Hardware (GPU)

=

**Simplified feature engineering,
better performance...**

What about for event-based vision?

Difficulties exist due to
fundamentally different data type
compared to conventional cameras!



iteration no 0

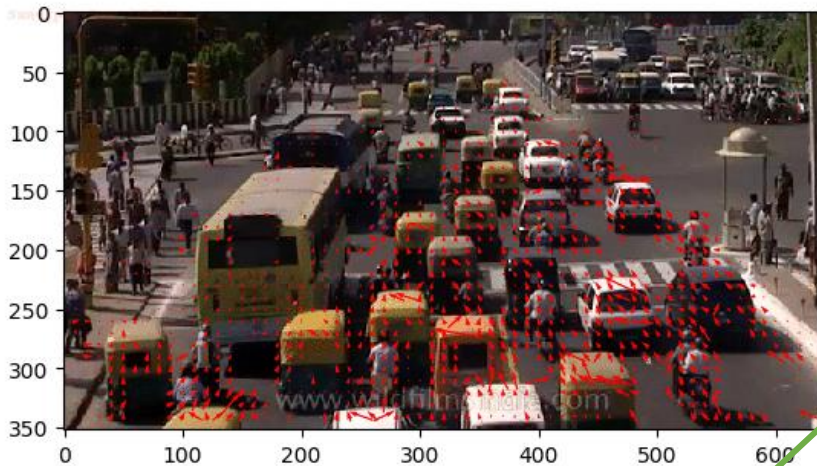
["CBL, Research Projects, Computational and Biological Learning Lab, Courant Institute, NYU"](#)

Summary – Detection and tracking

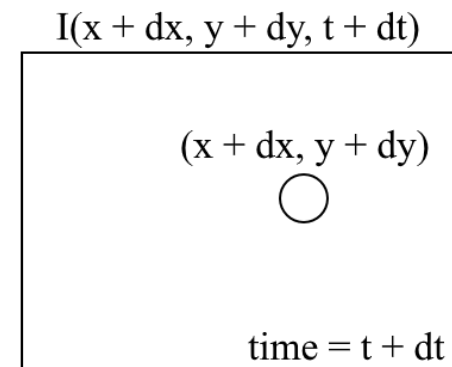
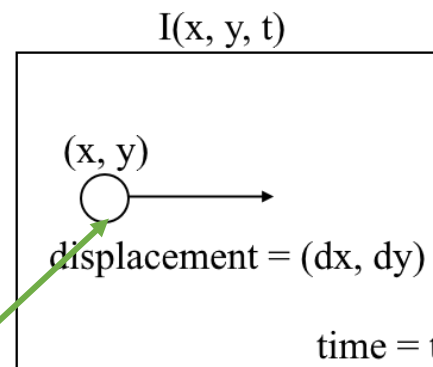
- Relation between detection and tracking
- Different abstraction levels of detection and tracking:
 - Event level → feature level → object level
- Why we need event-based feature detection?
 - To reduce the data rate, aiming at fast event data processing
- Feature detection and description
 - Descriptors required to “identify” features
 - From hand crafted to learned features
- Object-level detection and tracking
- Examples and papers

Optical Flow Estimation

- **Optical flow field vs. motion field**
 - Motion field reflects **object's real motion** in 3-D space
 - Optical flow field is the **distribution of apparent velocities** of movement of **brightness pattern** in an image (2-D space)



“brightness
pattern”



Is the “brightness pattern” always referring to the same physical point in 3-D space?

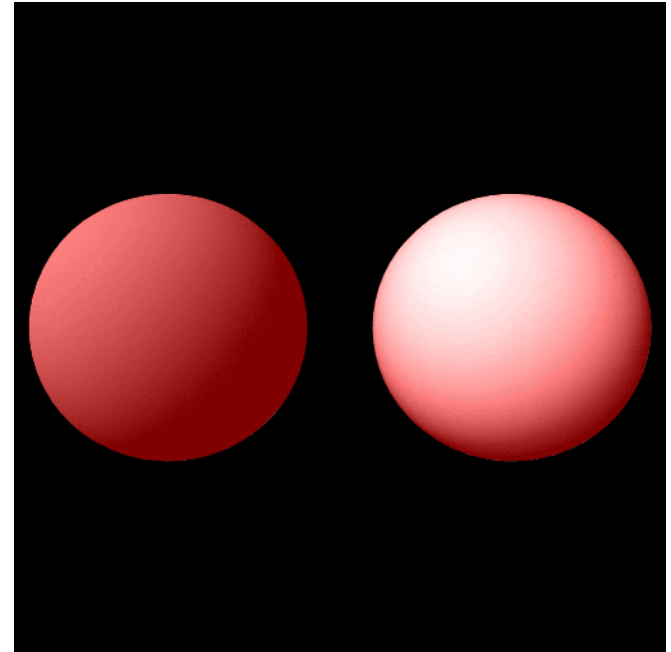
Optical Flow Estimation

- Does the following scenario exist?
 - There is no motion field or optical flow;
 - There is both motion field and optical flow;
 - There is motion field but no optical flow;
 - There is optical flow but no motion field.

From **motion field** to **optical flow field**:

Information loss may be due to

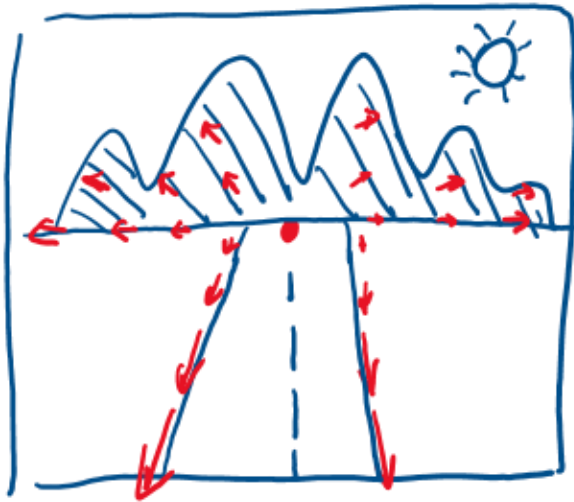
1. 3D to 2D projection
2. Non-informative texture on objects



Can you tell whether it is because of the motion of the sphere, or the light source?

Information in Optical Flow

- Motion information: translation and rotation



We always want to recover motion (2D/3D)!

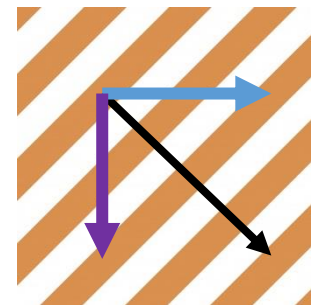
Information in Optical Flow

- Why motion recovery not easy?

- Flow depending on scene/background, but **scene unknown**
→ usually recover motion together with scene (reconstruction)

- Ego-motion coupled with translation and rotation, **different motion may lead to same flow**

- internal measurements (IMU, other sensors)
- build theoretically-robust model for motion and scene



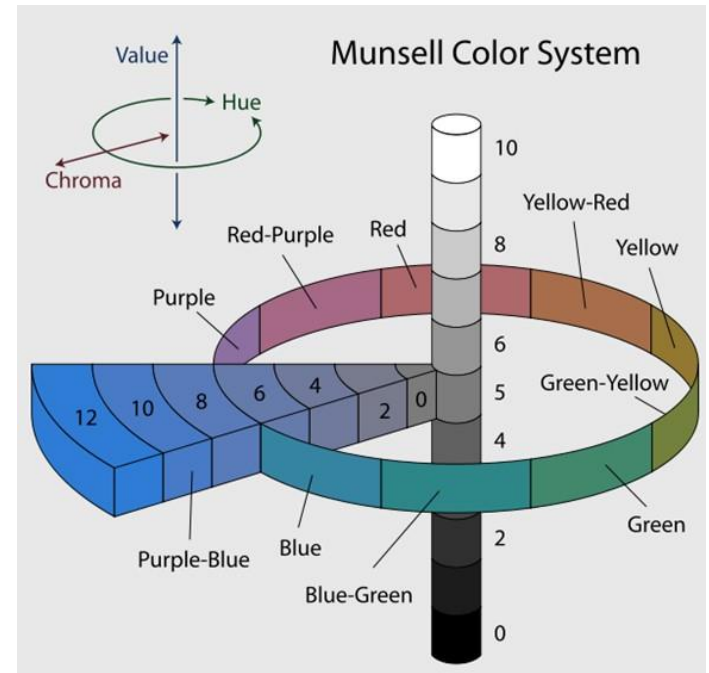
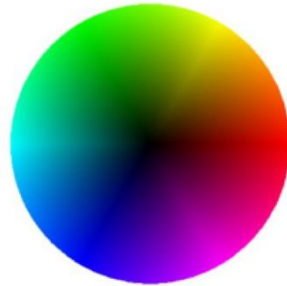
- 3-D to 2-D projection, **irreversible information loss**

- camera needs to move, observe from multi-angles to reconstruct 3-D

**Estimating motion flow is ideal but difficult.
As a compromise, estimating optical flow is the first step.**

Visualization of Optical Flow

- HSV instead of RGB representation
 - Hue = direction
 - Saturation/Chroma set to max
 - Value/brightness = magnitude



Frame-based Flow Estimation

- The problem formulation:

Optimization of a global energy function

$$E_{\text{Global}} = E_{\text{Data}} + \lambda E_{\text{Prior}}.$$

- **Data term:**

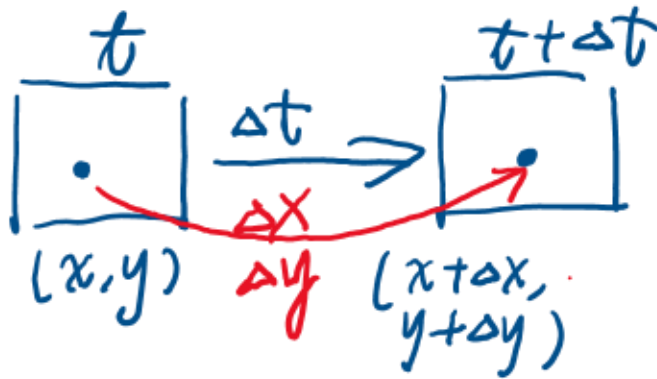
describes how consistent the optical flow is with the input images;

- **Prior term:**

it favors certain flow fields over others (e.g. smoothly varying flow fields).

Flow Estimation: Data Term

- Brightness consistency → **optical flow constraint equation**



$$I(x, y, t) = I(x + \Delta x, y + \Delta y, t + \Delta t)$$

$$= I(x, y, t) + \frac{\partial I}{\partial x} \Delta x + \frac{\partial I}{\partial y} \Delta y + \frac{\partial I}{\partial t} \Delta t + \text{H.o.T.}$$

$$\Rightarrow \frac{\partial I}{\partial x} \Delta x + \frac{\partial I}{\partial y} \Delta y + \frac{\partial I}{\partial t} \Delta t = 0$$

$$\frac{\partial I}{\partial x} \frac{\Delta x}{\Delta t} + \frac{\partial I}{\partial y} \frac{\Delta y}{\Delta t} + \frac{\partial I}{\partial t} = 0$$

$$\frac{\partial I}{\partial x} V_x + \frac{\partial I}{\partial y} V_y + \frac{\partial I}{\partial t} = 0$$

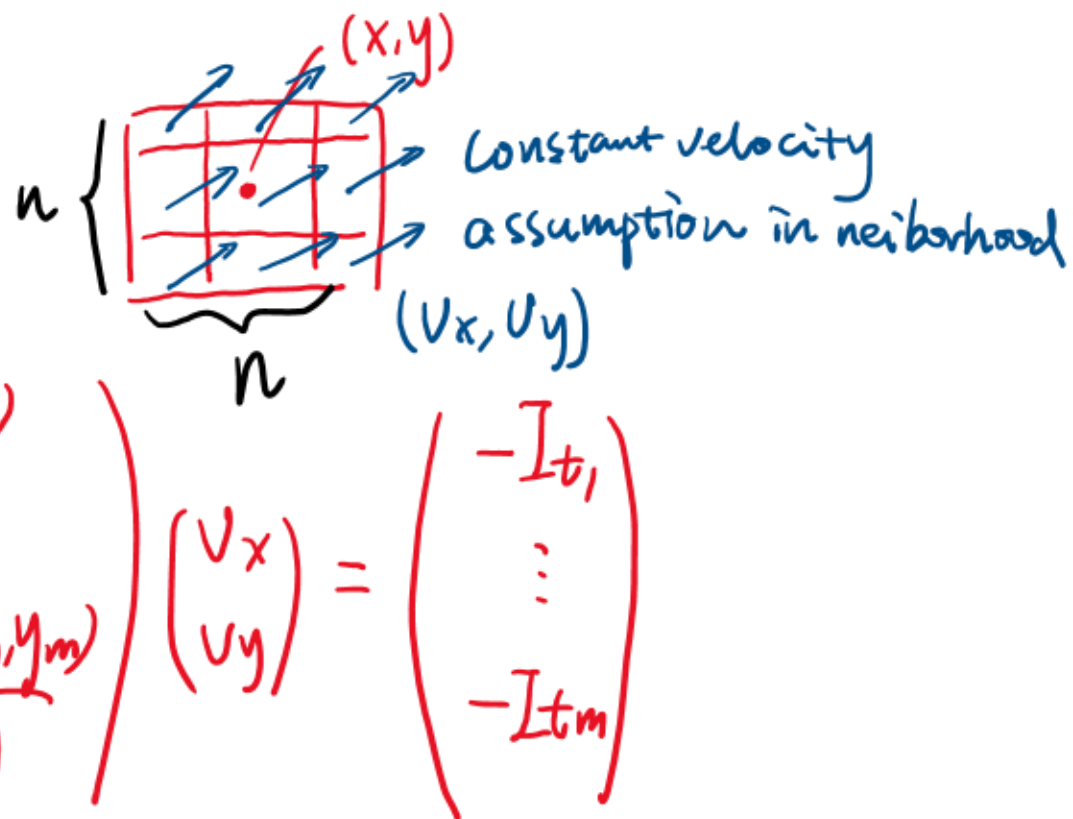
$$I_x V_x + I_y V_y = -I_t$$

Above equation leads to one error per pixel...

You have two unknown variables. You need to find more equations.

Flow Estimation: Data Term

- Solve optical flow **locally** (**Lucas-Kanade** method)



- Use the **Least Squares Method** to solve above **overdetermined system**.
- It only works for pixels with non-zero spatial gradient!
Thus, only **local** and **sparse flow** can be obtained.

Flow Estimation: Data Term

We also aim to estimate optical flow in regions with vanishing local spatial gradients.

In problem formulation, define the global optical flow **vector field** $[u(x, y), v(x, y)]$.

- Solve optical flow **globally**
 - L2 norm (used in **Horn and Schunck** algorithm)

$$E_{\text{Data}} = \sum_{x,y} \left[\textcircled{u} \frac{\partial I}{\partial x} + \textcircled{v} \frac{\partial I}{\partial y} + \frac{\partial I}{\partial t} \right]^2.$$

Note: we need to solve u, v for every pixel!

- L1 norm

$$\|\mathbf{E}\|_1 = \sum_{x,y} |E_{x,y}| \approx \sum_{x,y} \sqrt{\|E_{x,y}\|^2 + \epsilon^2},$$

For more robust estimation.

Flow Estimation: Prior Term

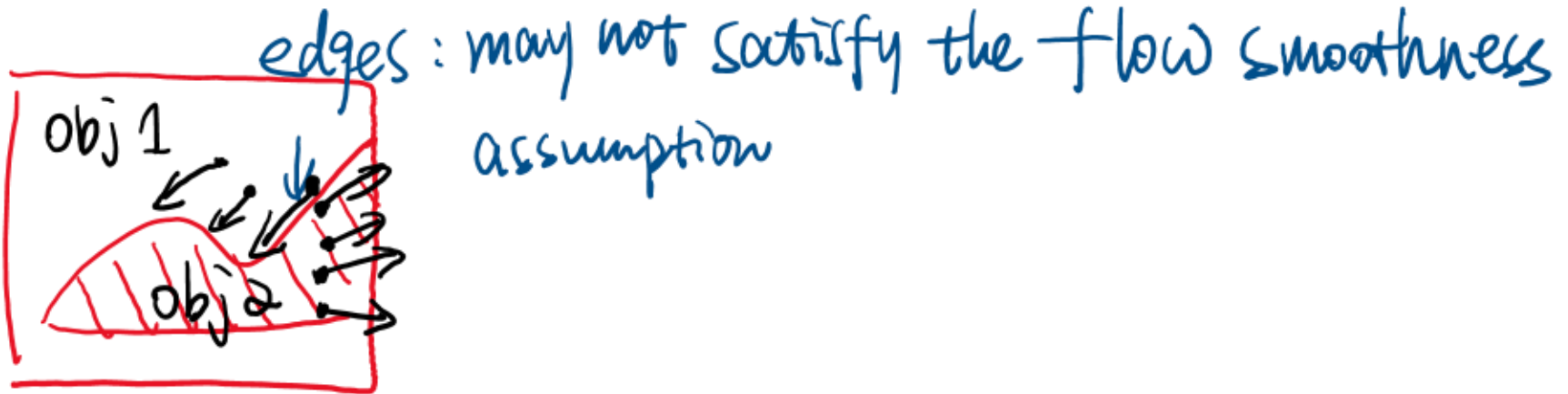
- Spatial and temporal smoothness
 - Favor small first-order derivatives (gradients) of the flow field (used in **Horn and Schunck** algorithm)

$$E_{\text{Prior}} = \sum_{x,y} \left[\left(\frac{\partial u}{\partial x} \right)^2 + \left(\frac{\partial u}{\partial y} \right)^2 + \left(\frac{\partial v}{\partial x} \right)^2 + \left(\frac{\partial v}{\partial y} \right)^2 \right].$$

- Also, possible to add temporal smoothness terms $\frac{\partial u}{\partial t}$ and $\frac{\partial v}{\partial t}$.

Flow Estimation: Prior Term

- Different weights for different locations. **Why?**
 - Flow may not be smooth at edges.
 - We may want to reduce the weight of the prior at edges (with high $|\nabla I|$).



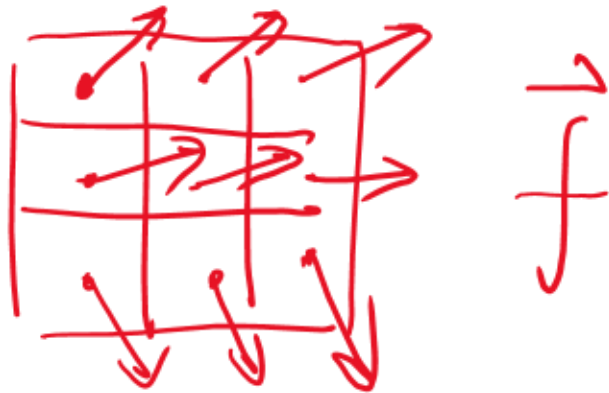
To do that, design the prior term as

$$E_{\text{Prior}} = \sum_{x,y} w(\nabla I) \left[\left(\frac{\partial u}{\partial x} \right)^2 + \left(\frac{\partial u}{\partial y} \right)^2 + \left(\frac{\partial v}{\partial x} \right)^2 + \left(\frac{\partial v}{\partial y} \right)^2 \right].$$

Use proportional or inverse proportional relation between weights and spatial gradients?

Flow Estimation: Optimization

- Recap: **How to formulate the flow estimation problem?**



Recall the flow est. prob.:

argmin $\{E_{\text{Global}}\}$

vector field

The calculus of variations will help to solve the above problem.

- Hard to find the analytical solution due to various E_{Global}
- Numerical and iterative solution

Event-based Flow Estimation

- **Lucas-Kanade variants**

No frames/intensities... How to obtain spatial and temporal gradients?

1. For each neighborhood, use events to compute temporal gradients and approximate spatial gradients
2. Find the local flow based on L-K method

+ Straightforward and easy to implement

- Noisy results, needs further filtering
- Approximation on spatial gradients not accurate
- Only sparse flow available

$$\begin{cases} \frac{\partial e(x,y,t)}{\partial x} \sim \sum_{t-\Delta t}^t e(x,y,t) - \sum_{t-\Delta t}^t e(x-1,y,t) \\ \frac{\partial e(x,y,t)}{\partial y} \sim \sum_{t-\Delta t}^t e(x,y,t) - \sum_{t-\Delta t}^t e(x,y-1,t) \end{cases}$$

$$\frac{\partial e(x,y,t)}{\partial t} \sim \frac{\sum_{t-\Delta t}^{t_1} e(x,y,t) - \sum_{t-\Delta t}^t e(x,y,t)}{t-t_1}$$

$$\text{grad}^t(I) \begin{pmatrix} v_x \\ v_y \end{pmatrix} + \frac{\partial I}{\partial t} = 0$$

Benosman, Ryad, et al. "Asynchronous frameless event-based optical flow." Neural Networks 27 (2012): 32-37.

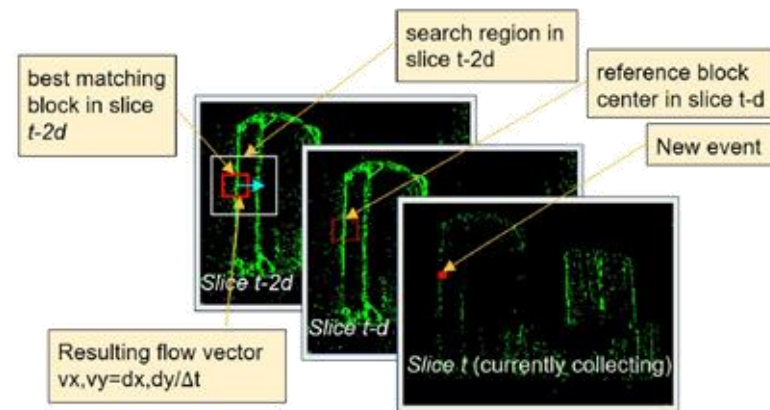
Event-based Flow Estimation

- **Block matching**

- Idea: match **block**, instead of a single pixel

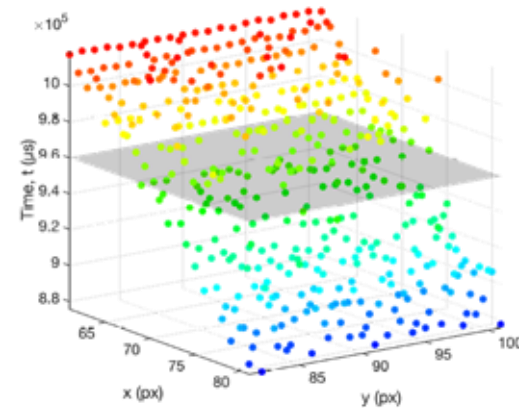
+ More robust as block may contain more information

- Block features' location may not be as accurate as point features
- Not good for complex scenes due to repetitive textures and contours
- Defining block features is much more difficult than defining point features!



Event-based Flow Estimation

- **Local plane fitting**
 1. Create **event time surface**
 2. Use a local plane to fit the time surface
 3. Gradient (normal vector) of the plane indicates the flow
 - Noisy results that requires further filtering



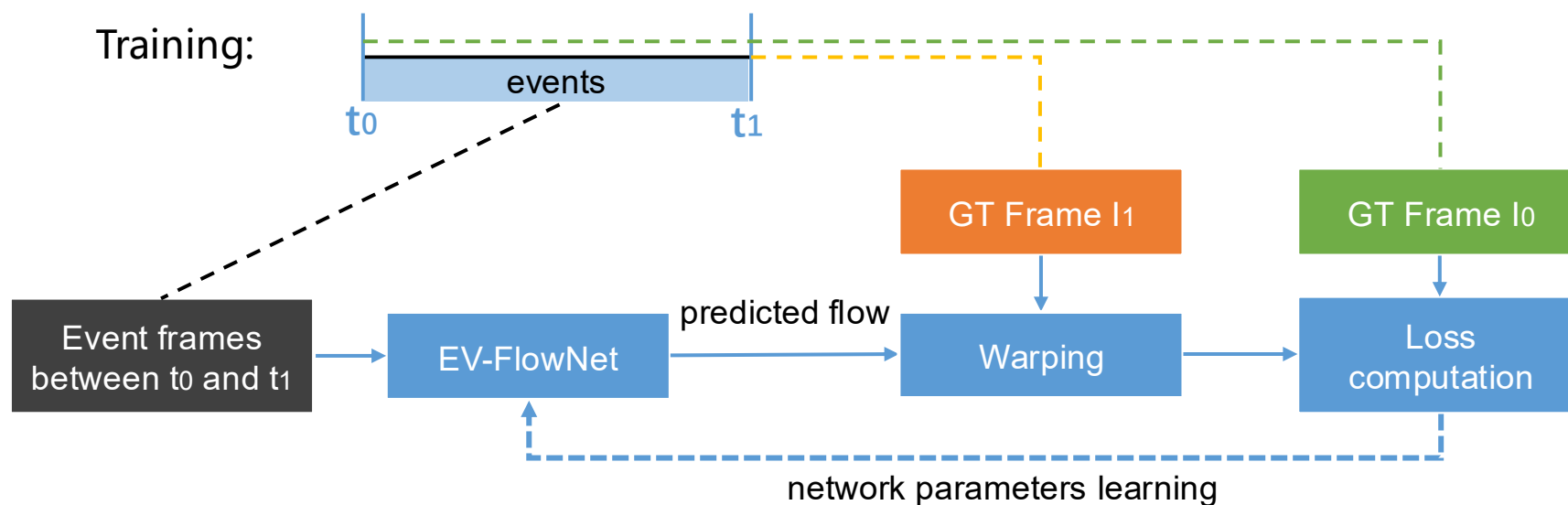
- **Energy minimization/optimization** (deep learning)
 - + Global method
 - + Able to provide dense flow
 - High computational complexity
 - Data-driven for deep learning

$$\min_{\mathbf{u}, L} \int_{\Omega} \int_T \left(\lambda_1 \|\mathbf{u}_x\|_1 + \lambda_2 \|\mathbf{u}_t\|_1 + \lambda_3 \|L_x\|_1 + \lambda_4 \|\langle L_x, \delta_t \mathbf{u} \rangle + L_t\|_1 + \lambda_5 h_{\theta}(L - L(t_p)) \right) dt dx + \int_{\Omega} \sum_{i=2}^{|P(\mathbf{x})|} \|L(t_i) - L(t_{i-1}) - \theta \rho_i\|_1 d\mathbf{x},$$

leng, Sio Hoi, João Carneiro, and Ryad Benjamin Benosman. "Event-Based 3D Motion Flow Estimation Using 4D Spatio Temporal Subspaces Properties." *Frontiers in neuroscience* 10 (2017): 596.
Benosman, Ryad, et al. "Event-based visual flow." *IEEE transactions on neural networks and learning systems* 25.2 (2013): 407-417.
Low, Weng Fei, et al. "SOFEA: A Non-iterative and Robust Optical Flow Estimation Algorithm for Dynamic Vision Sensors." *CVPR Workshops*. 2020.
Pan, Liyuan, Miaomiao Liu, and Richard Hartley. "Single image optical flow estimation with an event camera." *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2020.
Bardow, Patrick, Andrew J. Davison, and Stefan Leutenegger. "Simultaneous optical flow and intensity estimation from an event camera." *CVPR*. 2016.

Paper: EV-FlowNet

- **Self-Supervised** Optical Flow Estimation for Event-based Cameras
 - Self-supervised: **no ground-truth flow** required
 - **Groundtruth frames** required **only in training stage**



Inference: The input only contains events.

Paper: EV-FlowNet

- Event representation
 - **Image form representation** instead of the point cloud form
 - Input to the network: **4 channel image**, same resolution as the camera



Paper: EV-FlowNet

- Self-supervised loss

- **Photometric loss:** minimize the difference in intensity between the warped second image and the first image

$$\ell_{\text{photometric}}(u, v; I_t, I_{t+1}) = \sum_{x,y} \rho(I_t(x, y) - I_{t+1}(x + u(x, y), y + v(x, y)))$$

where ρ is the Charbonnier loss function for outlier rejection.

- **Smoothness loss:** regularize the output flow by minimizing the difference in flow between neighboring pixels horizontally and vertically

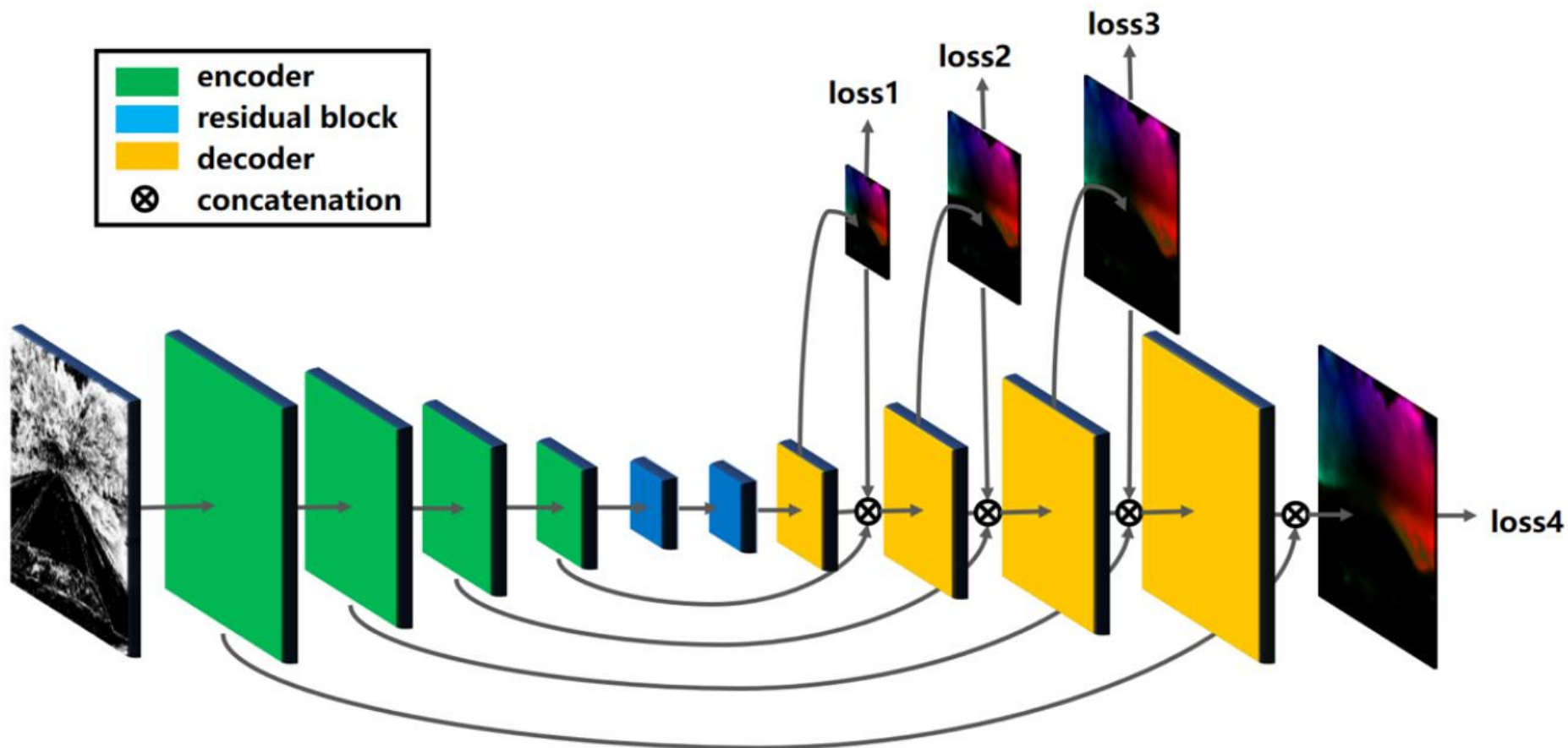
$$\ell_{\text{smoothness}}(u, v) = \sum_{x,y} \sum_{i,j \in \mathcal{N}(x,y)} \rho(u(x, y) - u(i, j)) + \rho(v(x, y) - v(i, j))$$

where $\mathcal{N}$ is the set of neighbors around (x, y) .

Total loss

Paper: EV-FlowNet

- EV-FlowNet architecture



Recap on CNN (For self-learning)

- Convolution

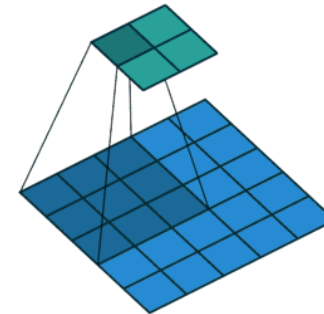
1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

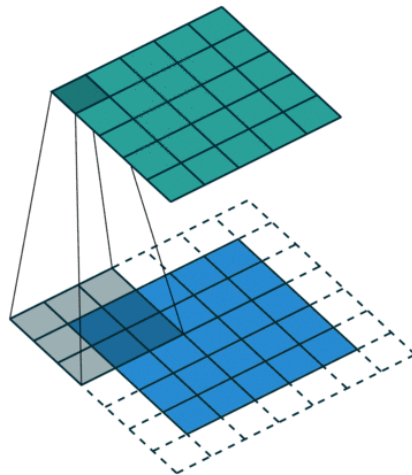
4		

Convolved
Feature

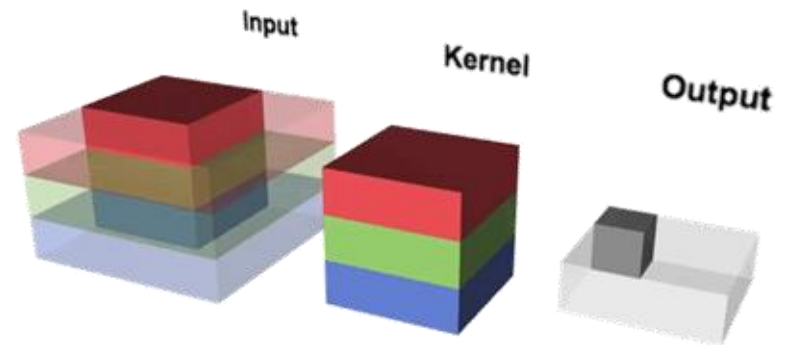
- Stride ($s = 2$) **Reduce output size**



- Padding **Increase output size**



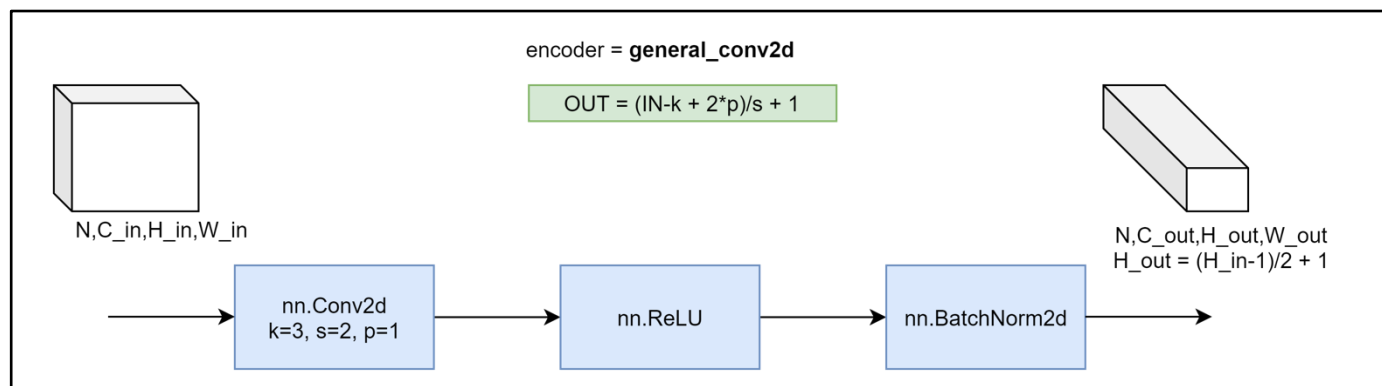
- Multi-channel convolution



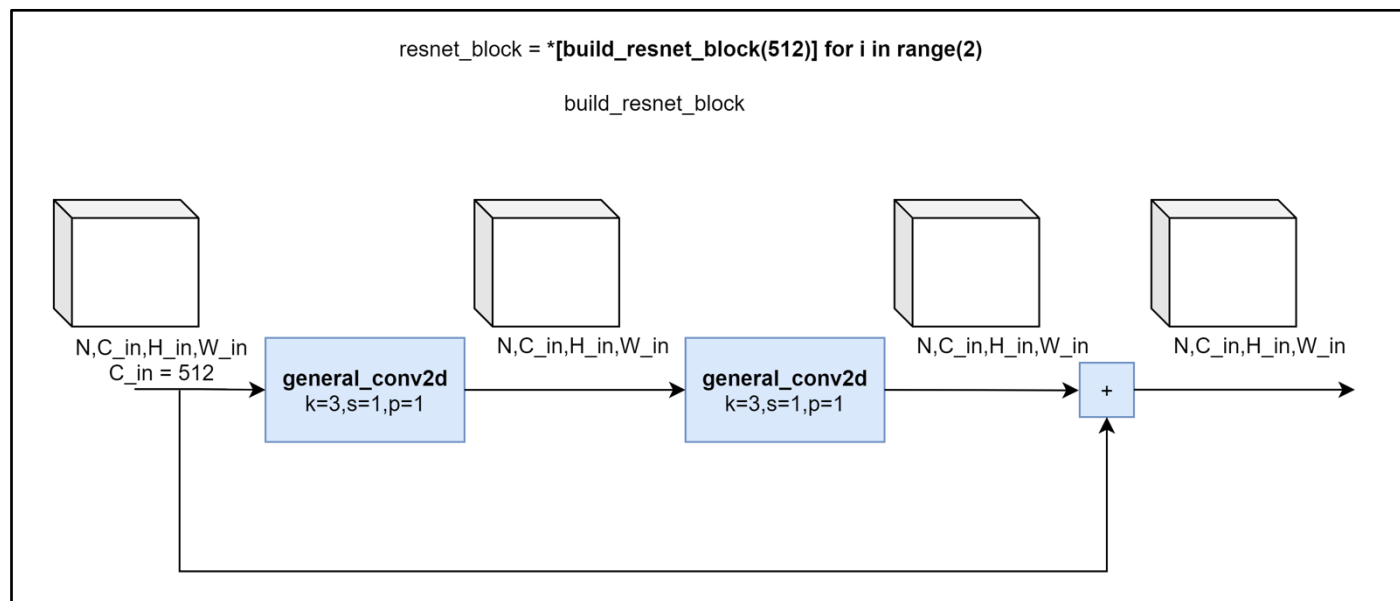
CNN Part - I · Mandar Deshpande (mandroid6.github.io)
[Multi-Channel Convolutions explained with... MS Excel! | by Thom Lane | Apache MXNet | Medium](#)
[File:Convolution arithmetic - Padding strides.gif - Wikimedia Commons](#)

Paper: EV-FlowNet

- (For self-learning) EV-FlowNet architecture in details: sub-modules

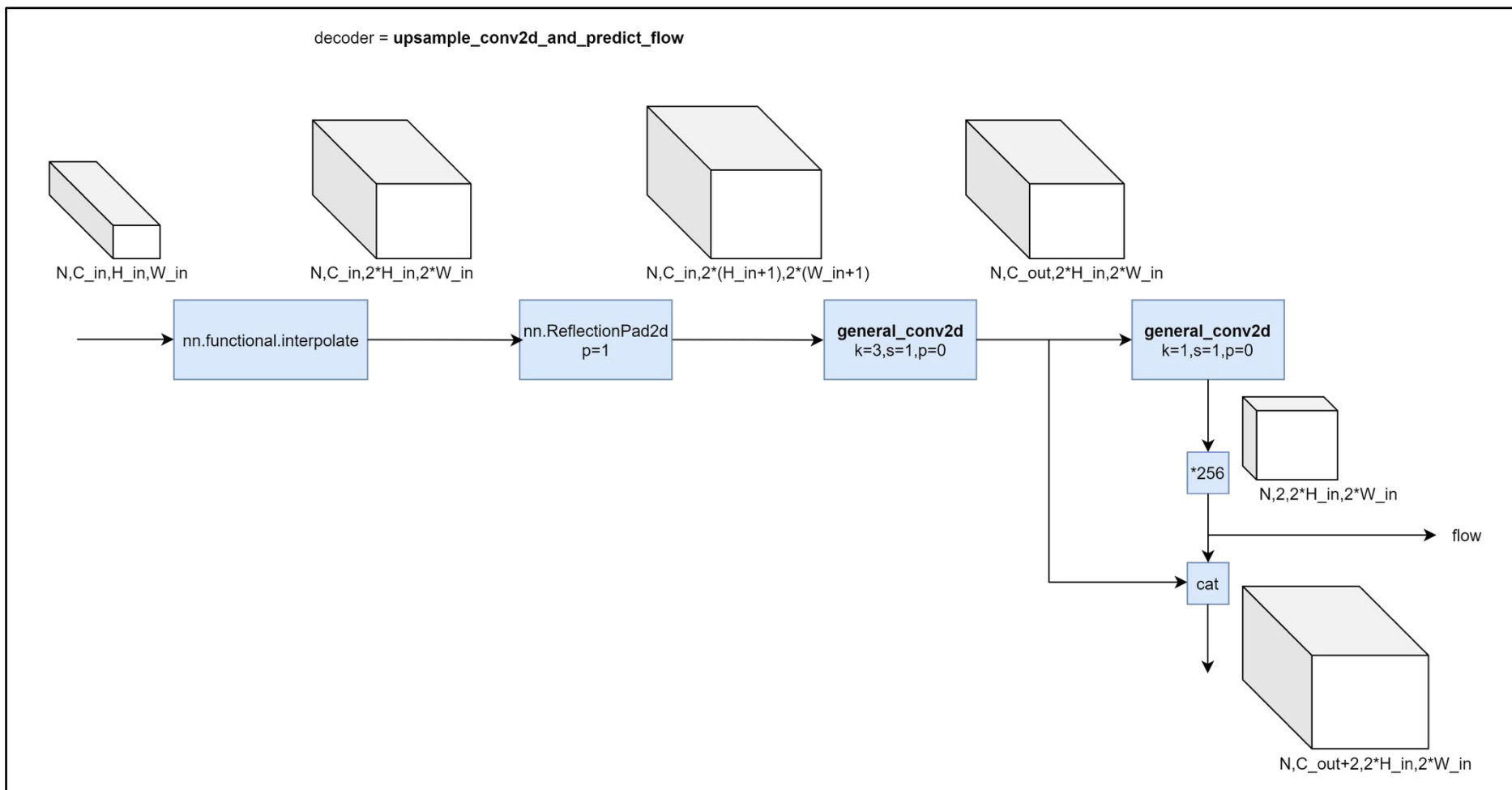


k: kernel size
s: stride
p: padding



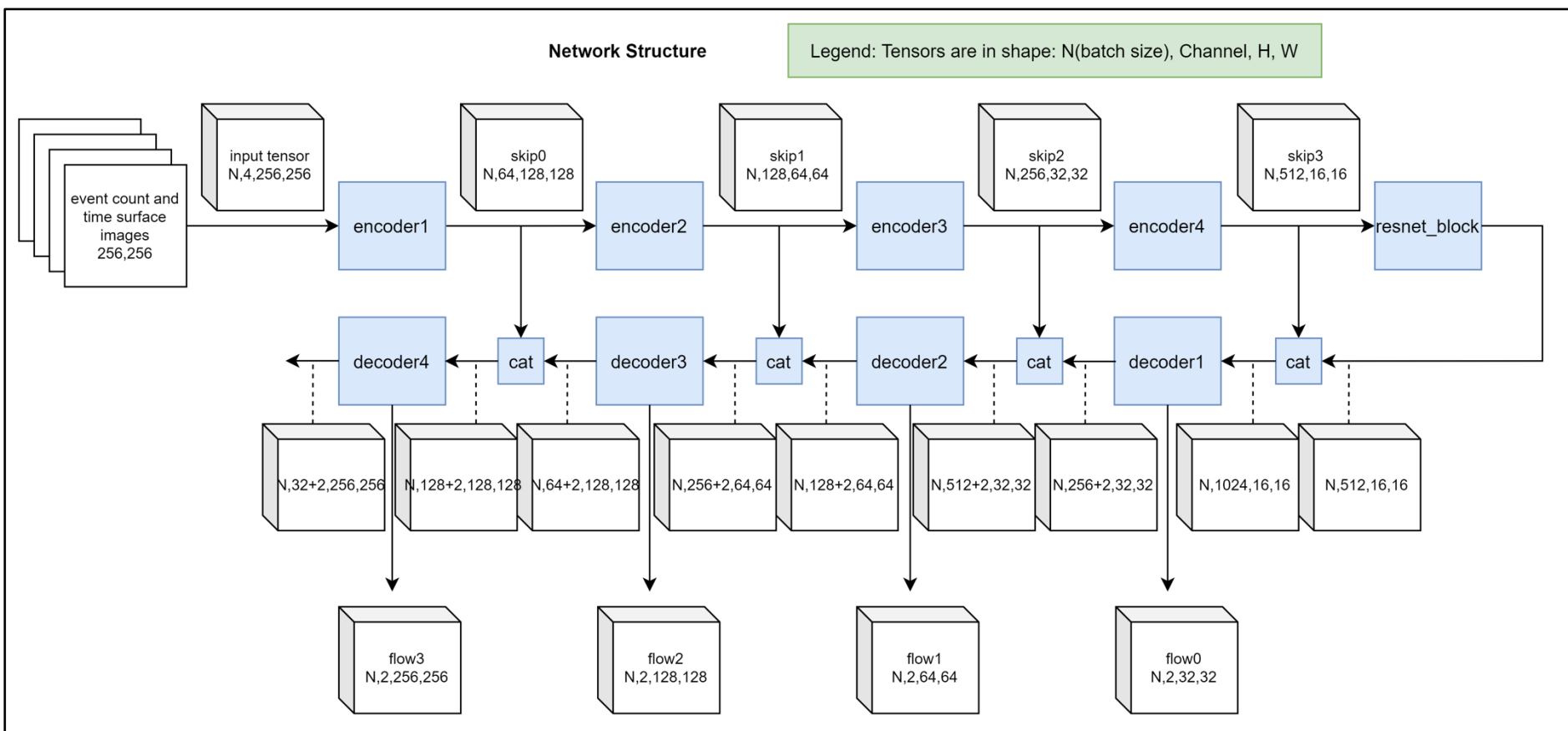
Paper: EV-FlowNet

- (For self-learning) EV-FlowNet architecture in details: sub-modules



Paper: EV-FlowNet

- (For self-learning) EV-FlowNet architecture in details: framework



EV-FlowNet: Self-Supervised Optical Flow Estimation for Event-based Cameras

Alex Zihao Zhu, Liangzhe Yuan, Kenneth Chaney, Kostas Daniilidis



Summary – Optical flow estimation

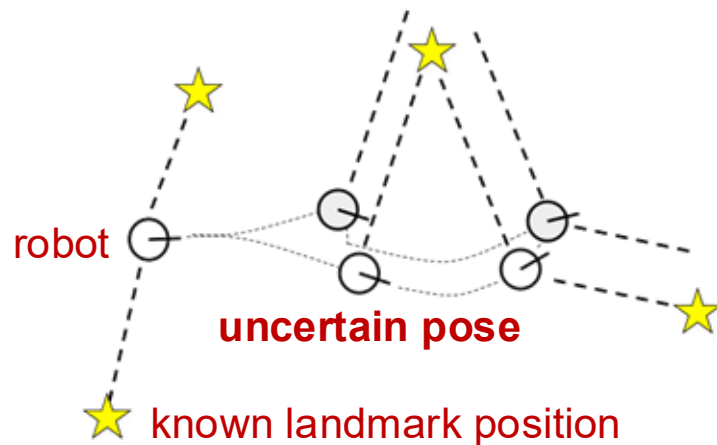
- Optical flow field vs. motion field
- Information in optical flow: Why it is necessary to obtain optical flow?
- Visualization of optical flow
- Frame-based optical flow estimation
- Event-based flow estimation
 - Lucas-Kanade variants
 - Block matching
 - Local plane fitting
 - Energy minimization/optimization (with/without deep learning)

Break - 15 mins

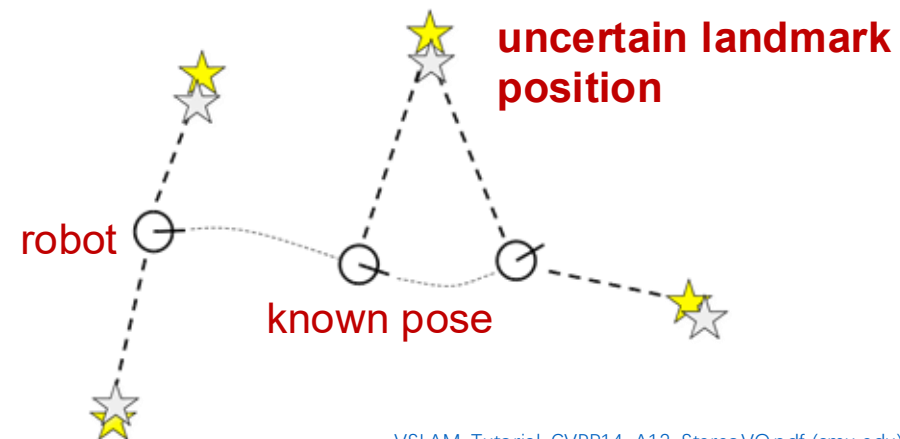
Pose Estimation and SLAM

- Problem
 - Pose estimation:
Estimating **poses only**.
 - SLAM (Simultaneous Localization And Mapping):
Computing the **poses** and the **map of the environment** at the same time.

Example of localization/pose estimation



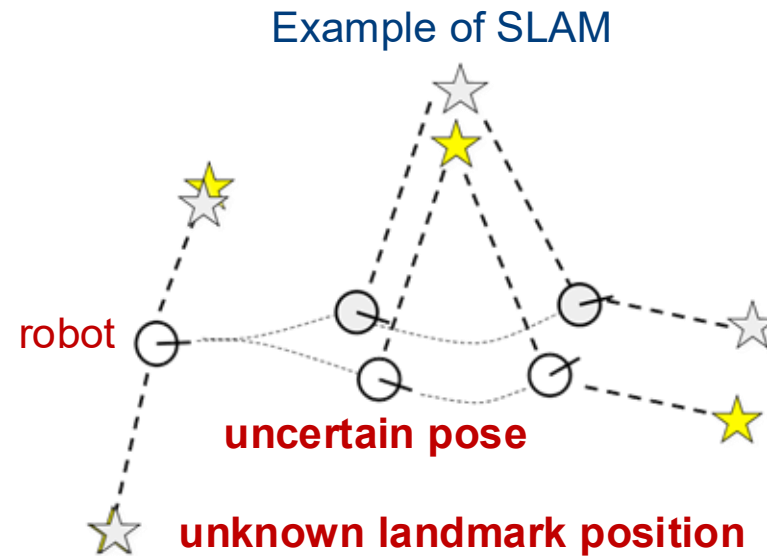
Example of mapping



[VSLAM-Tutorial-CVPR14-A12-StereoVO.pdf \(cmu.edu\)](#)
[Slide 1 \(toronto.edu\)](#)

[Robot Mapping - WS 2013/14 - Arbeitsgruppe: Autonome Intelligente Systeme \(uni-freiburg.de\)](#)

Pose Estimation and SLAM



- SLAM is a chicken-or-egg problem:
 - a map is needed for localization and
 - a pose estimate is needed for mapping

Pose Estimation and SLAM

- Definition of the SLAM Problem

Given

- The robot's controls

$$u_{1:T} = \{u_1, u_2, u_3 \dots, u_T\}$$

- Observations

$$z_{1:T} = \{z_1, z_2, z_3 \dots, z_T\}$$

Wanted

- Map of the environment

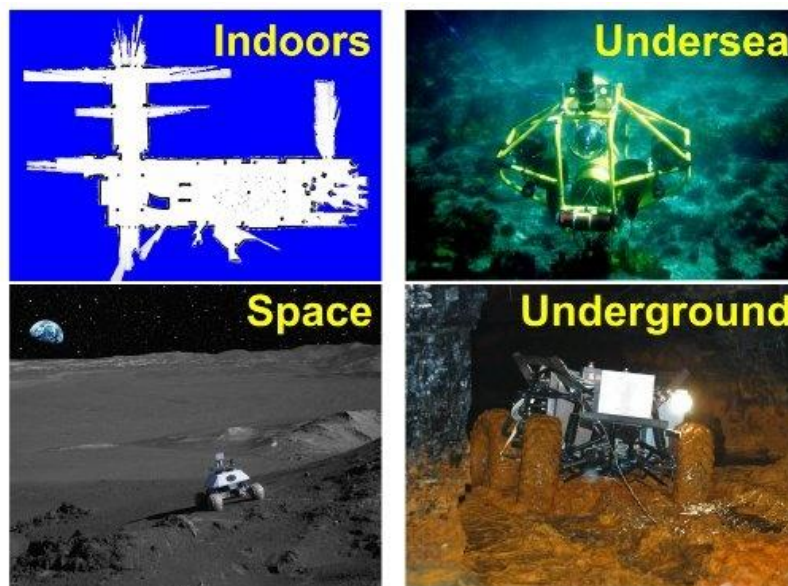
$$m$$

- Path of the robot

$$x_{0:T} = \{x_0, x_1, x_2 \dots, x_T\}$$

Pose Estimation and SLAM

- Why we need SLAM since we have the GPS and the Google Map?



- What we want for SLAM solutions? Besides accuracy...
 - Fast response for fast control feedback;
 - Environment flexibility, e.g., working in high dynamic range scenes;
 - Low power consumption and hardware complexity.

Pose Estimation and SLAM

- What sensors have been used?

- Passive sensors

Monocular

Stereo

Omnidirectional

- Active sensors

Lidar

Time-of-flight

RGB-Depth

- Hybrid sensors

Uses multiple sensors



UEye Camera



Point Grey Stereo Cam



Bubl Omnicam



Velodyne Lidar



Mesa TOF Cam

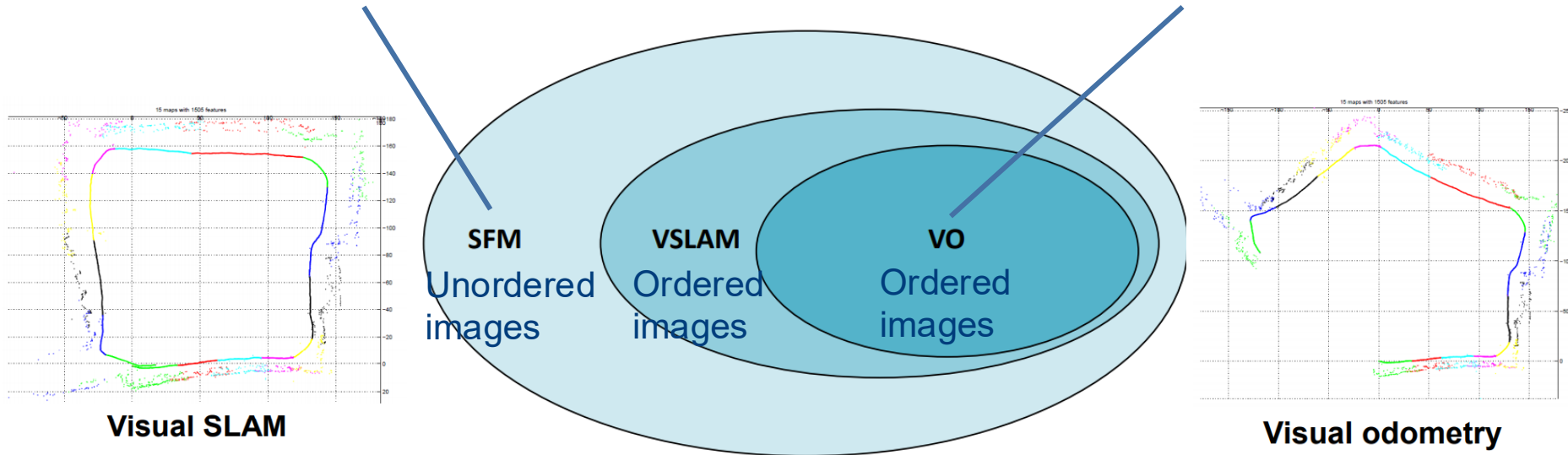


Kinect RGB-D

Visual Odometry (VO), VSLAM, and SFM

3-D reconstruction and 6-DOF pose estimation from **unordered image sets**

Estimating the 3D motion of the camera **sequentially and in real time**



- VO: Focusing on **local consistency** of the trajectory
- SLAM: Focusing on **global consistency** of the trajectory

VO can be used as a building block in a SLAM system and will be discussed here.

Visual Odometry (VO)

- VO Working Principle

1. Compute the relative motion T_k from images I_{k-1} to image I_k

$$T_k = \begin{bmatrix} R_{k,k-1} & t_{k,k-1} \\ 0 & 1 \end{bmatrix}$$



How?

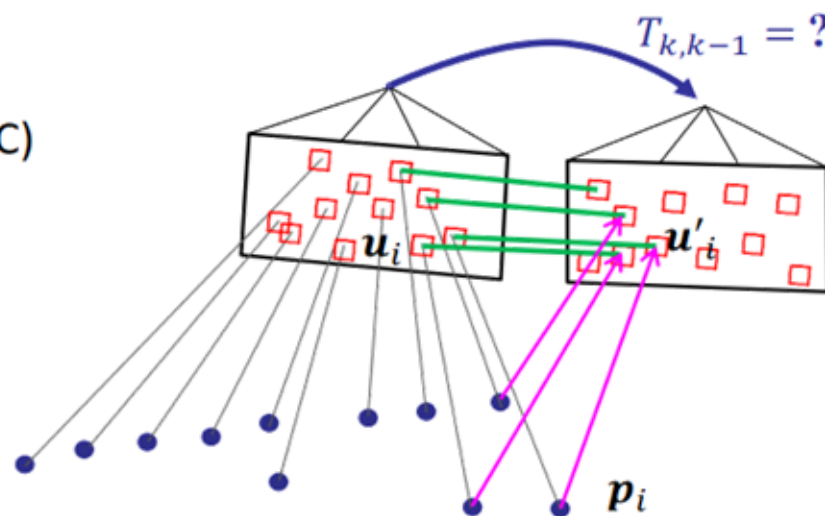
Estimating the Relative Motion T_k from Image Pairs

Feature-based methods

1. Extract & match features (+RANSAC)
2. Minimize **Reprojection error**

Similar to camera calibration!

$$T_{k,k-1} = \arg \min_T \sum_i \| \mathbf{u}'_i - \pi(\mathbf{p}_i) \|^2_{\Sigma}$$



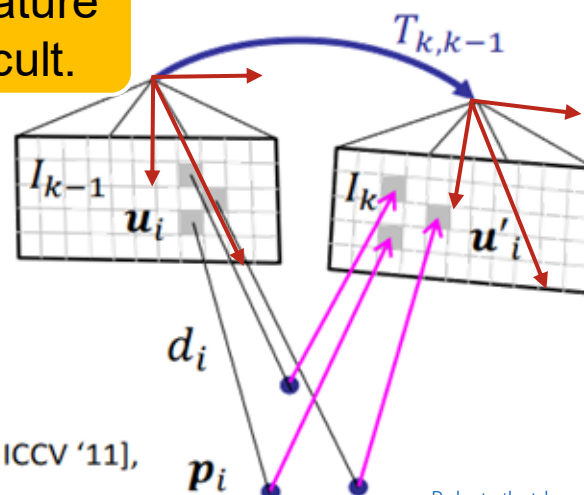
Direct methods

Effective when feature extraction is difficult.

Minimize **photometric error**

$$T_{k,k-1} = \arg \min_T \sum_i \| I_k(\mathbf{u}'_i) - I_{k-1}(\mathbf{u}_i) \|^2_{\sigma}$$

where $\mathbf{u}'_i = \pi(T \cdot (\pi^{-1}(\mathbf{u}_i) \cdot d))$



[Jin, Favaro, Soatto'03] [Silveira, Malis, Rives, TRO'08], [Newcombe et al., ICCV '11],
[Engel et al., ECCV'14], [Forster et al., ICRA'14]

[Robots that know what they do \(upenn.edu\)](http://robots.upenn.edu)

Visual Odometry (VO)

- VO Working Principle

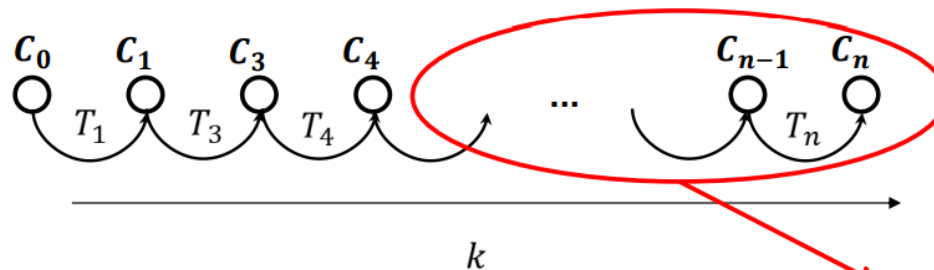
1. Compute the relative motion T_k from images I_{k-1} to image I_k

$$T_k = \begin{bmatrix} R_{k,k-1} & t_{k,k-1} \\ 0 & 1 \end{bmatrix}$$

2. Concatenate them to recover the full trajectory

camera pose $C_n = C_{n-1}T_n$ transformation

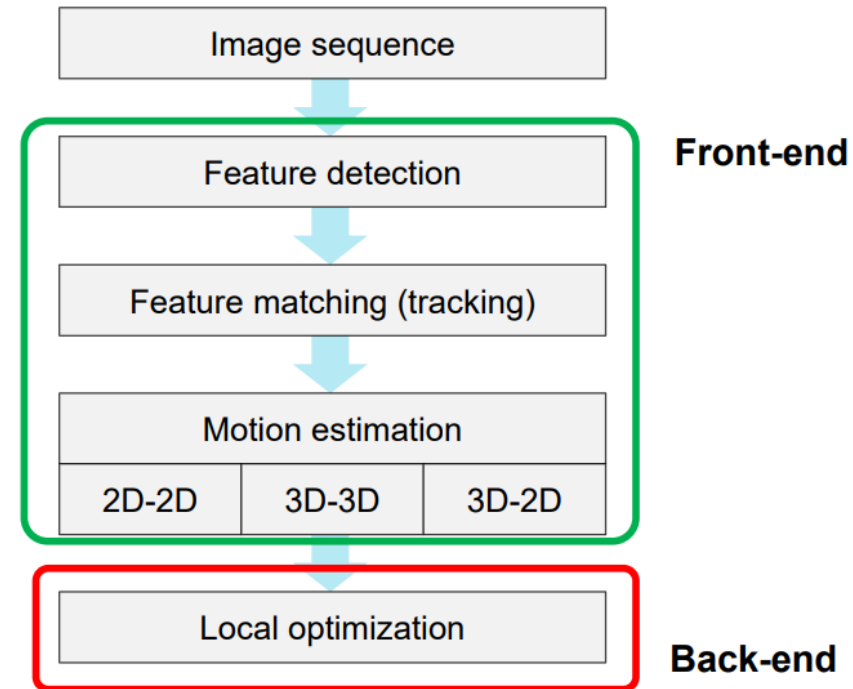
3. An optimization over the last m poses can be done to refine locally the trajectory (Pose-Graph or Bundle Adjustment)



Why not optimize over the whole trajectory?

Visual Odometry (VO) System Review

- VO systems contains the front-end and the back-end.
- The **front-end** is responsible for
 - Feature extraction, matching, and outlier removal
 - Motion estimation between two frames
- The **back-end** is responsible for the pose and structure optimization (e.g., with libraries iSAM, g2o, Google Ceres)



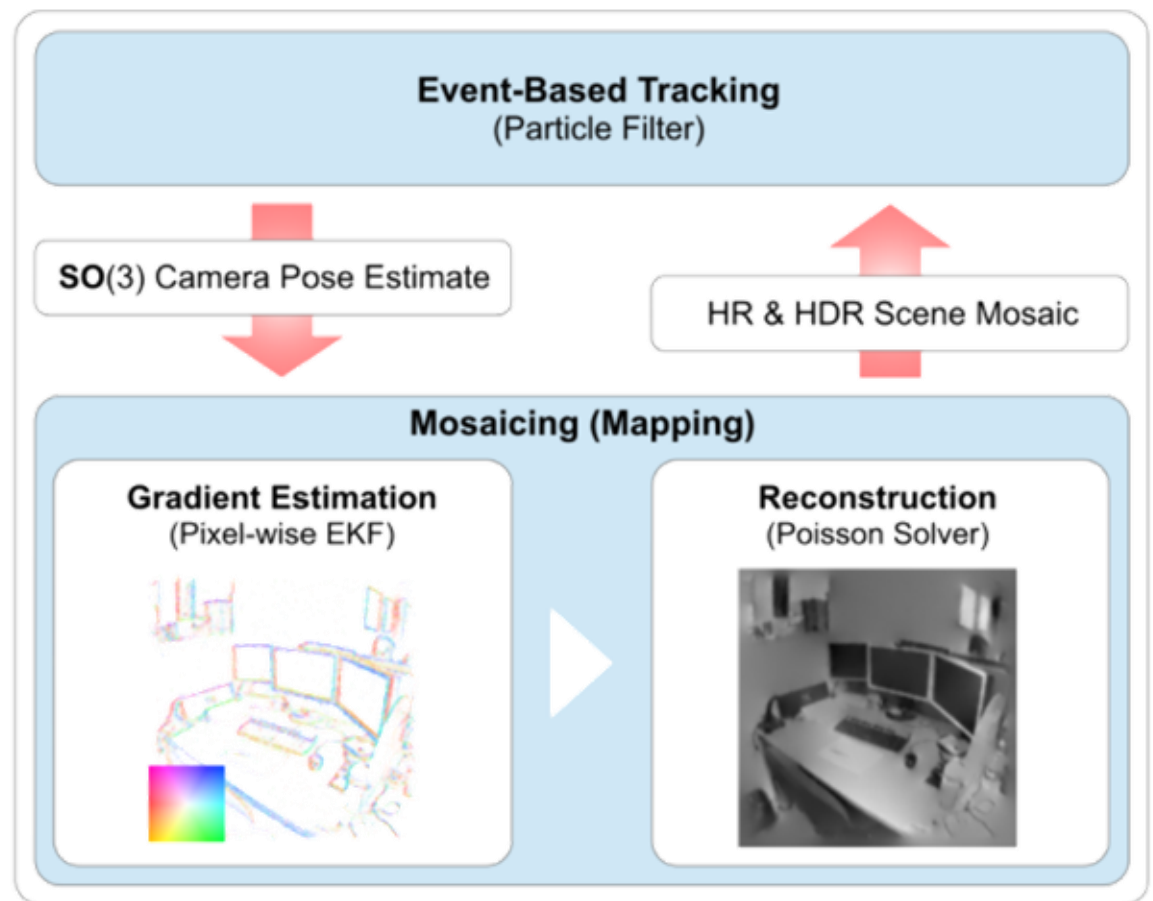
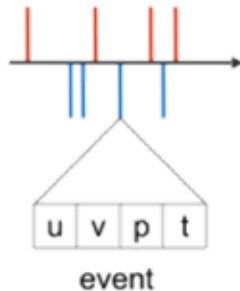
Pose Estimation and SLAM with an Event Camera

- Challenges
 - Event streams cannot be fed directly to existing methods designed for standard cameras!
- Ideas
 - Classification by data type:
event frames v.s. event point clouds
 - Classification by way of processing:
batch-based v.s. event-based
- Practical considerations (besides accuracy)
 - Latency
 - Computation complexity / power consumption

Which one would you prefer?

Paper: Simultaneous Mosaicing and Tracking with an Event Camera

- Proposed algorithm
 - Simplified problem: only considers 3-DoF camera ego-motion - **rotation**
 - No frame involved



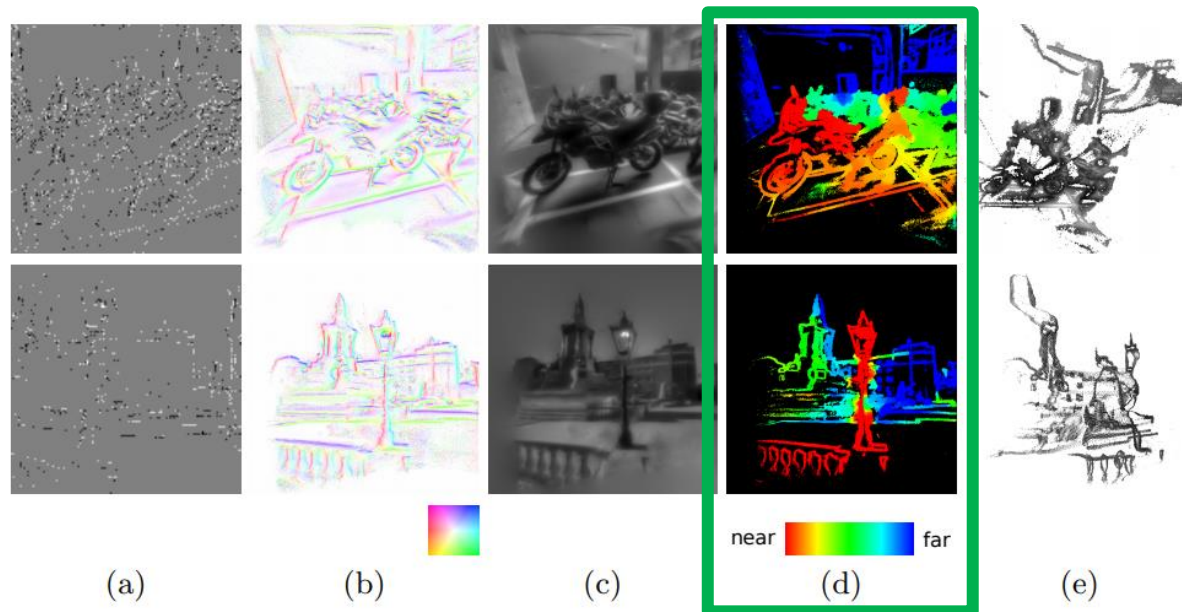
Simultaneous Mosaicing and Tracking with an Event Camera

Hanme Kim [1], Ankur Handa [2], Ryad Benosman [3],
Sio-Hoi Ieng [3], Andrew J. Davison [1]

[1] Imperial College London
[2] University of Cambridge
[3] UPMC Univ Paris 06

Paper: Simultaneous Mosaicing and Tracking with an Event Camera

- The authors have expanded their work into 6-DoF pose estimation
 - Based on 3 decoupled probabilistic filters, estimating 6-DoF **camera motion**, scene log **intensity gradient**, and scene **inverse depth**, respectively.



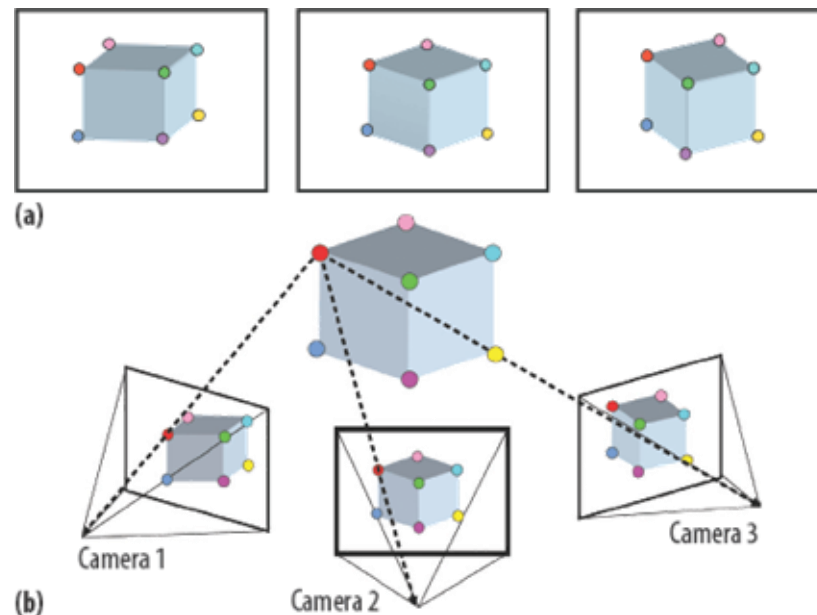
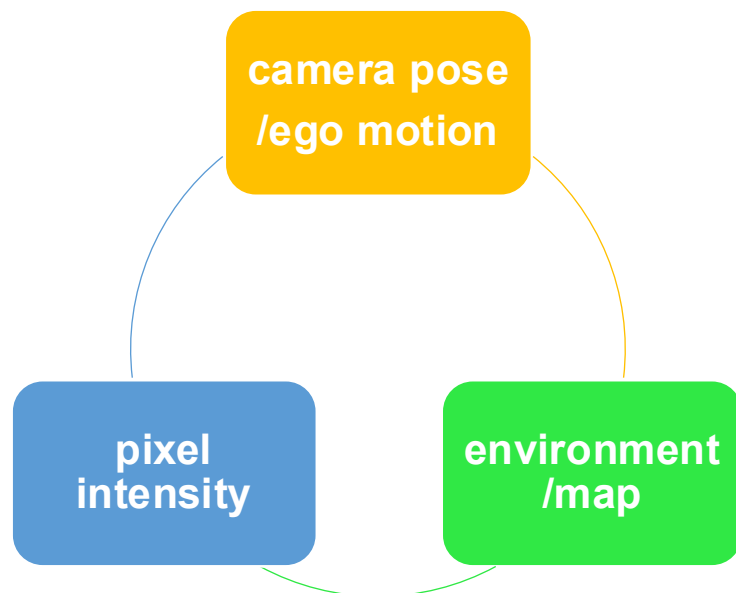
(a) visualisation of the input event stream; (b) estimated gradient keyframes; (c) reconstructed intensity keyframes with super resolution and high dynamic range properties; (d) estimated depth maps; (e) semi-dense 3D point clouds.

Summary – Pose estimation and SLAM

- Problem formulation
- Challenges for event-based SLAM
- Two main streams for event-based SLAM
 - Taking events as frames (frame-based)
 - Taking events as point clouds (pure event-based)
- Visual odometry (VO) - the core in a SLAM system
- Examples and related work
 - Survey for papers and datasets:
<https://github.com/kun150kun/ESLAM-Survey>

Image Reconstruction

- Close related to pose estimation, SLAM, and optical flow estimation.



- Here we define the **image reconstruction problem** as:
 - Estimate the **intensity** of each pixel on images
 - **Without explicitly** estimating camera's pose or environment

Usually focusing on image quality instead of the camera motion / environment modelling.

Image Reconstruction

- Why brings event cameras into image reconstruction?

- High speed imaging



- High dynamic range imaging



- High resolution imaging
by reconstructing sub-pixel
intensity.

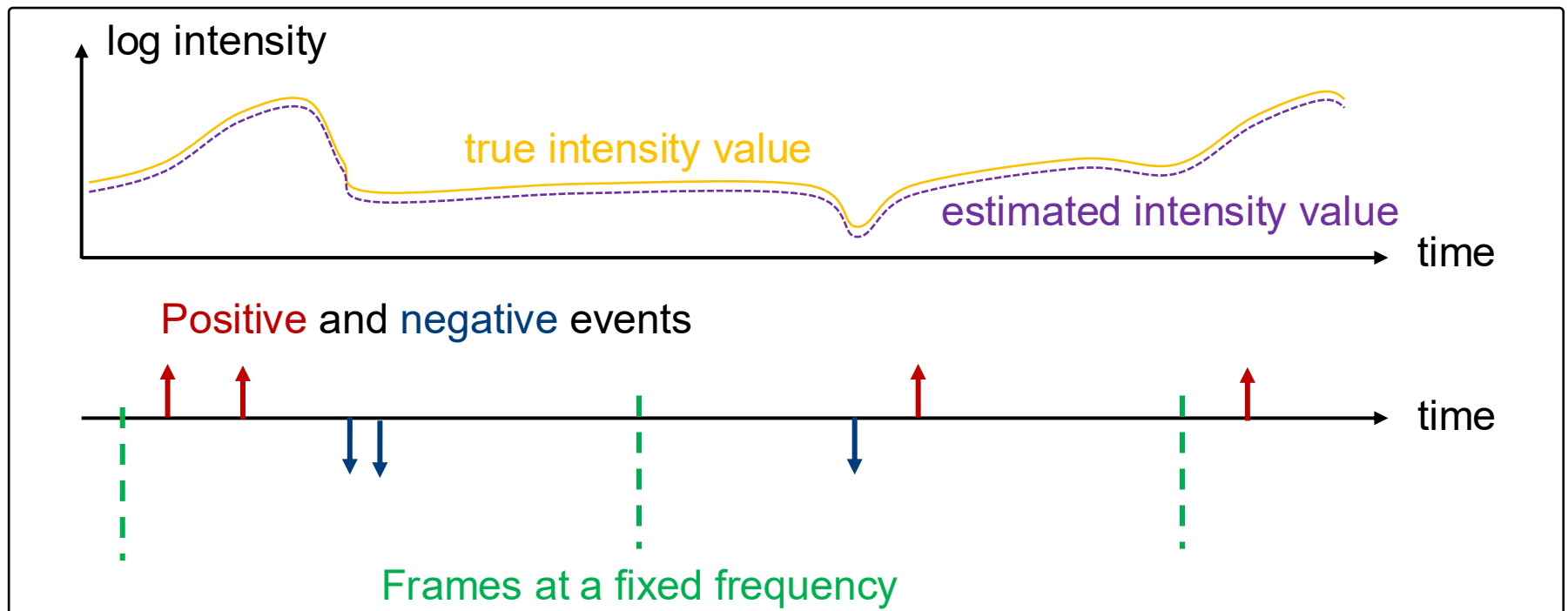


[paper066.pdf\(bmva.org\)](http://paper066.pdf(bmva.org))

Image Reconstruction

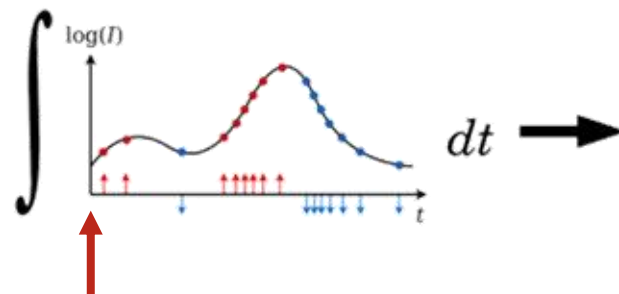
- Ideas
 - Pure event-based reconstruction
 - **Frame + event fusion**

Formulate *single-pixel frame + event fusion* as a **state estimation problem**



Paper: Continuous-time Intensity Estimation Using Event Cameras

- A naïve and simple idea would be pure integration



frame provides absolute intensity here

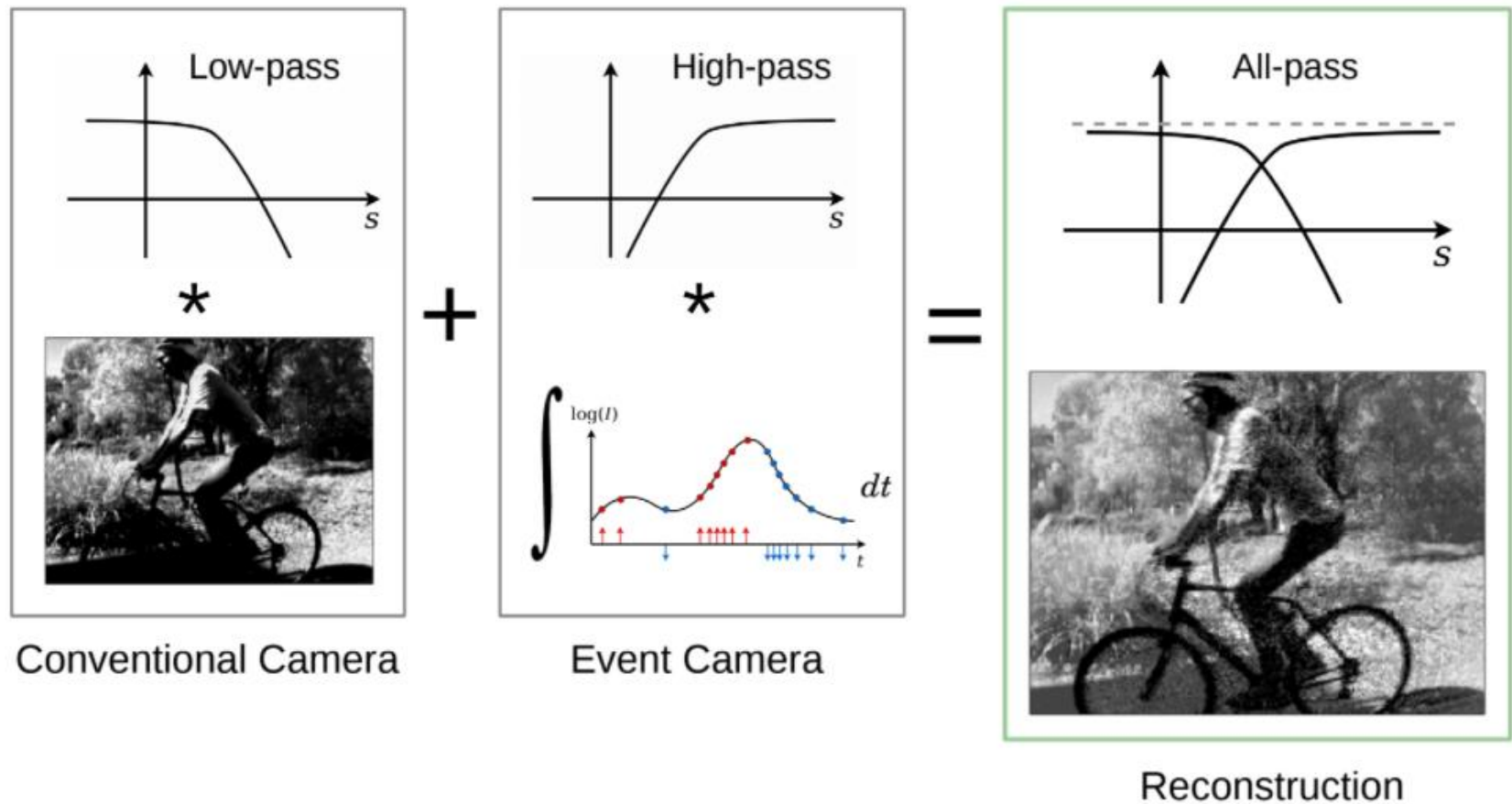


- Note that:
 - An event camera essentially outputs **high-frequency information**.
 - A conventional camera outputs **low-frequency information**.

Can we fuse low-frequency information from frames with high-frequency information from events? And how?

Paper: Continuous-time Intensity Estimation Using Event Cameras

- The complementary filter



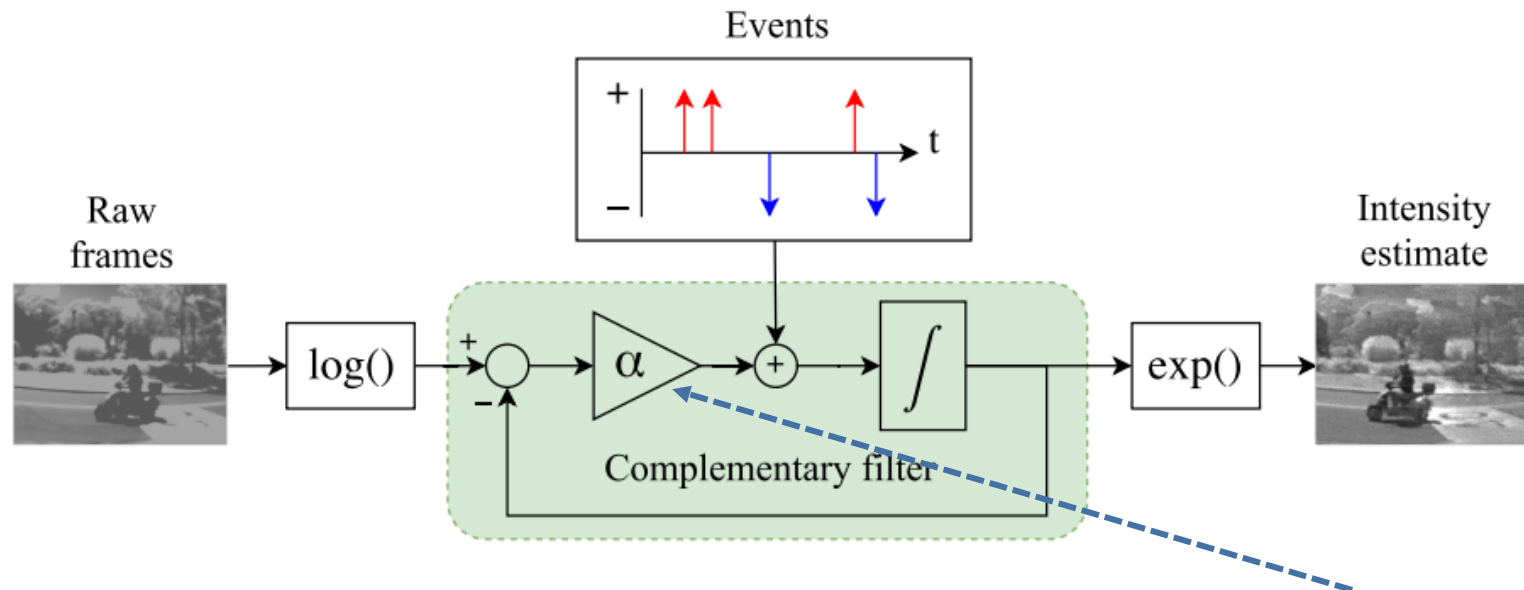


Continuous-time Intensity Estimation Using Event Cameras

Cedric Scheerlinck, Nick Barnes, Robert Mahony



Paper: An Asynchronous Kalman Filter for Hybrid Event Cameras

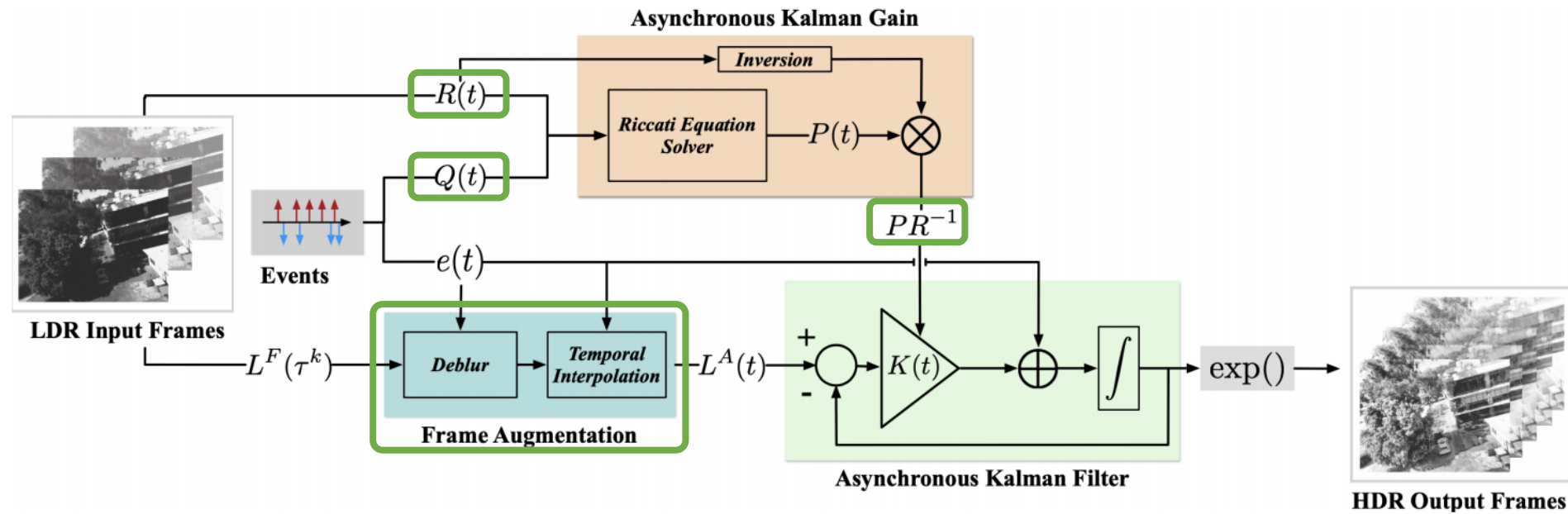


- A Complementary Filter has been proposed. There is a **filter parameter** that requires manual selection. Practical issues:
 - Hard to decide the parameter for different scenarios.
 - Not adaptive to scene changes.

Is it possible to set the filter parameter automatically?

Paper: An Asynchronous Kalman Filter for Hybrid Event Cameras

- Framework



- Contributions

- Varying and adaptive filter gain
- EVS + CIS uncertainty models
- Deblur and temporal interpolation for frame augmentation



An Asynchronous Kalman Filter for Hybrid Event Cameras

Ziwei Wang, Yonhon Ng, Cedric Scheerlinck,
Robert Mahony

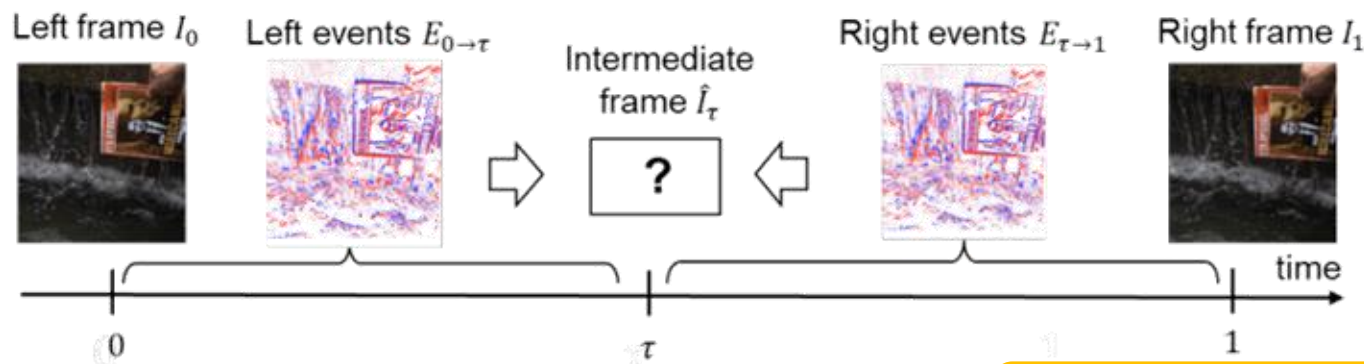


Australian
National
University

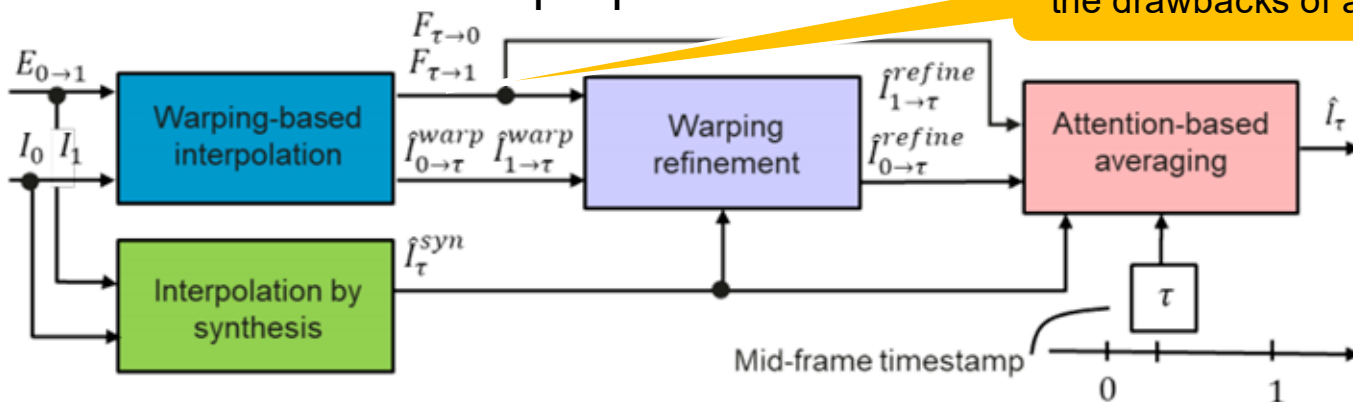
Systems Theory &
Robotics Group

Paper: TimeLens: Event-based Video Frame Interpolation

- Deep learning-based approach.
- Event-based interpolation approach utilizes **events** and **frames from two directions**.



- Overview framework of the proposed method:



TimeLens: Event-based Video Frame Interpolation

Stepan Tulyakov*, Daniel Gehrig*, Stamatios Georgoulis,
Julius Erbach, Mathias Gehrig, Yuanyou Li, Davide Scaramuzza

Code & Dataset: <http://rpg.ifi.uzh.ch/timelens>



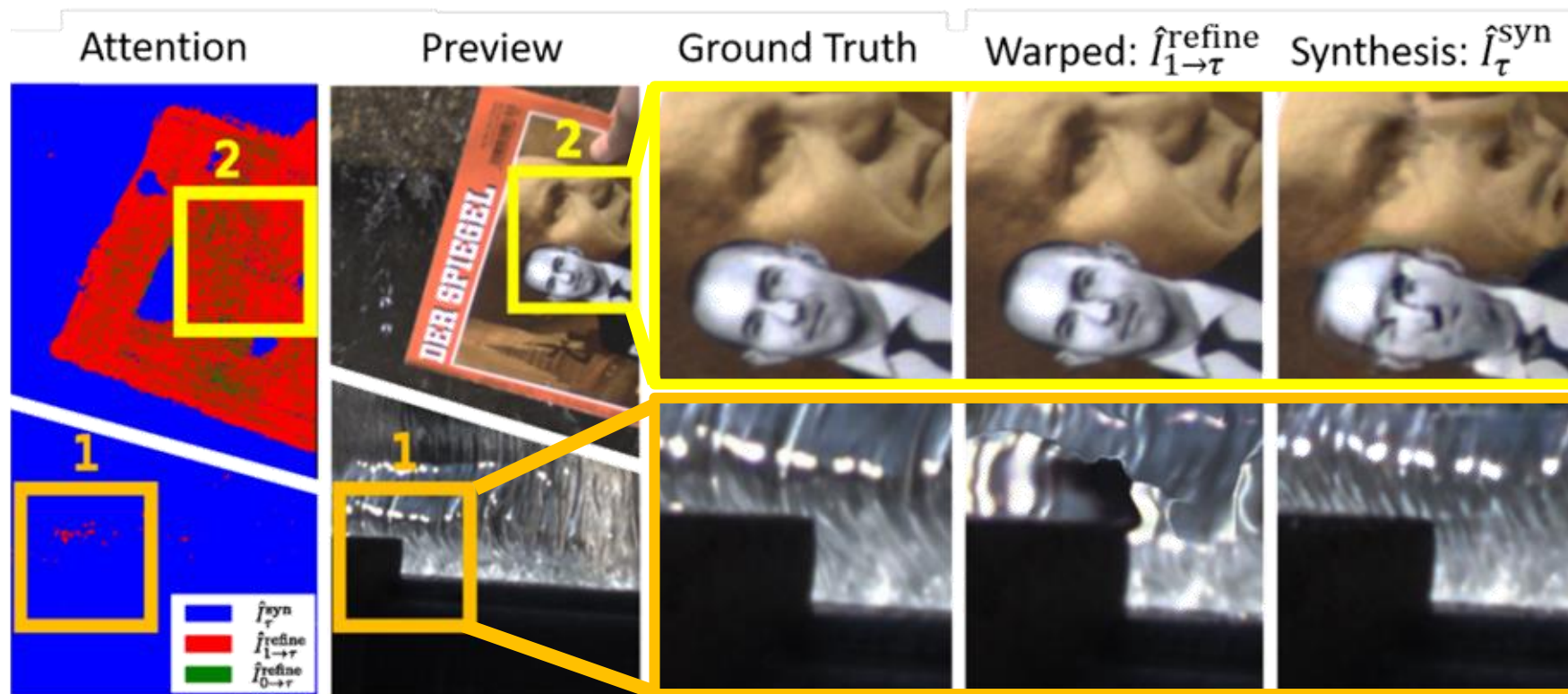
**University of
Zurich**^{UZH}



* these authors contributed equally

Paper: TimeLens: Event-based Video Frame Interpolation

- Complementarity of warping- and synthesis-based interpolation.
 - The attention network learns where to trust synthesized interpolation results, and where to trust warped interpolation results.



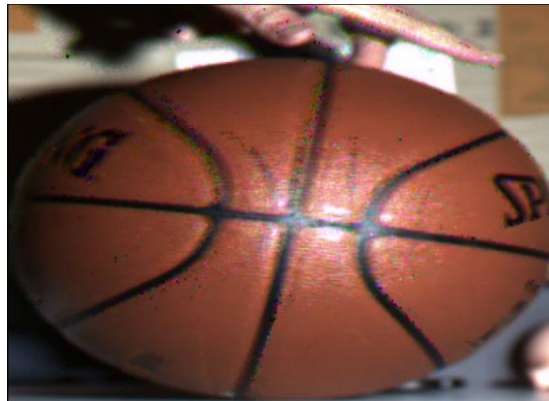
Summary – Image reconstruction

- Image reconstruction: closely related to pose estimation, SLAM, and optical flow estimation.
 - Modelling-based approaches
 - Learning-based approaches

Supplementary Materials for Self-Learning

Paper: EVS-assisted joint Deblurring, Rolling-Shutter Correction and Video Frame Interpolation through Sensor Inverse Modeling

- EVS Nonidealities
 - EVS nonidealities, such as pixel and readout latency, significantly affect frame interpolation image quality.
 - Under low-light conditions, EVS response becomes slower, resulting in ghosting or blurry frame.
 - Pixel refractory period are inevitable and worsen artifacts.



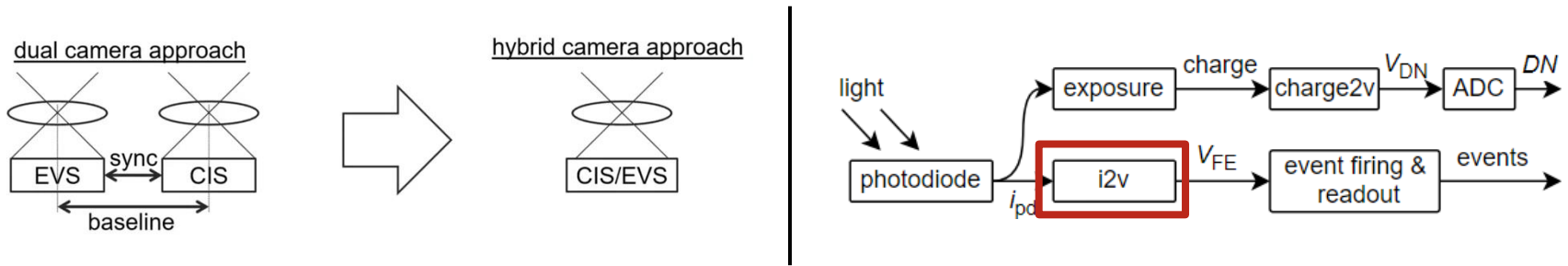
Reconstruction with latency EVS



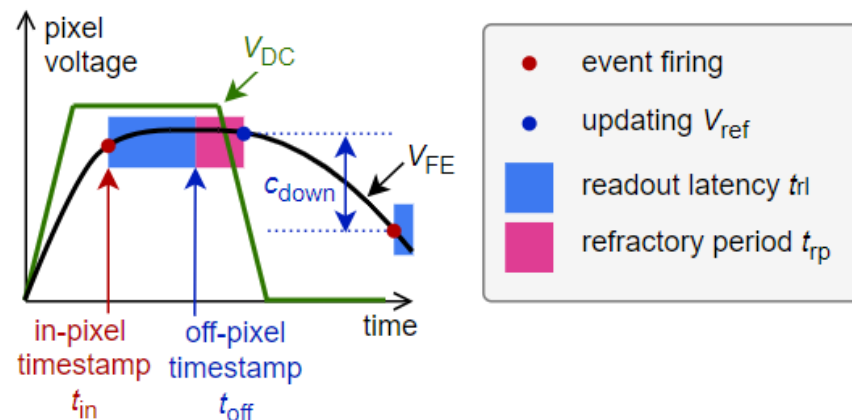
Reconstruction with compensated EVS

Paper: EVS-assisted joint Deblurring, Rolling-Shutter Correction and Video Frame Interpolation through Sensor Inverse Modeling

- Revisiting Sensor's Principle
 - Hybrid sensor with EVS and CIS pixels on the same array



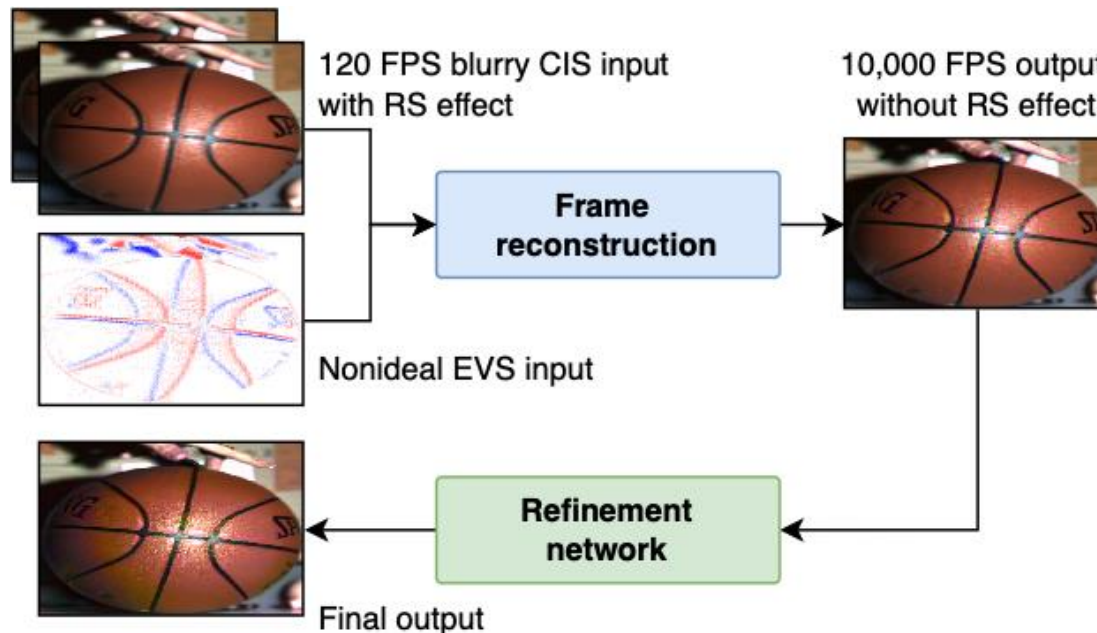
- Event triggering mechanism for a single EVS pixel



Paper: EVS-assisted joint Deblurring, Rolling-Shutter Correction and Video Frame Interpolation through Sensor Inverse Modeling

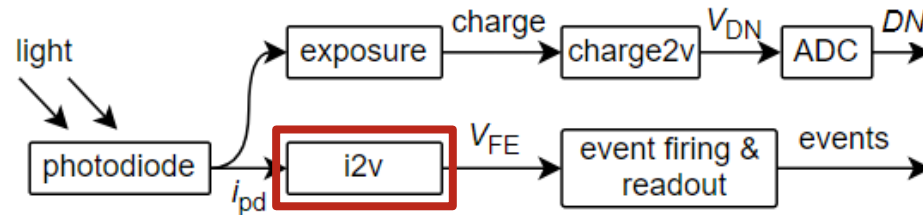
- Framework Overview

- The joint model formulation combines deblurring, rolling-shutter correction, and frame interpolation.
- An optimization method jointly solves the coupled inverse problem.
- A post-processing network is proposed for artifact removal.



Paper: EVS-assisted joint Deblurring, Rolling-Shutter Correction and Video Frame Interpolation through Sensor Inverse Modeling

- Frame reconstruction: Problem formulation



- EVS pixel latency model

The $i_{pd} - V_{FE}$ relation is modeled as a Low-Pass Filter (LPF).

- The update for V_{FE} between events can be formulated as

$$V_{FE}(k+1) = V_{FE}(k) + \gamma(k)(V_{DC}(k+1) - V_{FE}(k)) \quad (1)$$

where $V_{DC}(k+1)$ represents the V_{FE} under DC excitation. $\gamma(k)$ is the coefficient of the LPF.

- By assuming $V_{FE}(1) = V_{DC}(1)$, we have the recurrence relation:

$$V_{FE}(k+1) = f_{\gamma}(i_{pd}(k))(f_{DC}(i_{pd}(k+1)) - V_{FE}(k)) \quad (7)$$

Paper: EVS-assisted joint Deblurring, Rolling-Shutter Correction and Video Frame Interpolation through Sensor Inverse Modeling

- Frame reconstruction: Problem formulation
 - CIS-EVS fusion

CIS measurement:

$$DN = G \int_{t_s}^{t_e} i_{pd}(t) dt \quad (5)$$

where DN is the CIS intensity frame measured in digital numbers; G indicates the total gain.

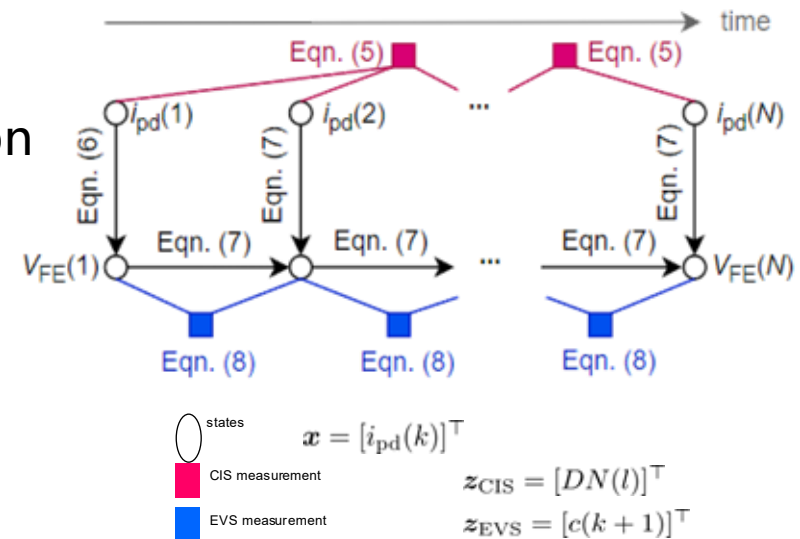
EVS measurement:

$$c(k+1) \approx V_{FE}(k+1) - V_{FE}(k) \quad (8)$$

where $c(k)$ is the known triggering thresholds that are configurable.

Paper: EVS-assisted joint Deblurring, Rolling-Shutter Correction and Video Frame Interpolation through Sensor Inverse Modeling

- Frame reconstruction:
Solving the Inverse Model via Optimization



- Minimizing the cost function

$$\mathbf{x}^* = \operatorname{argmin}_{\mathbf{x}} \left(\mathbf{e}_{\text{CIS}}^\top \boldsymbol{\Omega}_{\text{CIS}} \mathbf{e}_{\text{CIS}} + \mathbf{e}_{\text{EVS}}^\top \boldsymbol{\Omega}_{\text{EVS}} \mathbf{e}_{\text{EVS}} \right)$$

where $e_{\text{CIS}} = z_{\text{CIS}} - \tilde{z}_{\text{CIS}}$, $e_{\text{EVS}} = z_{\text{EVS}} - \tilde{z}_{\text{EVS}}$ denote measurement error vectors.

The cost function is minimized using a graph optimization with recurrent relation (7) and measurement constraints (5), (8).

Paper: EVS-assisted joint Deblurring, Rolling-Shutter Correction and Video Frame Interpolation through Sensor Inverse Modeling

- Refinement network
 - A refinement network is developed to further mitigate the noise and artifacts.
 - NAF basic block is employed.
 - The network pays attention to artifacts and noise areas with spatial attention.
 - Perceptual loss helps to alleviate blurriness or texture degradation.

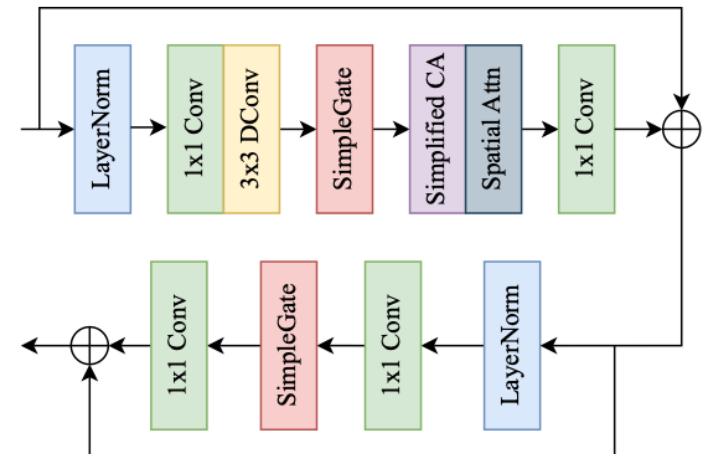
- Loss function

$$\mathcal{L}_s = w_l \mathcal{L}_1 + w_{\text{VGG}} \mathcal{L}_{\text{VGG}} + w_{\text{Gram}} \mathcal{L}_{\text{Gram}}$$

Pixel-wise L1
loss

Perceptual
loss

Style Gram
loss



Paper: EVS-assisted joint Deblurring, Rolling-Shutter Correction and Video Frame Interpolation through Sensor Inverse Modeling

- Video


OMNIVISION



Supplementary Video for

EVS-assisted joint Deblurring, Rolling-Shutter Correction and Video Frame Interpolation through Sensor Inverse Modeling

CVPR 2024

Rui Jiang*, Fangwen Tu*, Yixuan Long, Aabhaas Vaish, Bowen Zhou, Qinyi Wang, Wei Zhang, Yuntan Fang, Luis Eduardo Garcia Capel, Bo Mu, Tiejun Dai, Andreas Suess

OMNIVISION

*Equal Contribution

Paper: Simultaneous Estimation for Flow and Intensity

- Method: Sliding window **variational optimisation**
 - Idea on how to design the energy function:

$\vec{u} \& L$: functions $\vec{u}(\vec{x}, t), L(\vec{x}, t)$

joint estimation
on flow &
intensity

smoothness of the flow & intensities

$$\text{grad}^t(I) \begin{pmatrix} v_x \\ v_y \end{pmatrix} + \frac{\partial I}{\partial t} = 0$$

$$\min_{\vec{u}, L} \int_{\Omega} \int_T \left(\lambda_1 \|\mathbf{u}_x\|_1 + \lambda_2 \|\mathbf{u}_t\|_1 + \lambda_3 \|L_x\|_1 + \lambda_4 \|\langle L_x, \delta_t \mathbf{u} \rangle + L_t\|_1 + \left\{ \lambda_5 h_{\theta}(L - L(t_p)) \right\} \right) dt dx$$

brightness consistency

time of the last event

$$+ \int_{\Omega} \sum_{i=2}^{|P(\mathbf{x})|} \left\{ \|L(t_i) - L(t_{i-1}) - \theta \rho_i\|_1 \right\} dx,$$

all events fired
at $\vec{x}$

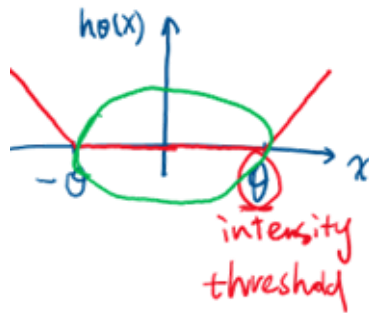
polarity

Paper: Simultaneous Estimation for Flow and Intensity

- The first 3 terms: regularise the smoothness of the flow (both spatially and temporally) and the smoothness of the intensities.
- The 4th term: first order Taylor approximation of the brightness consistency equation.
- The last 2 terms: the **no-event data term** and the **event data term**.

No-event data term:

$$\lambda_5 h_\theta(L - L(t_p))$$



$$h_\theta(x) = \begin{cases} |x| - \theta, & \text{if } |x| > \theta \\ 0, & \text{otherwise,} \end{cases}$$

Event data term:

$$\int_{\Omega} \sum_{i=2}^{|P(\mathbf{x})|} \|L(t_i) - L(t_{i-1}) - \theta \rho_i\|_1$$

We want event data to satisfy

$$|L(\mathbf{x}, t) - L(\mathbf{x}, t_p(\mathbf{x}, t))| \geq \theta$$

as much as possible.

Conference on Computer Vision and Pattern Recognition (CVPR 2016)

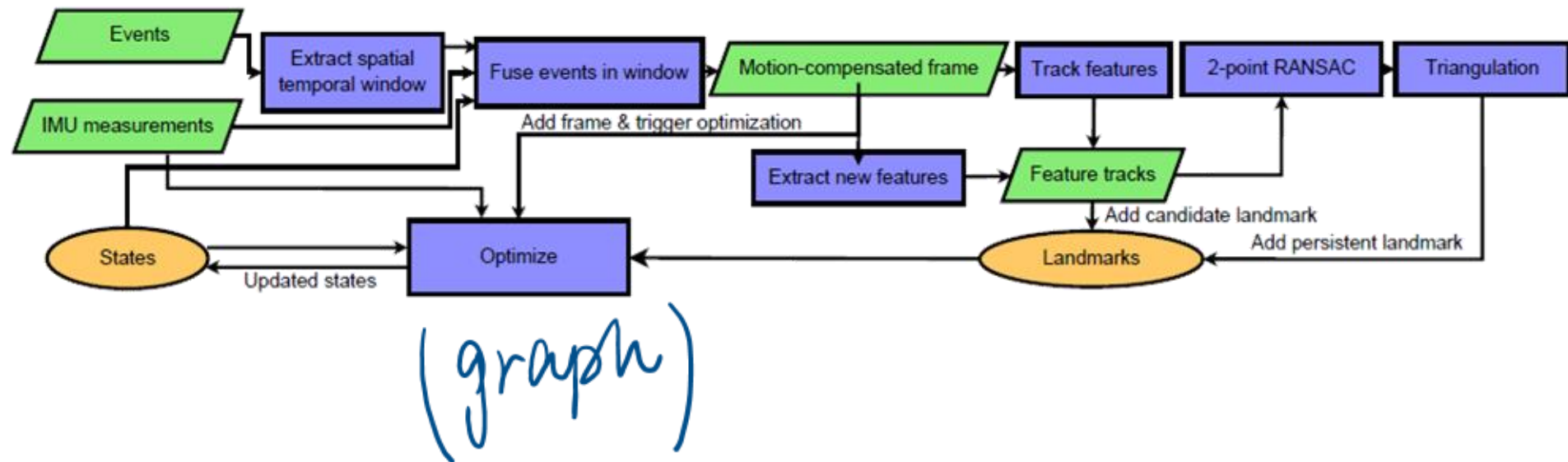
Simultaneous Optical Flow and Intensity Estimation from an Event Camera

Patrick Bardow, Andrew Davison and Stefan Leutenegger

Dyson Robotics Laboratory,
Imperial College London

Paper: Real-time Visual-Inertial Odometry for Event Cameras using Keyframe-based Nonlinear Optimization

- Overview



Paper: Real-time Visual-Inertial Odometry for Event Cameras using Keyframe-based Nonlinear Optimization

- Creating Motion-Corrected Event Frames

$$\mathbf{x}'_j = \pi \left(T_{t_k^f, t_j}^f (Z(\mathbf{x}_j) \pi^{-1}(\mathbf{x}_j)) \right)$$

To obtain $T_{t_k^f, t_j}^f$ and $Z(\mathbf{x}_j)$

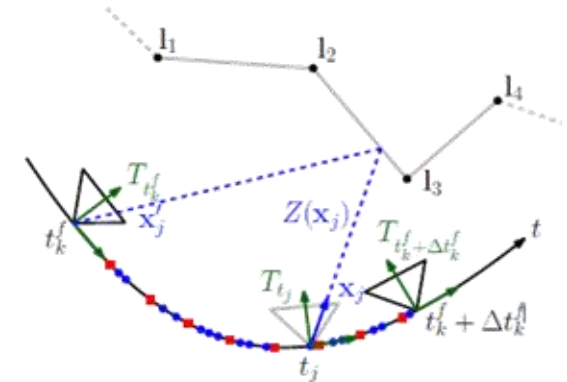
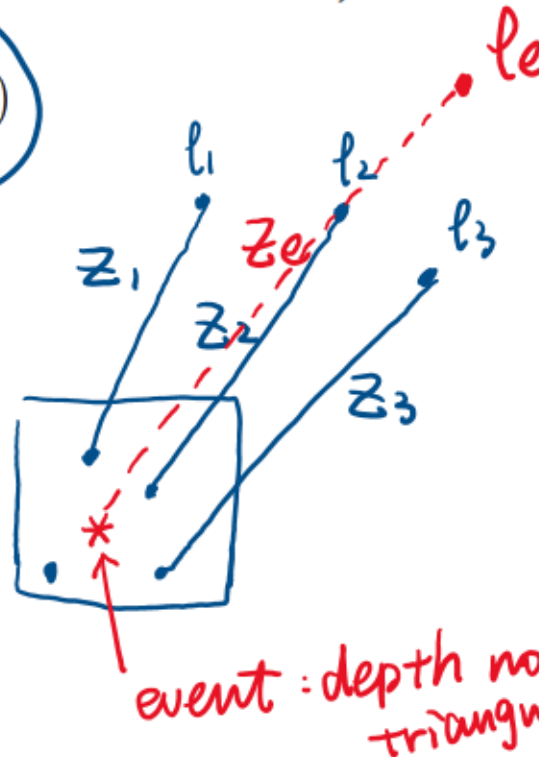
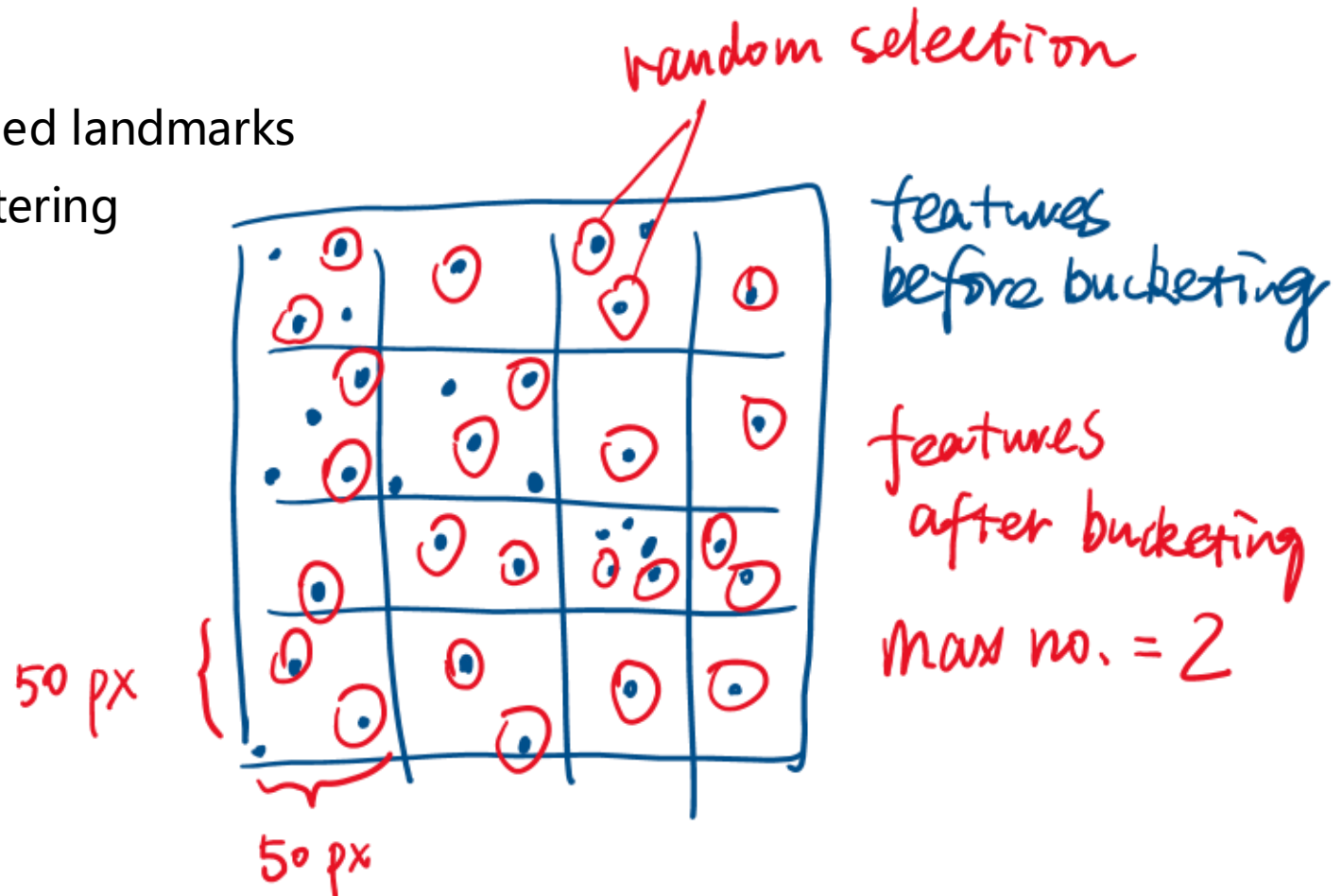


Figure 3: Motion correction: the inertial measurements in $[t_k^f, t_k^f + \Delta t_k^f]$ (red squares) are integrated to compute the relative transformation $T_{t_k^f, t_k^f + \Delta t_k^f}^f$. Each event (blue dot) e_j is reprojected to camera frame $C_{t_k^f}$ using the linearly interpolated pose T_{t_j} and the linearly interpolated depth $Z(\mathbf{x}_j)$.

Paper: Real-time Visual-Inertial Odometry for Event Cameras using Keyframe-based Nonlinear Optimization

- Feature Tracking and Landmark Triangulation
 - FAST corner detector
 - Bucketing
 - Dispose failed landmarks
 - RANSAC filtering



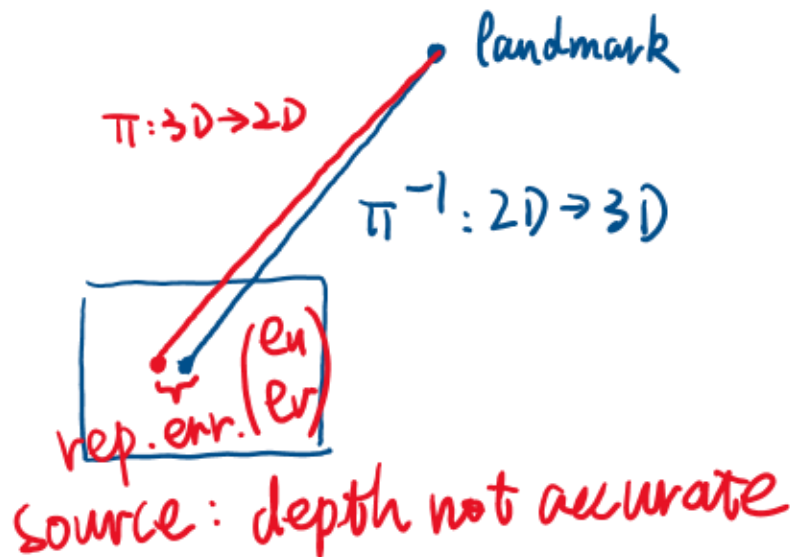
Paper: Real-time Visual-Inertial Odometry for Event Cameras using Keyframe-based Nonlinear Optimization

- Optimization on Reprojection Error and Inertial Error:

$$J := \sum_{k=1}^K \sum_{j \in \mathcal{J}(k)} \mathbf{e}^{j,kT} \mathbf{W}_r^{j,k} \mathbf{e}^{j,k} + \sum_{k=1}^{K-1} \mathbf{e}_s^kT \mathbf{W}_s^k \mathbf{e}_s^k$$

j: landmark index

k: frame index



eg. in 2D

eg. in 2D

$$\tilde{x}_2 = \hat{x}_1 + \hat{v}_1 \Delta t + \frac{1}{2} \hat{a}_1 \Delta t^2$$

pred. est. est. | meas.

$$e := \tilde{x}_2 - \hat{x}_2$$

pred. est.

Paper: Simultaneous Mosaicing and Tracking with an Event Camera

- **Event-based camera tracking based on a particle filter**

Use N particles to represent camera pose: $\{\mathbf{p}_1^{(t)}, \mathbf{p}_2^{(t)}, \dots, \mathbf{p}_N^{(t)}\}$

3D rotation group $\boxed{R_i^{(t)} \in \mathbf{SO}(3)} \quad \boxed{w_i^{(t)}}$

Two-step process:

1) Motion prediction: constant pose motion model

$$R_i^{(t)} = R_i^{(t-\tau)} \exp \left(\sum_{k=1}^3 \underbrace{\mathbf{n}_k}_{\text{Gaussian distributed noise vector}} \underbrace{G_k}_{\text{Lie group generators}} \right)$$

Paper: Simultaneous Mosaicing and Tracking with an Event Camera

- **Event-based camera tracking based on a particle filter**

2) Measurement update using Bayes rule

$$w_i^{(t)} = P(z|R_i^{(t)})w_i^{(t-\tau)}$$

How to define measurement z ?

For the **event triggering pixel**,
define measurement as the
difference on log intensity **before**
and after rotation.

$$z = \log(\text{map}(\mathbf{M}(\mathbf{p}_m^{(t)}))) - \log(\mathbf{M}(\mathbf{p}_m^{(t-\tau_c)})) ,$$

where $\mathbf{p}_m^{(t)} = \pi \left(\mathbf{R}_i^{(t)} \mathbf{K}^{-1} \dot{\mathbf{p}}_c \right) .$

$$\dot{\mathbf{p}}_c = (u, v, 1)^\top$$

$\mathbf{K}$ is the camera intrinsics;

$\pi(\mathbf{p}) = \frac{1}{p_2} (p_0, p_1)^\top$ is the projection function.

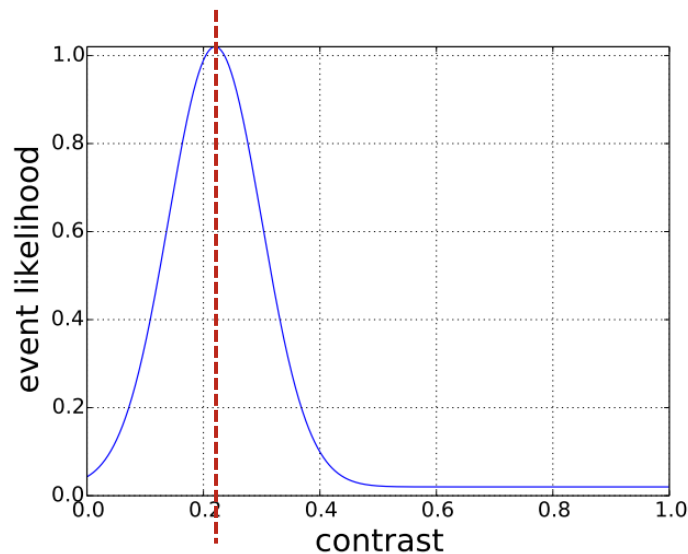
Paper: Simultaneous Mosaicing and Tracking with an Event Camera

- **Event-based camera tracking based on a particle filter**

2) **Measurement update** using Bayes rule

$$w_i^{(t)} = P(z|R_i^{(t)})w_i^{(t-\tau)}$$

How to define conditional probability ***P***?



Theoretically, peak should be triggering threshold

Paper: Simultaneous Mosaicing and Tracking with an Event Camera

▪ Mapping - Gradient estimation based on pixel-wise EKF

Two-step process:

1) Process equation

Define the **gradient vector** as **state vector**

Apply the **constant gradient process model**

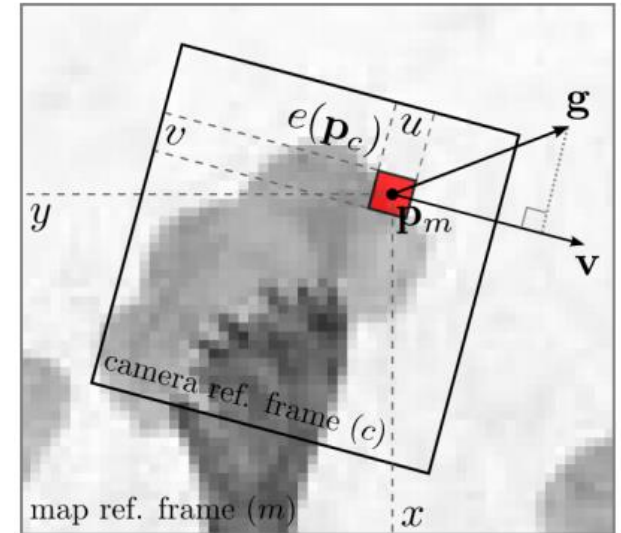
2) Measurement equation

$$\left(\mathbf{g}^{(t)} \cdot \mathbf{v}^{(t)} \right) \tau_c = \pm C$$

Define the measurement z as the **instantaneous event rate** at this pixel, we have the measurement equation:

$$h^{(t)} = \frac{\mathbf{g}^{(t)} \cdot \mathbf{v}^{(t)}}{C}$$

$$z^{(t)} = \frac{1}{\tau_c}$$



Paper: Simultaneous Mosaicing and Tracking with an Event Camera

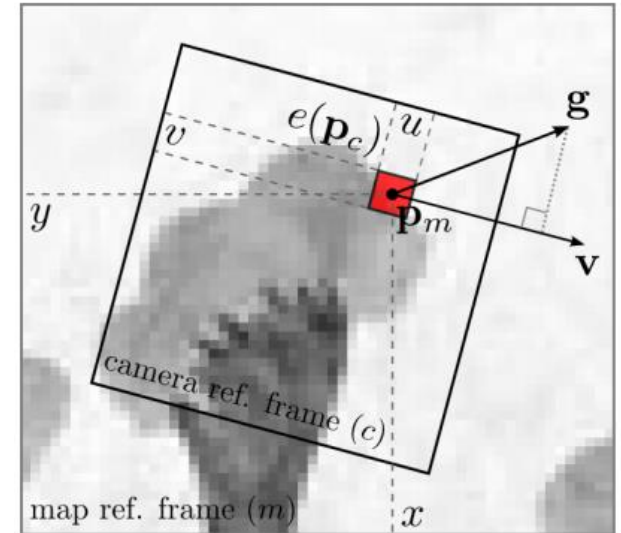
- **Mapping - Gradient estimation based on pixel-wise EKF**

Two-step process:

2) Measurement equation

How to find the velocity of the camera at a pixel $\mathbf{p}_m$?

$$\mathbf{v}^{(t)} = \begin{pmatrix} v_x \\ v_y \end{pmatrix} = \frac{\mathbf{p}_m^{(t)} - \mathbf{p}_m^{(t-\tau_c)}}{\tau_c}$$



We can now iterate to estimate the pixel gradient based on EKF process.

Paper: Simultaneous Mosaicing and Tracking with an Event Camera

▪ Mapping - Reconstruction from gradients

Since we already obtained the gradients, how to reconstruct the intensity?

By optimization! Design the cost function as:

$$J(M) = \int \int (M_x - g_x)^2 + (M_y - g_y)^2 dx dy .$$

where M_x, M_y are **true gradients** in x and y directions (we don't have);

g_x, g_y are **estimated gradients** in x and y directions (we have);

M denotes the intensity that we aim to solve.

Minimizing $J(M)$ leads to the well known Poisson equation:

$$\nabla^2 M = \frac{\partial}{\partial x} g_x + \frac{\partial}{\partial y} g_y .$$

where $\nabla^2 M = \frac{\partial^2 M}{\partial x^2} + \frac{\partial^2 M}{\partial y^2}$ is the Laplacian.

Fortunately, many methods are available to solve the above Poisson equation!

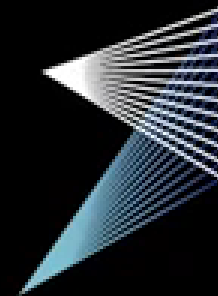
Real-time Visual-Inertial Odometry for Event Cameras using Keyframe-based Nonlinear Optimization

Henri Rebecq, Timo Horstschafer, Davide Scaramuzza



University of
Zurich^{UZH}

Department of Informatics



ROBOTICS &
PERCEPTION
GROUP

rpg.ifi.uzh.ch

Recognition

- Tasks (from easy to hard)
 - Detecting the presence of a simple shape (such as a circle)
 - Classification of more complex shapes, such as card pips, block letters and faces
 - More difficult objects: similar to conventional Caltech-101 and Caltech-256 datasets
 - Gestures and Motions

Recording Neuromorphic-Caltech101

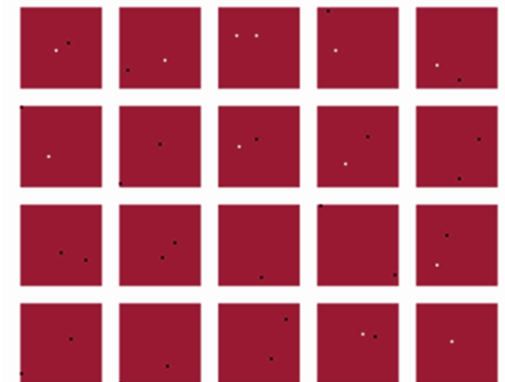
camera
shaking



Card symbols recognition



Event-based MNIST datasets

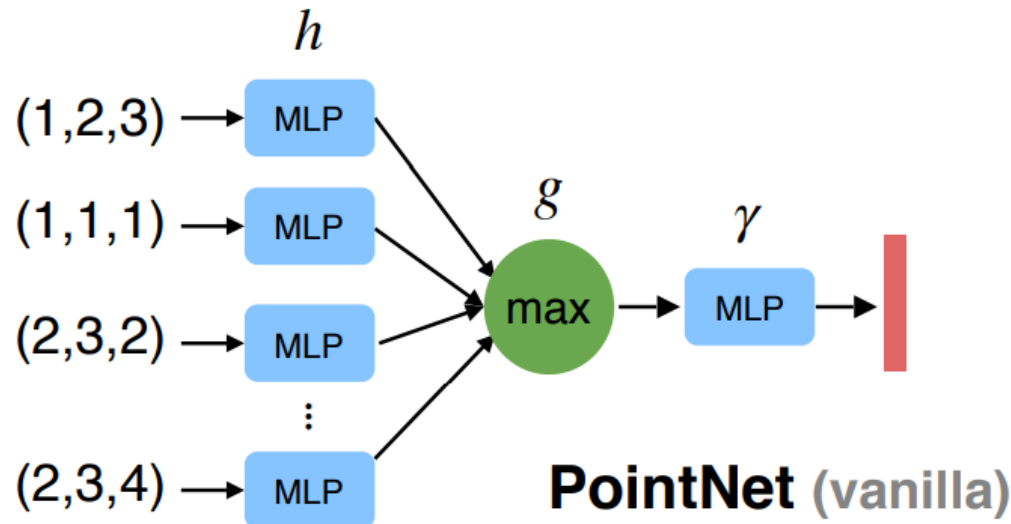


[Datasets | Neuromorphic Engineering \(greg-cohen.com\)](http://Datasets.NeuromorphicEngineering(greg-cohen.com))
www2.imse-cnm.csic.es/caviar/POKERDVS.html
Garrick Orchard - N-Caltech101

Recognition

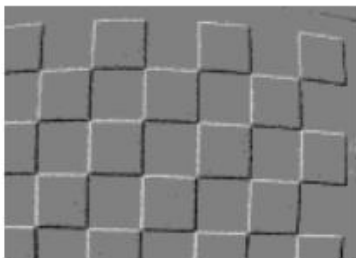
- Ideas
 - **Event representation:** event frames, point clouds, ...
 - **Recognition algorithm:** mostly using neural networks, including Convolutional Neural Network (CNN), Multi-layer Perceptron (MLP), and Spiking Neural Network (SNN)

The Pointnet (which is based on MLP), can be used to process event data.

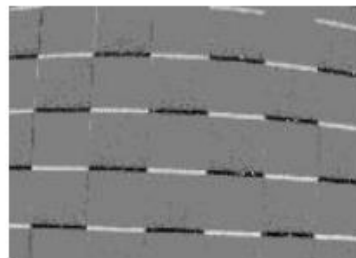


Recognition

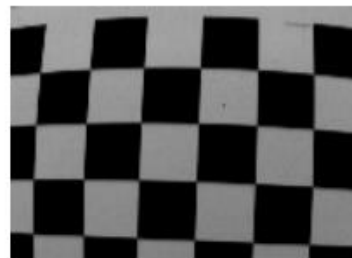
- Advantages
 - Recognizing fast-moving objects / in HDR scene
 - Sparse data: may lead to less computational complexity / power consumption
 - Good at capturing motion within a time interval, instead of a timestamp
- Challenges
 - Still yet to find the most appropriate application scenarios
 - Inadequate dataset, compared to frame-base recognition
 - Requires relative motion to generate data, and data pattern depends on the motion



(a)



(b)

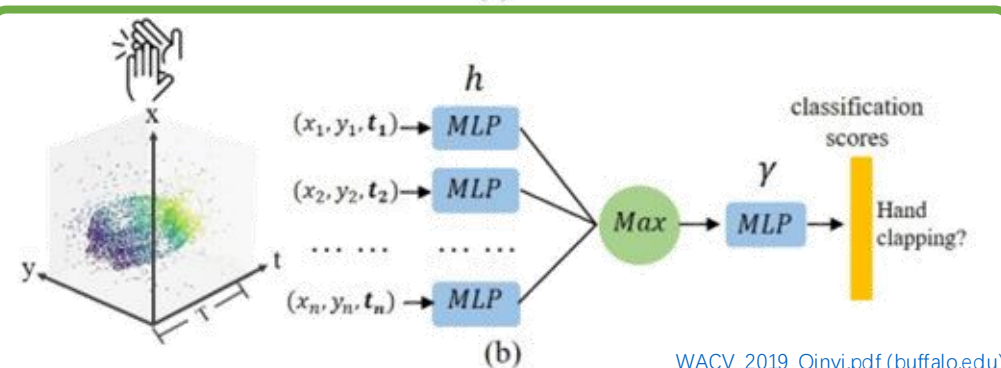
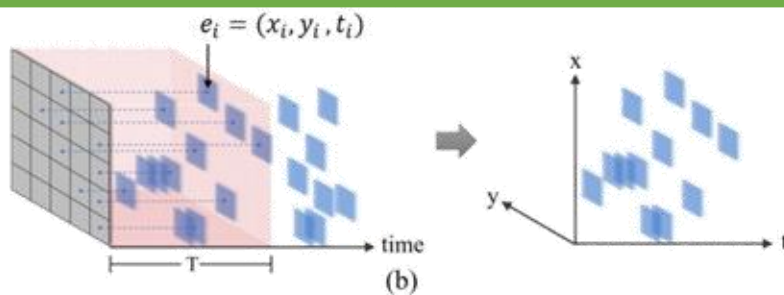
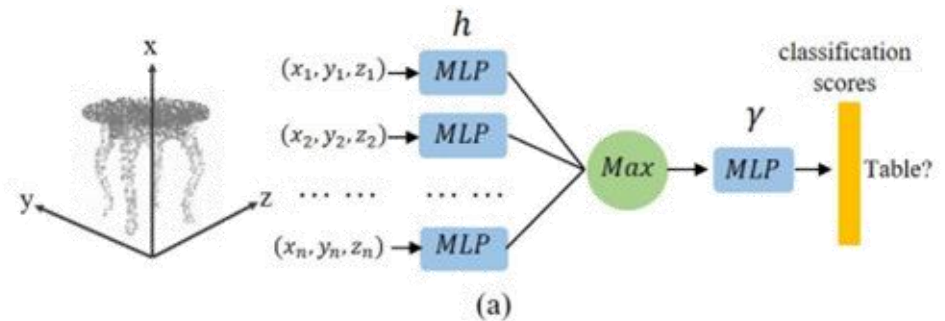
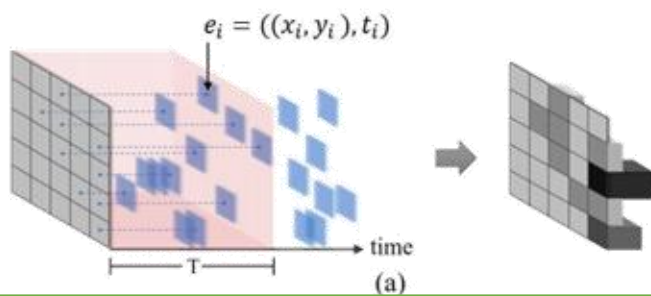


(c)

Different relative motions cause different event patterns, which brings difficulties in recognition!

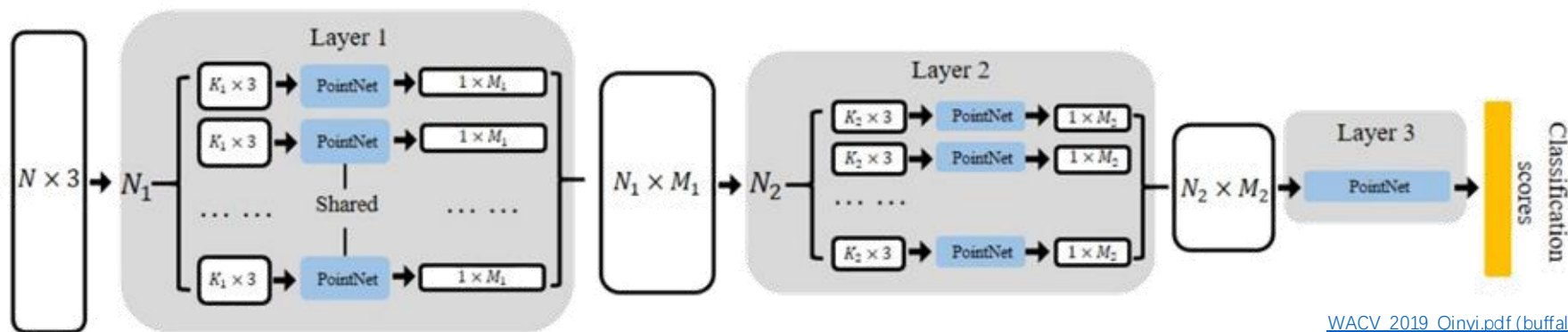
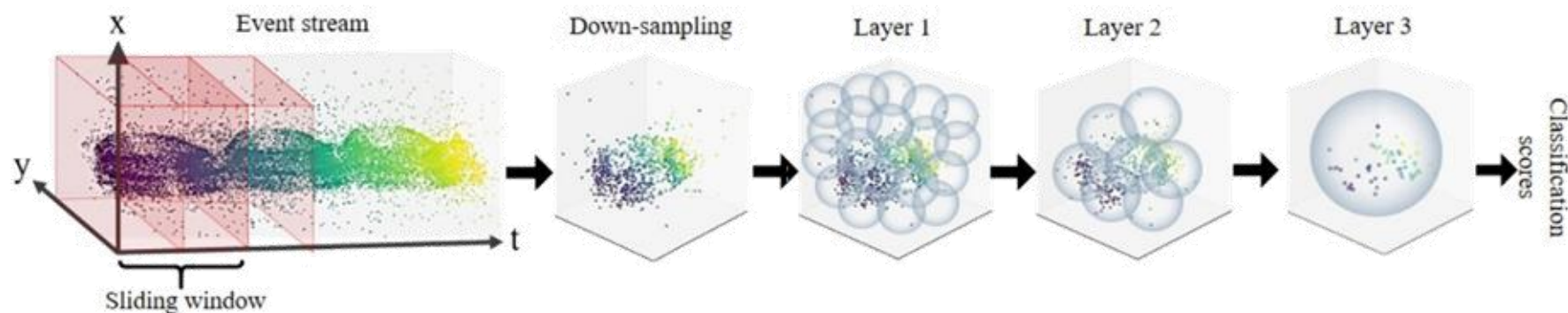
Paper: Space-time Event Clouds for Gesture Recognition

- Left: the authors treat events as space-time event clouds in this paper.
- Right: the PointNet framework can be adapted to consume space-time event cloud for gesture recognition.



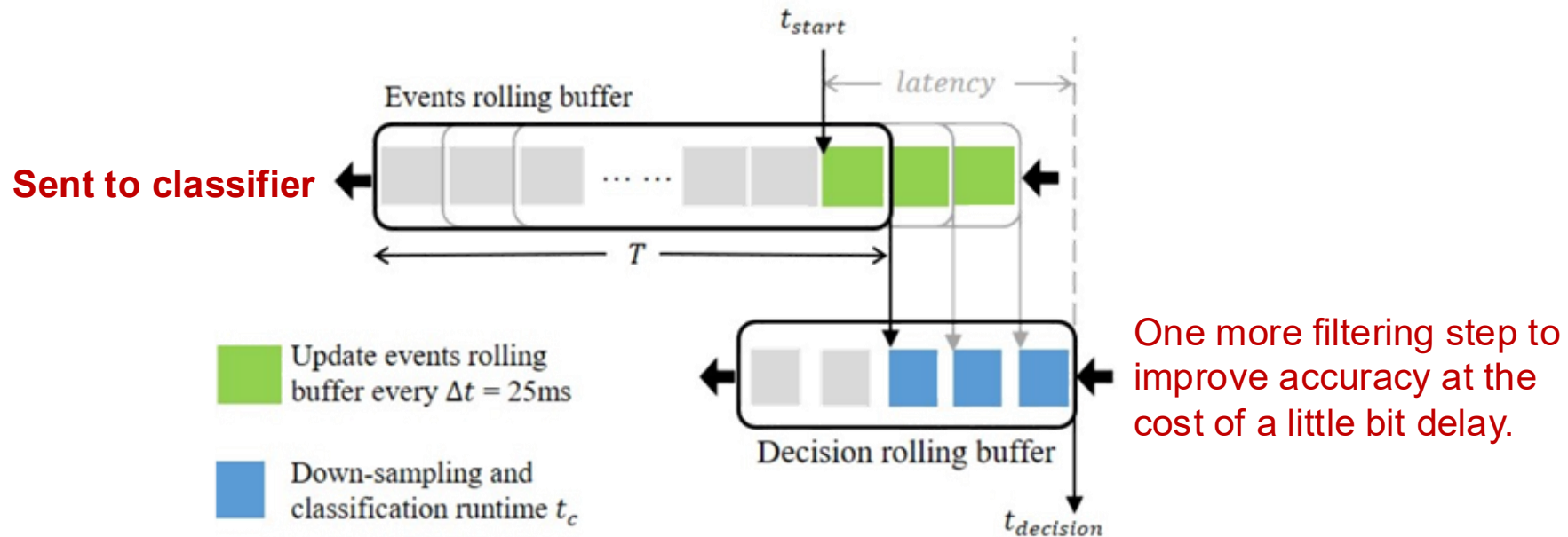
Paper: Space-time Event Clouds for Gesture Recognition

- The gesture classification framework
 - Trained based on PointNet++, which applies PointNet recursively on a nested partitioning of the input set.
 - Fine-to-coarse, to extract local and global features!



Paper: Space-time Event Clouds for Gesture Recognition

- The rolling buffer framework for real-time recognition
 - **Events rolling buffer** to store the event point clouds.
 - **Decision rolling buffer** to store the classification results.



Paper: Space-time Event Clouds for Gesture Recognition

- Results
 - Tested on IBM DVS128 Gesture dataset
 - Results for recognizing 10 classes >95%
 - Small data scale and efficient computation, compared to CNN-based approach!

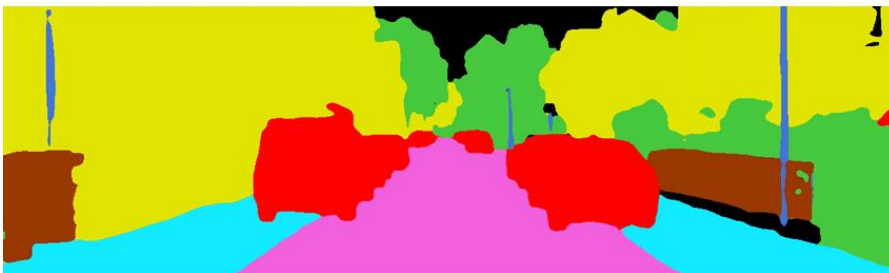
Experiment (model,classes,events)	Sliding window		
	$T=0.25s$	$T=0.50s$	$T=1.00s$
PointNet,10,256	87.85	89.63	88.54
PointNet,10,512	88.67	90.20	89.61
PointNet,10,1024	88.77	89.68	89.92
PointNet++,10,256	95.28	95.59	95.54
PointNet++,10,512	95.39	96.34	95.61
PointNet++,10,1024	94.93	95.89	95.97
PointNet++,11,256	91.92	93.38	93.61
PointNet++,11,512	92.23	94.10	93.83
PointNet++,11,1024	91.87	91.91	92.63

Why $T=0.50s$ results are the best?

Why 1024-points results worse than 512-point results?

Motion Segmentation

- **Image segmentation** is
 - the process of partitioning a digital image into multiple segments.
 - the process of assigning a label to every pixel in an image such that pixels with the same label share certain characteristics.



 Road	 Sidewalk	 Building	 Fence
 Pole	 Vegetation	 Vehicle	 Unlabel



(4) (PDF) Towards a Meaningful 3D Map Using a 3D Lidar and a Camera ([researchgate.net](https://www.researchgate.net/publication/351111111))

Deploying robotics applications — Using image segmentation to drive the Turtlebot3 Burger - Qualcomm Developer Network

Motion Segmentation

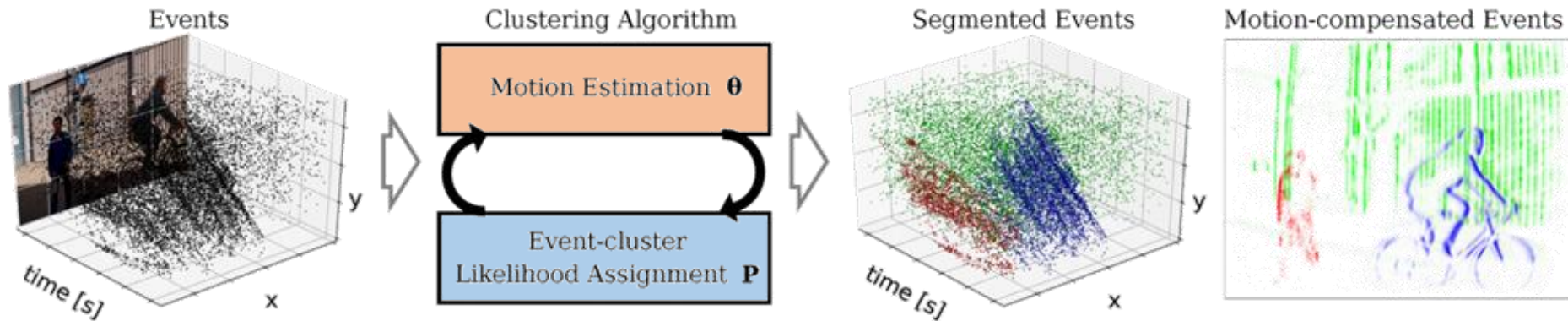
- Image segmentation with an event camera
 - Idea 1: **fuse events and frames**: combine motion patterns and texture/geometry patterns
 - Idea 2: use **events only**: segmenting objects based on motion patterns
→ “**motion segmentation**”

Paper: Event-Based Motion Segmentation by Motion Compensation

- Framework of motion segmentation

Idea: maximizing event alignment, i.e., maximizing the sharpness of motion compensated images (one per cluster) of warped events.

Cluster	C1	C2
motion (u, v)	(-400, 2)	(500, -5)



Event e	$P(e \in C1)$	$P(e \in C2)$
(128,40,+1)	0.75	0.25
(20,600,-,2)	0.30	0.70
(128,41,+3)	0.80	0.20
...	...	...

Paper:

Event-Based Motion Segmentation by Motion Compensation

- Mathematical formulation
 - **Soft event-cluster associations:**

$$p_{kj} = P(e_k \in \ell_j)$$

is the probability of the k -th event belonging to the j -th cluster.

Let $\mathbf{P} \equiv (p_{kj})$ be an $N_e \times N_\ell$ matrix with all event-cluster associations. Define the weighted IWE (Image of Warped Events) of the j -th cluster as

$$I_j(\mathbf{x}) \doteq \sum_{k=1}^{N_e} p_{kj} \delta(\mathbf{x} - \mathbf{x}'_{kj}),$$

with $\mathbf{x}'_{kj} = \mathbf{W}(\mathbf{x}_k, t_k; \boldsymbol{\theta}_j)$ the warped event location, and δ the Dirac delta. Above equation states that events are warped:

$$e_k \doteq (\mathbf{x}_k, t_k, s_k) \mapsto e'_k \doteq (\mathbf{x}'_k, t_{\text{ref}}, s_k)$$

and probabilities are accumulated at the warped locations.

Paper: Event-Based Motion Segmentation by Motion Compensation

We want to maximize the sharpness, but how to define the sharpness?

- Define the sharpness as the variance of the image:

$$\text{Var}(I_j) \doteq \frac{1}{|\Omega|} \int_{\Omega} (I_j(\mathbf{x}) - \mu_{I_j})^2 d\mathbf{x}$$

where μ_{I_j} is the mean of IWE over the image plane Ω .

Problem: to find the associations $\mathbf{P}$ and cluster parameters $\boldsymbol{\theta}$ that maximize the sum of contrasts of all clusters:

$$(\boldsymbol{\theta}^*, \mathbf{P}^*) = \arg \max_{(\boldsymbol{\theta}, \mathbf{P})} \sum_{j=1}^{N_{\ell}} \text{Var}(I_j).$$

No closed-form solution $\rightarrow$ Only iterative, alternating optimization!

Paper: Event-Based Motion Segmentation by Motion Compensation

- Alternating Optimization
 - Each iteration contains 2 steps

The proposed alternating method **converges locally** (i.e., there is no guarantee of convergence to a global solution).

Algorithm 1 Event-based Motion Segmentation

- 1: **Input:** events $\mathcal{E} = \{e_k\}_{k=1}^{N_e}$ in a space-time volume V of the image plane, and number of clusters N_ℓ .
 - 2: **Output:** cluster parameters $\boldsymbol{\theta} = \{\boldsymbol{\theta}_j\}_{j=1}^{N_\ell}$ and event-cluster assignments $\mathbf{P} \equiv p_{kj} \doteq P(e_k \in \ell_j)$.
 - 3: **Procedure:**
 - 4: Initialize the unknowns $(\boldsymbol{\theta}, \mathbf{P})$ (see Section 3.5).
 - 5: **Iterate** until convergence:
 - 6: • Compute the event-cluster assignments p_{kj} using (7).
 - 7: • Update the motion parameters of all clusters (6).
-

If associations $\mathbf{P}$ are fixed, update the motion parameters using

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \mu \nabla_{\boldsymbol{\theta}} \left(\sum_{j=1}^{N_\ell} \text{Var}(I_j) \right) \quad (6)$$

“gradient ascent”

Fix motion parameters $\boldsymbol{\theta}$, refine the associations based on probability partitioning law:

$$p_{kj} = \frac{c_j(\mathbf{x}'_k(\boldsymbol{\theta}_j))}{\sum_{i=1}^{N_\ell} c_i(\mathbf{x}'_k(\boldsymbol{\theta}_i))} \quad (7)$$

where $c_j(\mathbf{x}) \doteq I_j(\mathbf{x})$.

Paper: Event-Based Motion Segmentation by Motion Compensation

- Initialization
 - Requires initialization of θ , $\mathbf{P}$ to start the iteration.
 - Several initialization schemes are possible, depending on the motion models.

1. K-means initialization for **optical flow-type** motion;

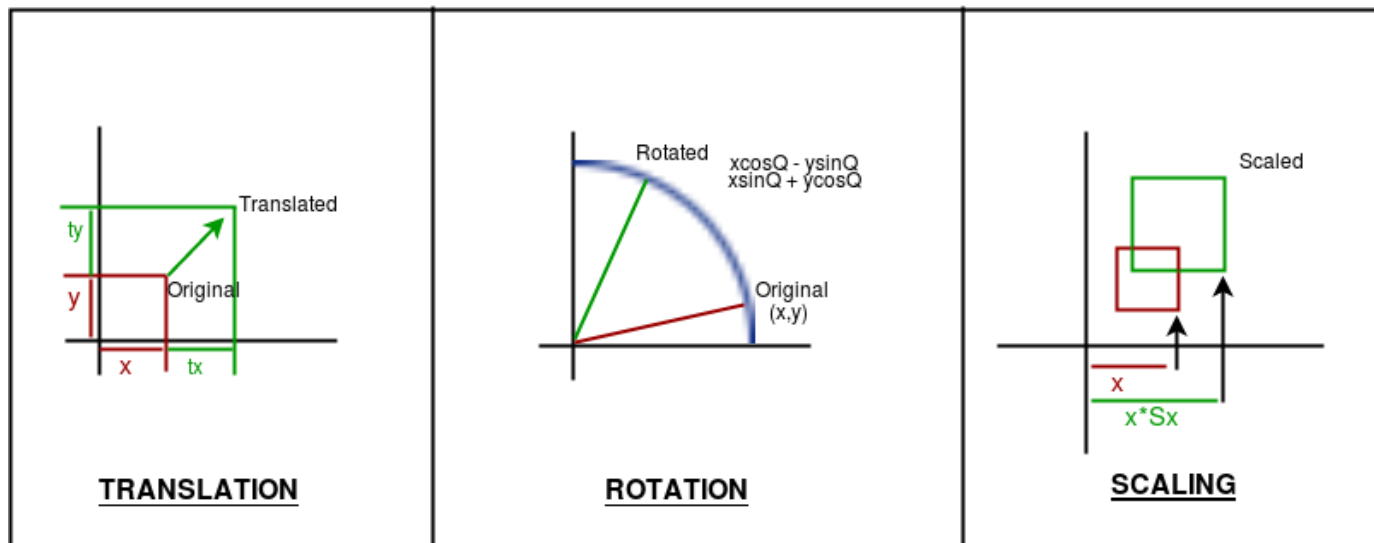


2. Greedy initialization for **all kinds of motion**.

No details/codes released from the authors.

Paper: Event-Based Motion Segmentation by Motion Compensation

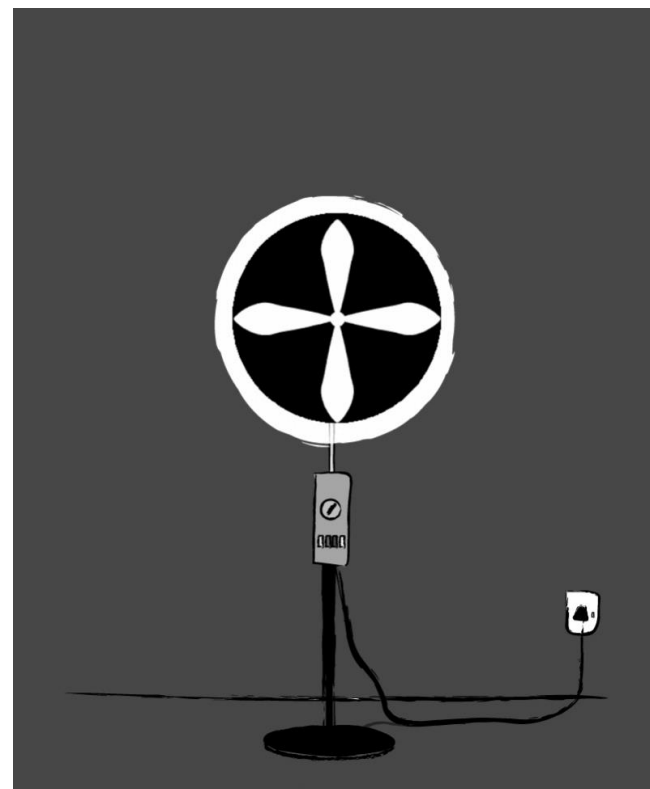
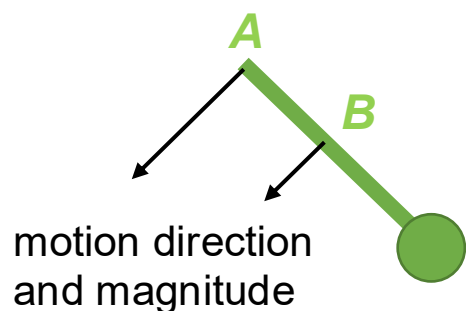
- Discussion on **motion model** $\mathbf{x}'_{kj} = \mathbf{W}(\mathbf{x}_k, t_k; \boldsymbol{\theta}_j)$
$$e_k \doteq (\mathbf{x}_k, t_k, s_k) \mapsto e'_k \doteq (\mathbf{x}'_k, t_{\text{ref}}, s_k)$$
 - Optical flow type motion (translation only)
 - Other types of motion (both translation and rotation)



An appropriate motion model helps to **reduce the cluster number!**

Paper: Event-Based Motion Segmentation by Motion Compensation

- Discussion on **motion model**
Consider the rotating fan scenario.
What motion model should be used?
- Do points A and B share the same motion parameter, if using an optical flow type model?
- What if using a rotation model instead?



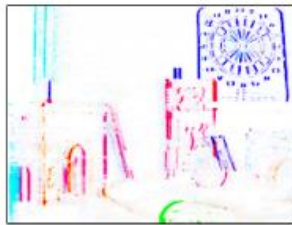
All points on the edge of blade have **different motion parameters**, if using an flow type model.
However, they do share the **same motion parameters**, if using a rotation model!

Paper: Event-Based Motion Segmentation by Motion Compensation

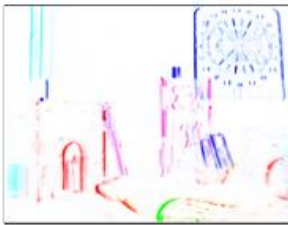
- Discussion on **limit of object number**
 - Algo requires to pre-define number of clusters N_l ;
 - It is noted that N_l is not a particularly important parameter;
 - If N_l too large, unnecessary clusters end up not having any events allocated to them and thus “die”;
 - N_l can simply be chosen to be large.



(a) Scene



(b) $N_l = 5 \times \text{Optical Flow}$



(c) $N_l = 10 \times \text{Optical Flow}$



(d) $N_l = 20 \times \text{Optical Flow}$



(e) $N_l = 5 \times \text{OF} + 5 \times \text{Rotation}$



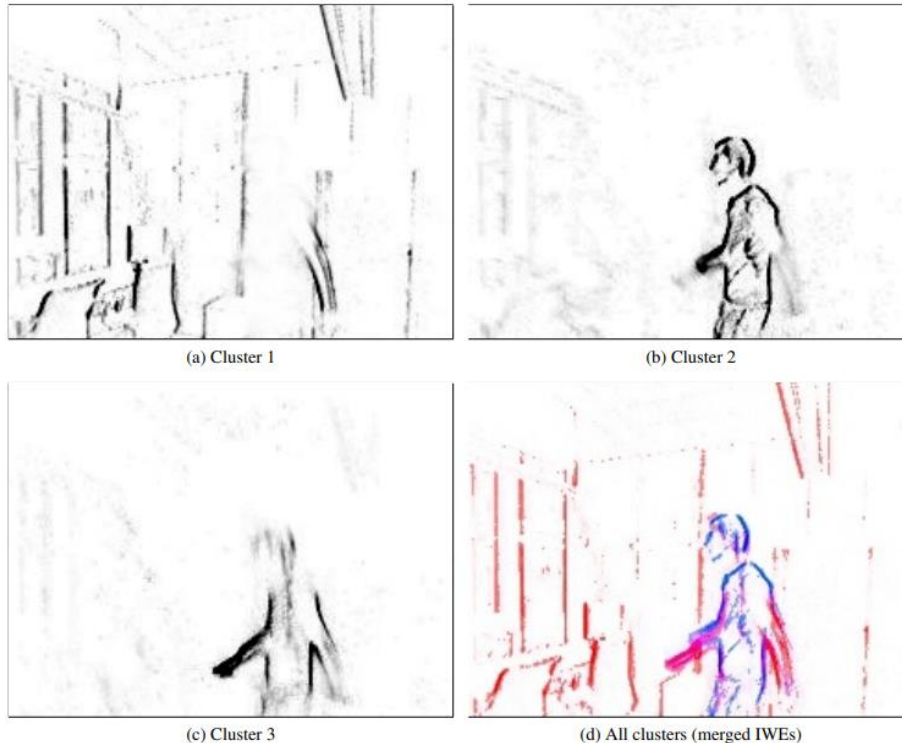
(f) $N_l = 10 \times \text{Rotation}$

Experiment on *slider depth* sequence. Motion-compensated images in (b) to (f) show events colored by cluster.

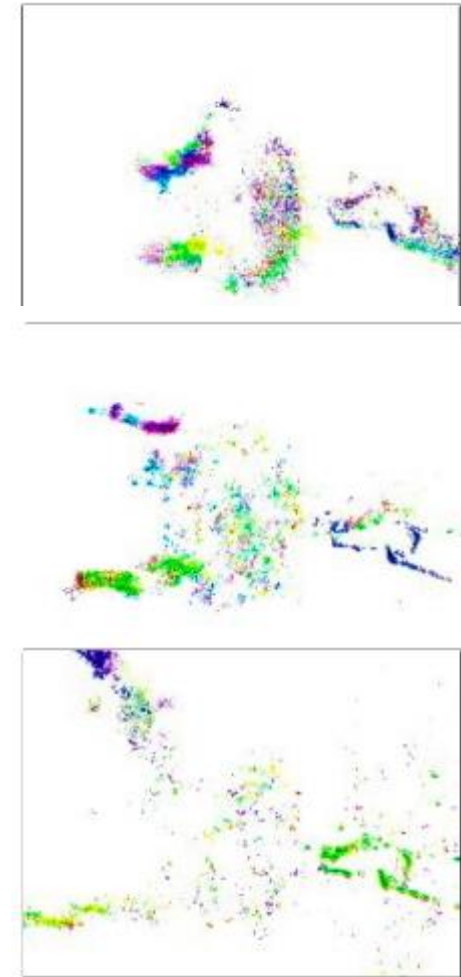
The real motion for all clusters should be in “Optical Flow” type.

Paper: Event-Based Motion Segmentation by Motion Compensation

- Discussion on **non-rigid objects' motion**



A person walks across a room, arms swinging.



Snapshots of segmentation of balloon popping.
Num. of clusters = 4,
Events colored by cluster membership.

Event-Based Motion Segmentation by Motion Compensation

Timo Stoffregen, Guillermo Gallego, Tom Drummond, Lindsay Kleeman,
Davide Scaramuzza



University of
Zurich^{UZH}

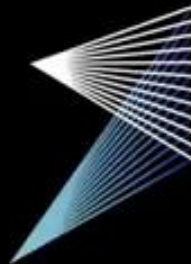
Department of Neuroinformatics

ETH zürich



University of
Zurich^{UZH}

Department of Informatics



ROBOTICS &
PERCEPTION
GROUP

rpg.ifi.uzh.ch



MONASH
University

Summary – Motion segmentation

- Image segmentation and motion segmentation
- Problem formulation
- Ideas
 - Events + frame fusion-based
 - Pure event-based
- Selected papers
 - Motion segmentation towards **contrast maximization**

Paper: Back to Event Basics: Self-Supervised Learning of Image Reconstruction for Event Cameras via Photometric Constancy

- One issue for learning-based reconstruction methods: **lack of data**.
Possible reasons:
 - The **cost** of event sensor is high;
 - The **data format** of event sensor is not unified.
 - ...?
- How to the reduce the dependence of data in learning-based methods?
 - Self-supervised learning/unsupervised learning.
e.g., EV-FlowNet for optical flow estimation:
No ground-truth optical flow is used in training; Ground-truth CIS images are required.

Can we train a network using events only?

Paper: Back to Event Basics: Self-Supervised Learning of Image Reconstruction for Event Cameras via Photometric Constancy



Back to Event Basics: Self-Supervised Learning of Image Reconstruction for Event Cameras via Photometric Constancy

Federico Paredes-Valles and Guido C. H. E. de Croon
(Poster Session Three, ID: 8305)

